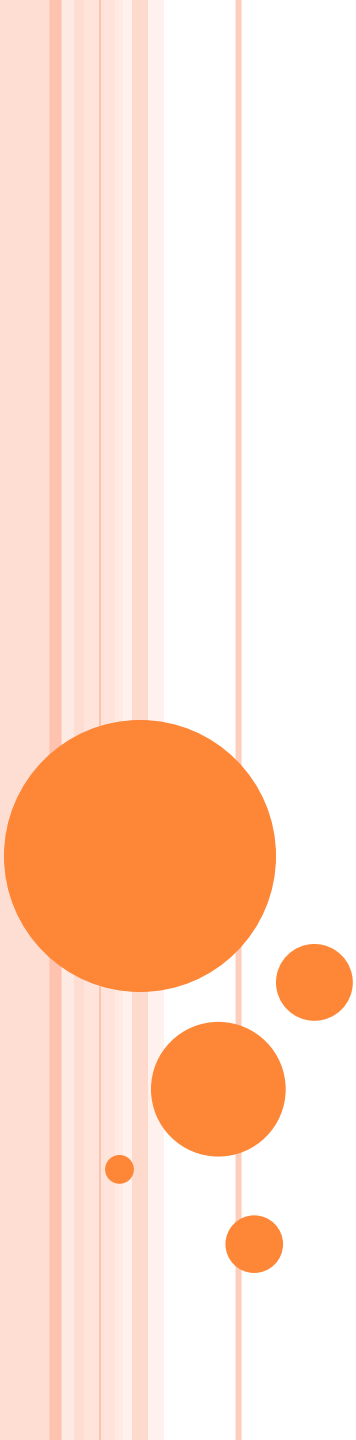




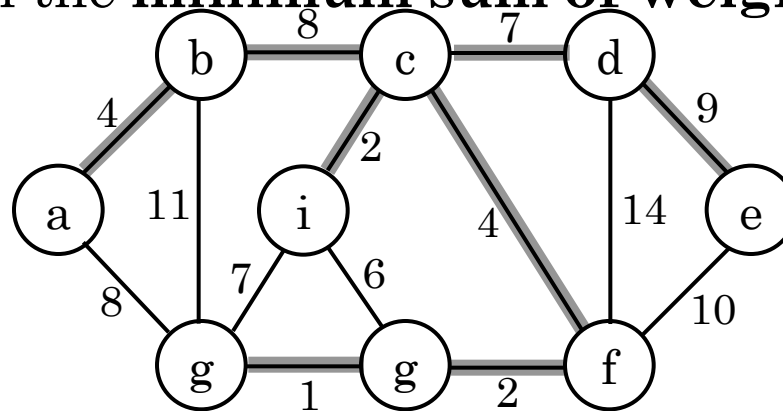
GRAPH THEORY

MINIMUM SPANNING TREE

Design and Analysis of Algorithms

MINIMUM SPANNING TREES

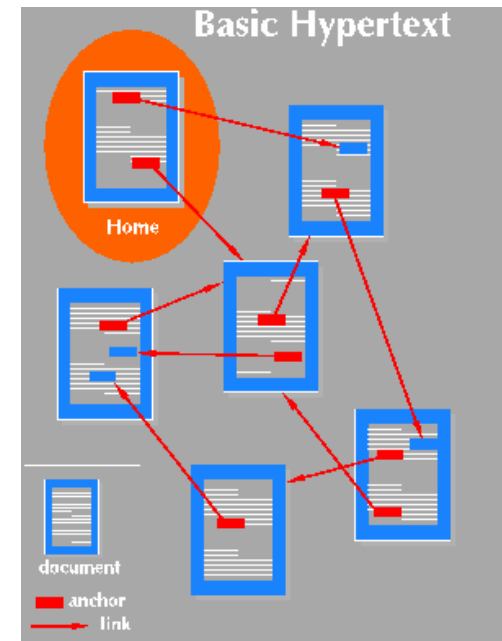
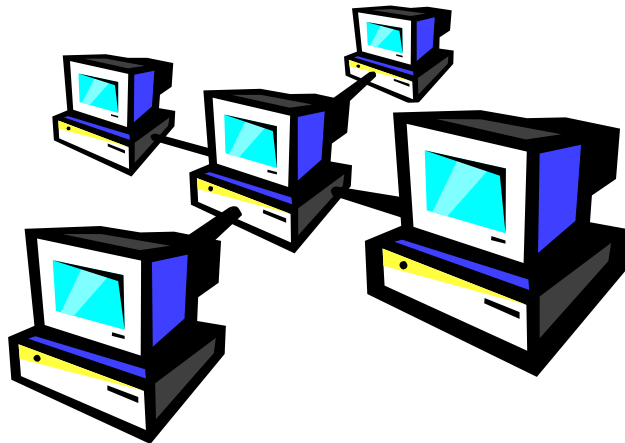
- Spanning Tree
 - A tree (i.e., connected, acyclic graph) which contains all the vertices of the graph
- Minimum Spanning Tree
 - Spanning tree with the **minimum sum of weights**



- Spanning forest
 - If a graph is not connected, then there is a spanning tree for each connected component of the graph

APPLICATIONS OF MST

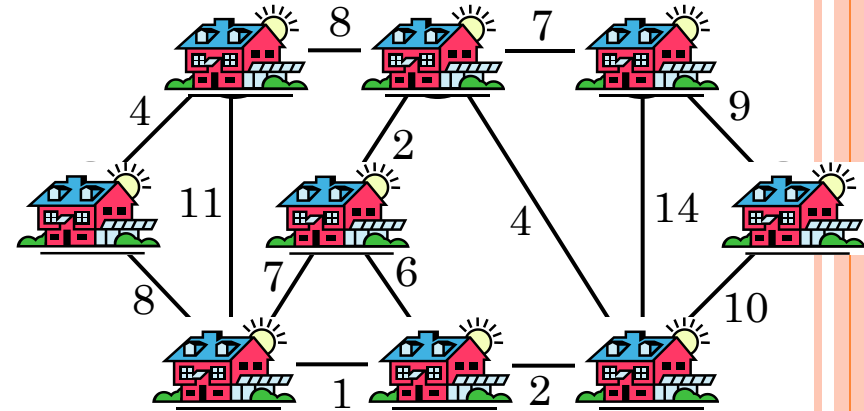
- Find the least expensive way to connect a set of cities, terminals, computers, etc.



EXAMPLE

Problem

- A town has a set of houses and a set of roads
- A road connects 2 and only 2 houses
- A road connecting houses u and v has a repair cost $w(u, v)$



Goal: Repair enough (and no more) roads such that:

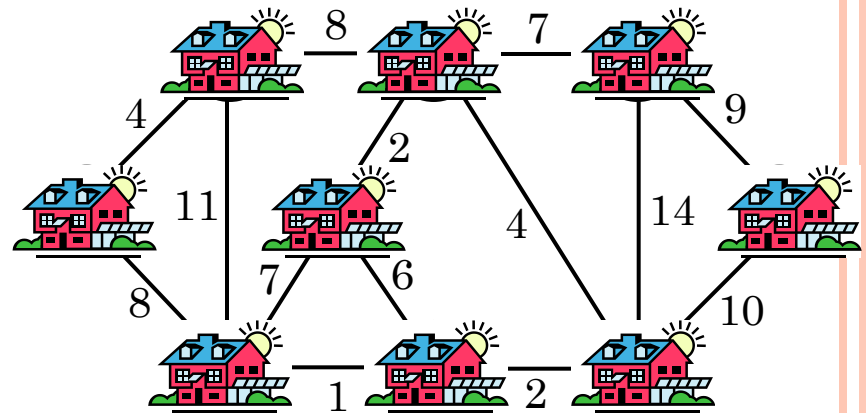
1. Everyone stays connected
i.e., can reach every house from all other houses
2. Total repair cost is minimum

MINIMUM SPANNING TREES

- A connected, undirected graph:
 - Vertices = houses, Edges = roads
- A **weight** $w(u, v)$ on each edge $(u, v) \in E$

Find $T \subseteq E$ such that:

1. T connects all vertices
2. $w(T) = \sum_{(u,v) \in T} w(u, v)$ is minimized



PROPERTIES OF MINIMUM SPANNING TREES

- Minimum spanning tree is **not** unique



- MST has no cycles – see why:
 - We can take out an edge of a cycle, and still have the vertices connected while reducing the cost
- # of edges in a MST:
 - $|V| - 1$

MINIMUM SPANNING TREES

- A ***spanning tree*** of an undirected connected graph is its connected acyclic subgraph (i.e., a tree) that contains all the vertices of the graph.
- If such a graph has weights assigned to its edges, a ***minimum spanning tree*** is its spanning tree of the smallest weight, where the ***weight*** of a tree is defined as the sum of the
- weights on all its edges.
- The ***minimum spanning tree problem*** is the problem of finding a minimum spanning tree for a given weighted connected graph.

MINIMUM SPANNING TREES

Input: A connected, undirected graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$.

- For simplicity, assume that all edge weights are distinct. (CLRS covers the general case.)

MINIMUM SPANNING TREES

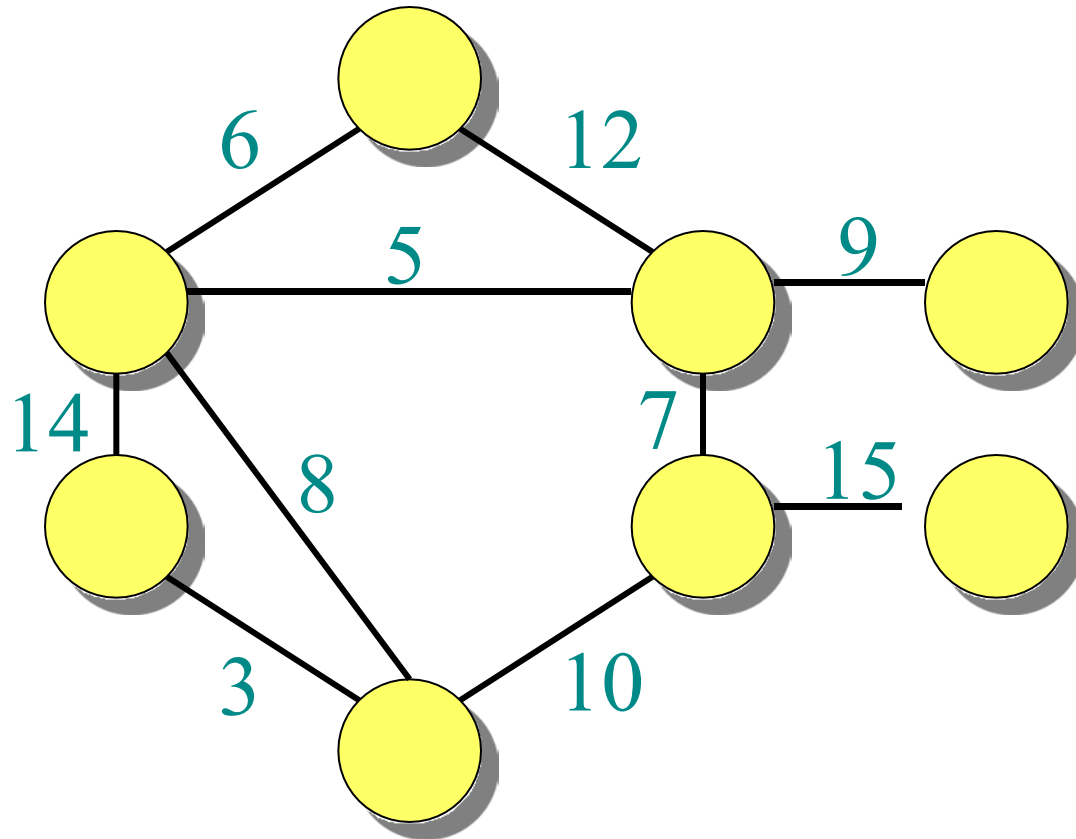
Input: A connected, undirected graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$.

- For simplicity, assume that all edge weights are distinct. (CLRS covers the general case.)

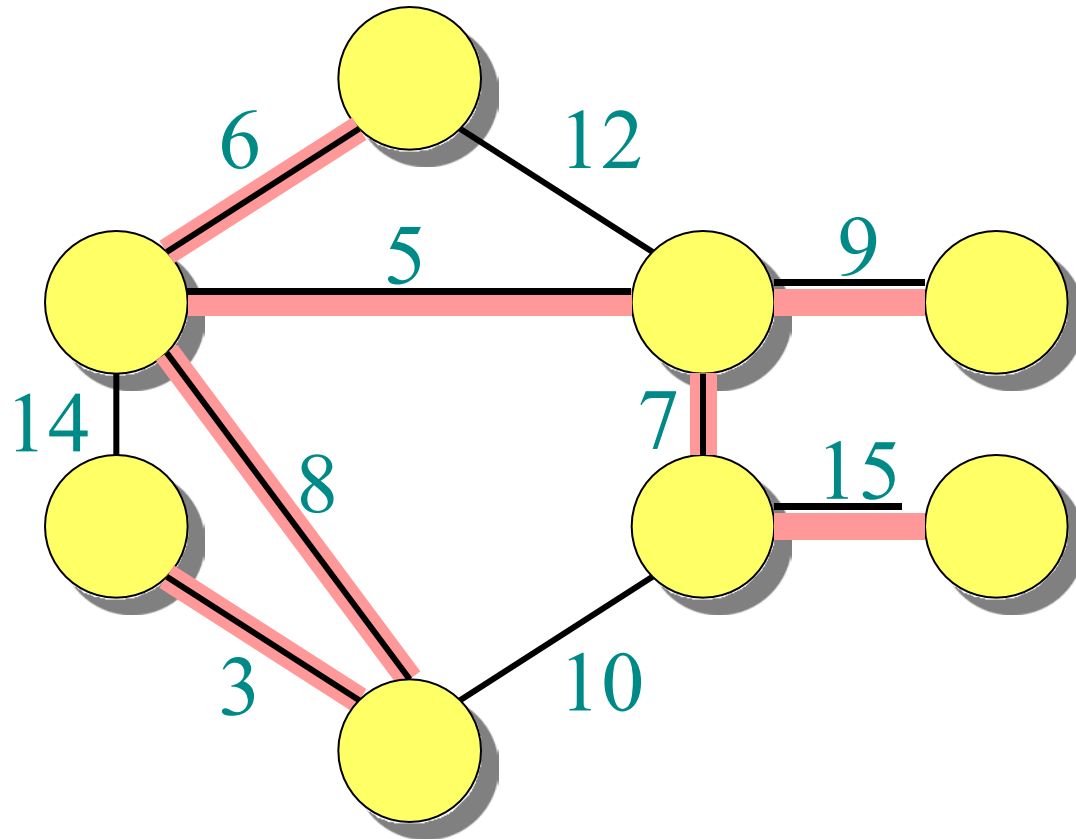
Output: A *spanning tree* T — a tree that connects all vertices — of minimum weight:

$$w(T) = \sum_{(u,v) \in T} w(u,v).$$

EXAMPLE OF MST



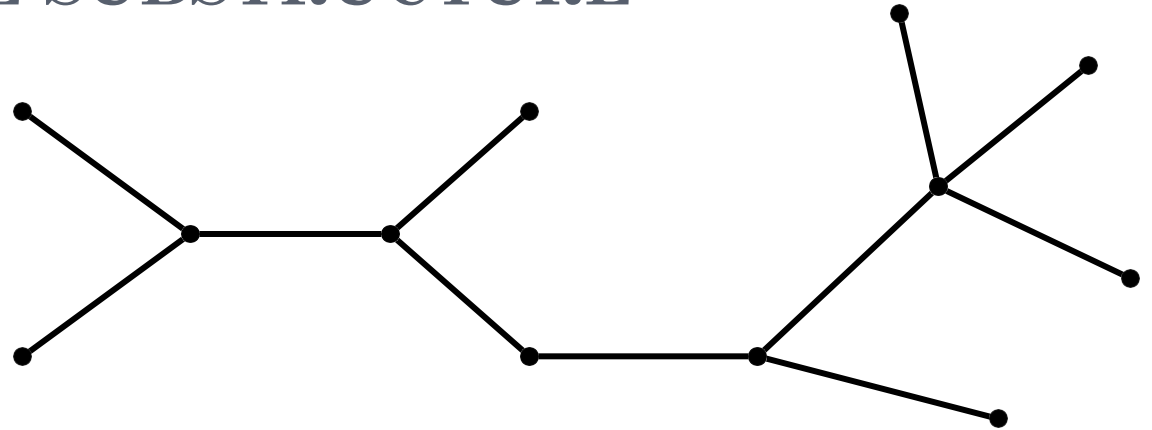
EXAMPLE OF MST



OPTIMAL SUBSTRUCTURE

MST T :

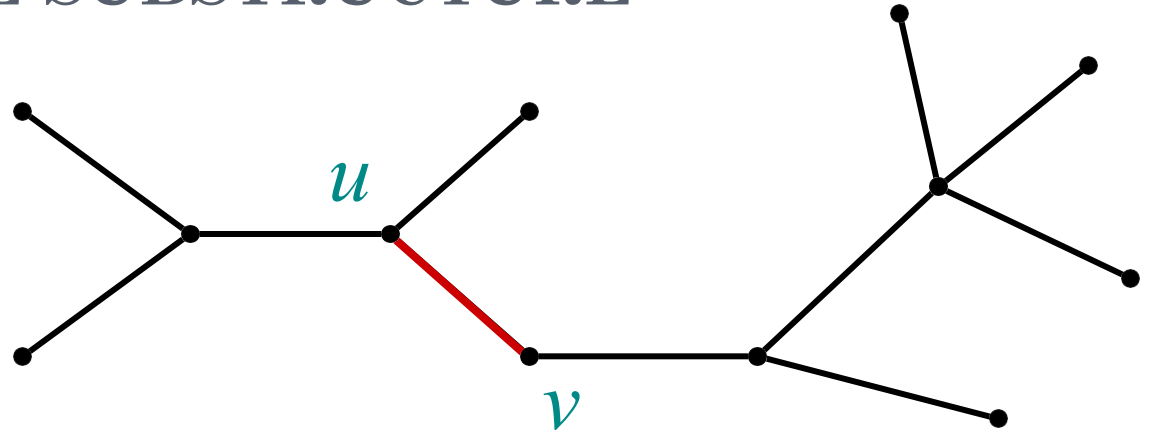
(Other edges of G
are not shown.)



OPTIMAL SUBSTRUCTURE

MST T :

(Other edges of G
are not shown.)

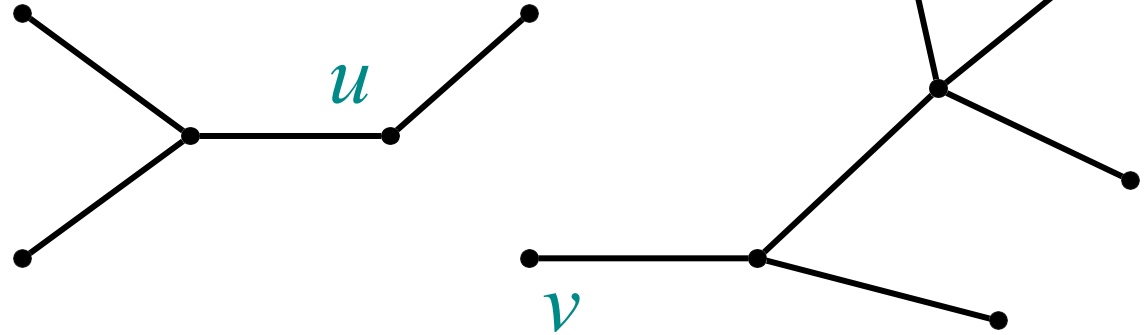


Remove any edge $(u, v) \in T$.

OPTIMAL SUBSTRUCTURE

MST T :

(Other edges of G
are not shown.)

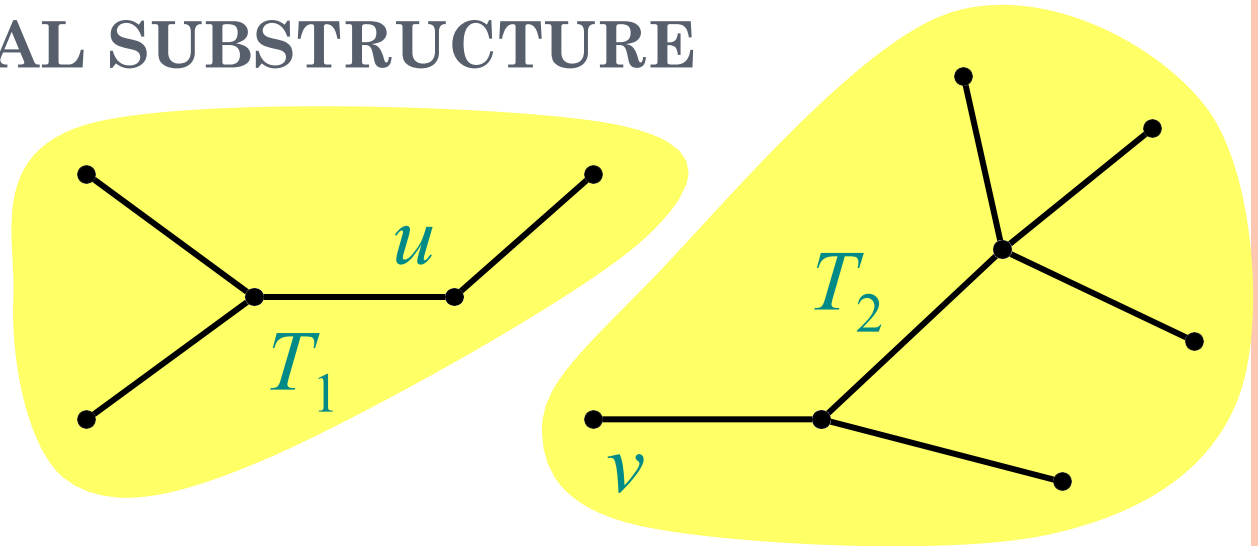


Remove any edge $(u, v) \in T$.

OPTIMAL SUBSTRUCTURE

MST T :

(Other edges of G
are not shown.)



Remove any edge $(u, v) \in T$. Then, T is partitioned into two subtrees T_1 and T_2 .

HALLMARK FOR “GREEDY” ALGORITHMS

Greedy-choice property

*A locally optimal choice
is globally optimal.*

GENERIC-MST(G, w)

```
1   $A = \emptyset$ 
2  while  $A$  does not form a spanning tree
3      find an edge  $(u, v)$  that is safe for  $A$ 
4       $A = A \cup \{(u, v)\}$ 
5  return  $A$ 
```

- The generic method manages a set of edges A , maintaining the following loop invariant:
- **Prior to each iteration, A is a subset of some minimum spanning tree.**
- At each step, we determine an edge $\{u, v\}$ that we can add to A without violating this invariant, in the sense that $A \cup \{u, v\}$ is also a subset of a minimum spanning tree.
- We call such an edge a ***safe edge*** for A , since we can add it safely to A while maintaining the invariant

HALLMARK FOR “GREEDY” ALGORITHMS

Greedy-choice property
*A locally optimal choice
is globally optimal.*

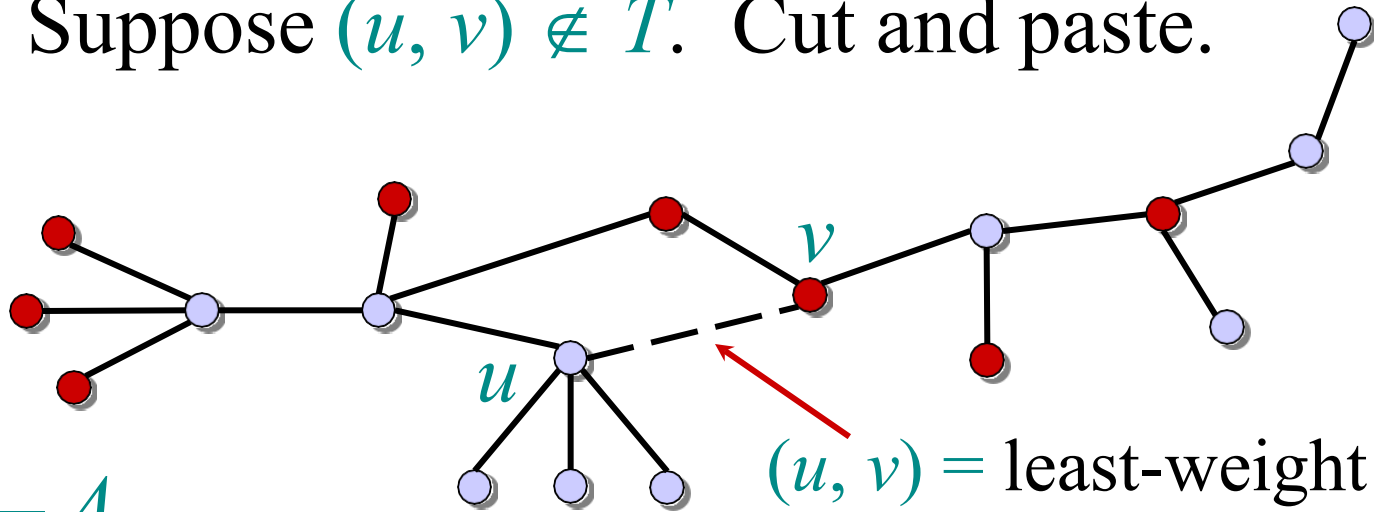
Theorem. Let T be the MST of $G = (V, E)$, and let $A \subseteq V$. Suppose that $(u, v) \in E$ is the least-weight edge connecting A to $V - A$. Then, $(u, v) \in T$.

PROOF OF THEOREM

Proof. Suppose $(u, v) \notin T$. Cut and paste.

T :

$\bullet \in A$
 $\bullet \in V - A$

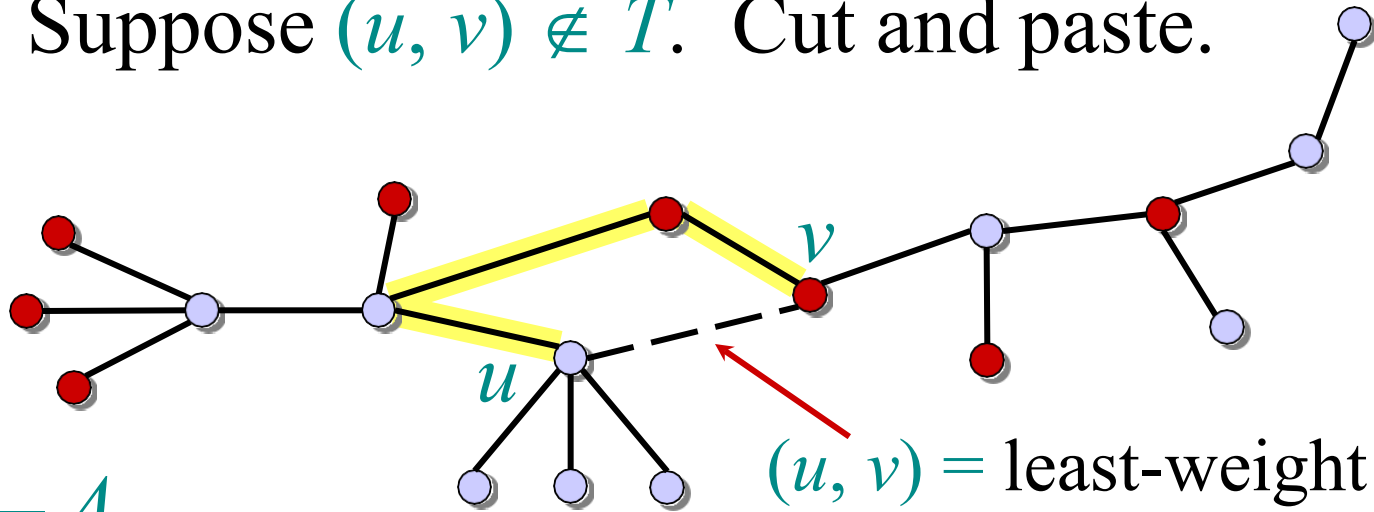


PROOF OF THEOREM

Proof. Suppose $(u, v) \notin T$. Cut and paste.

T :

$\bullet \in A$
 $\bullet \in V - A$



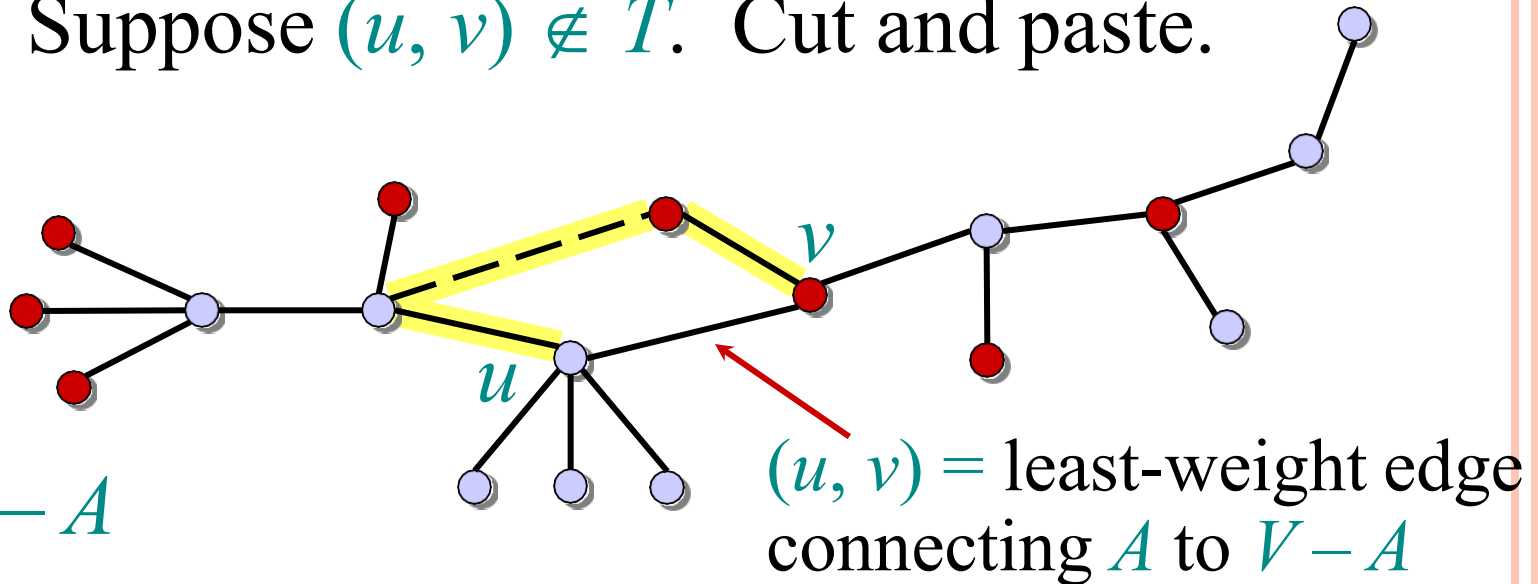
Consider the unique simple path from u to v in T .

PROOF OF THEOREM

Proof. Suppose $(u, v) \notin T$. Cut and paste.

T :

$\bullet \in A$
 $\bullet \in V - A$

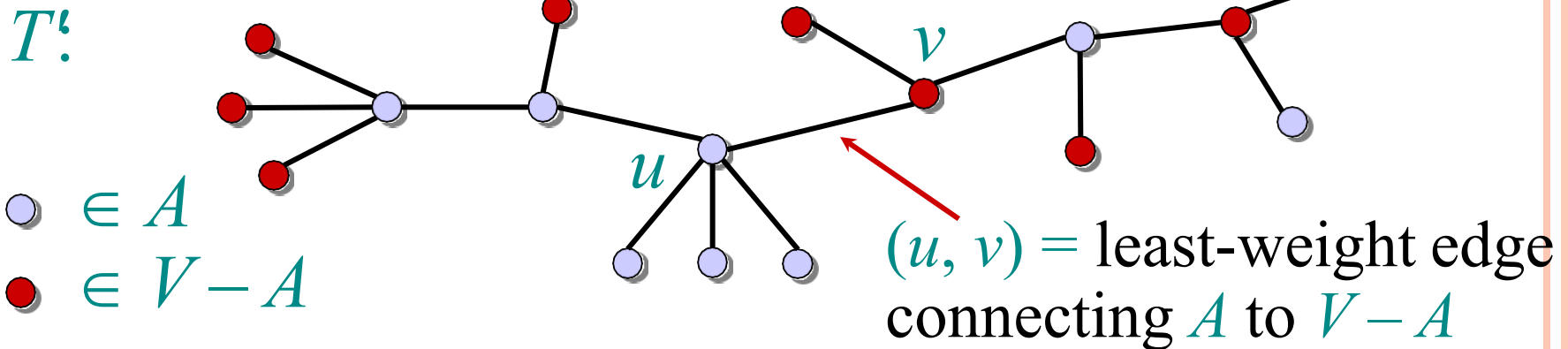


Consider the unique simple path from u to v in T .

Swap (u, v) with the first edge on this path that connects a vertex in A to a vertex in $V - A$.

PROOF OF THEOREM

Proof. Suppose $(u, v) \notin T$. Cut and paste.



Consider the unique simple path from u to v in T .

Swap (u, v) with the first edge on this path that connects a vertex in A to a vertex in $V - A$.

A lighter-weight spanning tree than T results.

PRIM'S ALGORITHM

IDEA: Maintain $V - A$ as a priority queue Q . Key each vertex in Q with the weight of the least-weight edge connecting it to a vertex in A .

$Q \leftarrow V$

$key[v] \leftarrow \infty$ for all $v \in V$

$key[s] \leftarrow 0$ for some arbitrary $s \in V$

while $Q \neq \emptyset$

do $u \leftarrow \text{EXTRACT-MIN}(Q)$

for each $v \in \text{Adj}[u]$

do if $v \in Q$ and $w(u, v) < key[v]$

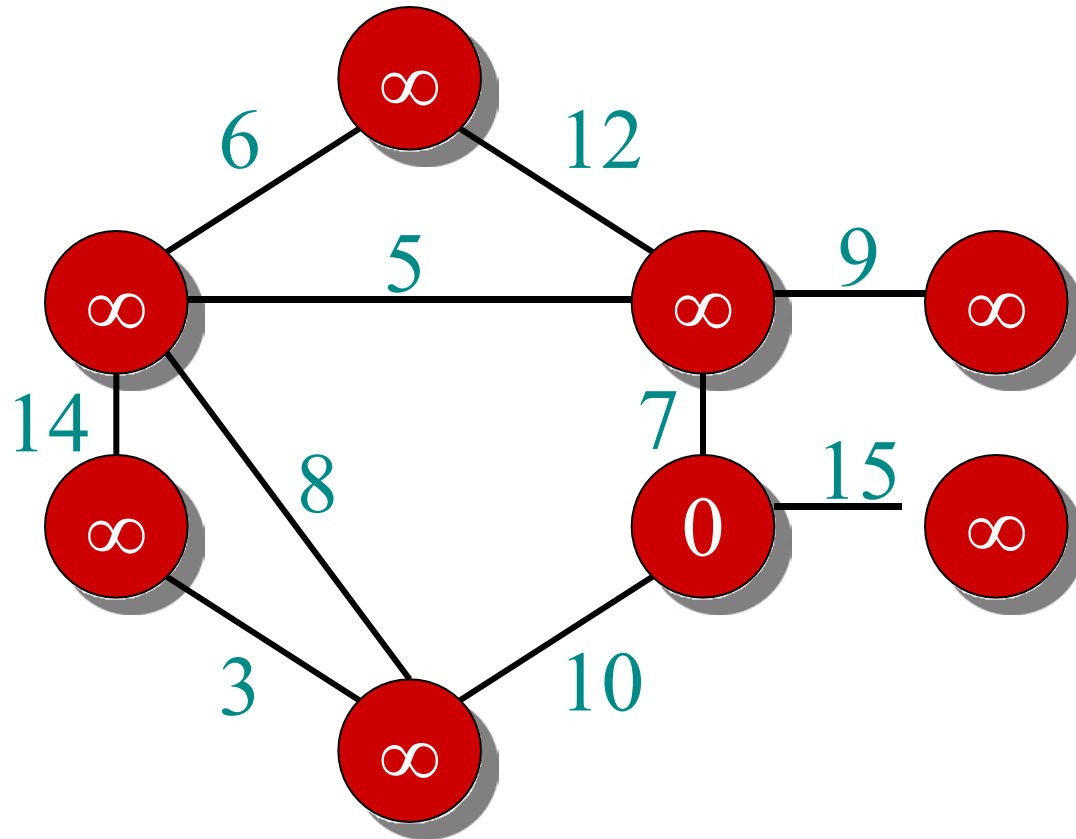
then $key[v] \leftarrow w(u, v)$ $\triangleright$ DECREASE-KEY

$\pi[v] \leftarrow u$

At the end, $\{(v, \pi[v])\}$ forms the MST.

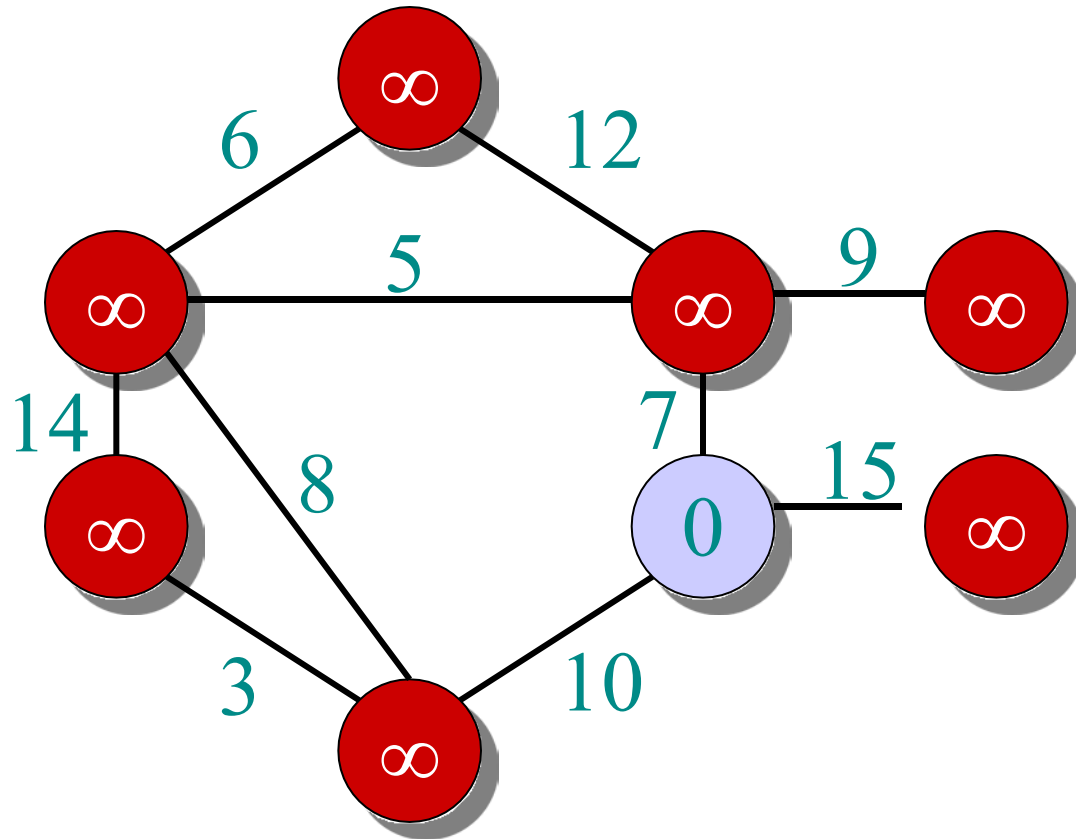
EXAMPLE OF PRIM'S ALGORITHM

 $\in A$
 $\in V - A$



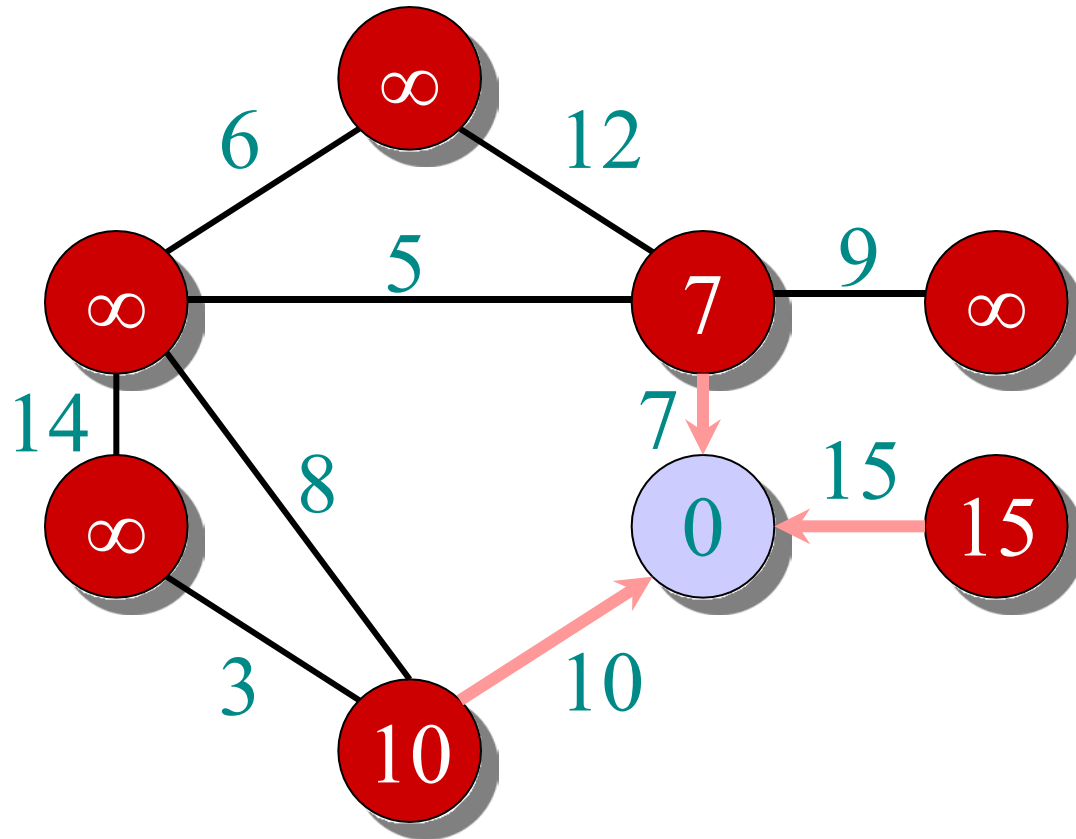
EXAMPLE OF PRIM'S ALGORITHM

 $\in A$
 $\in V - A$



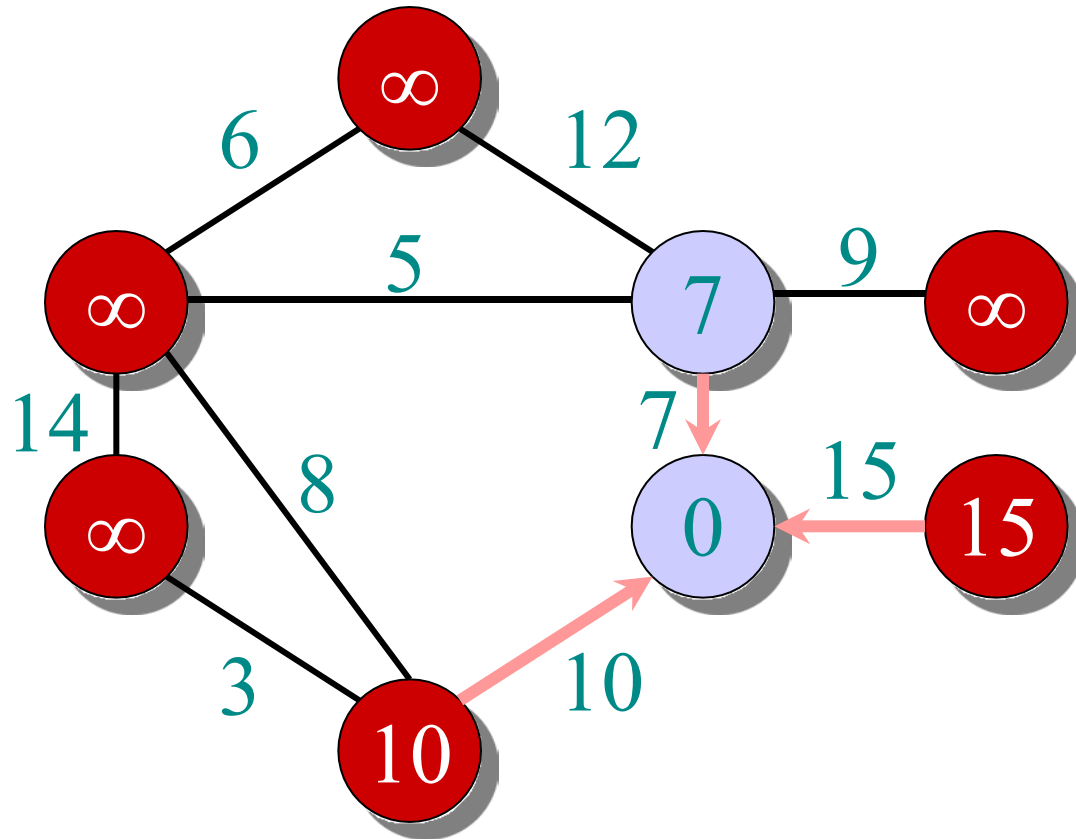
EXAMPLE OF PRIM'S ALGORITHM

 $\in A$
 $\in V - A$

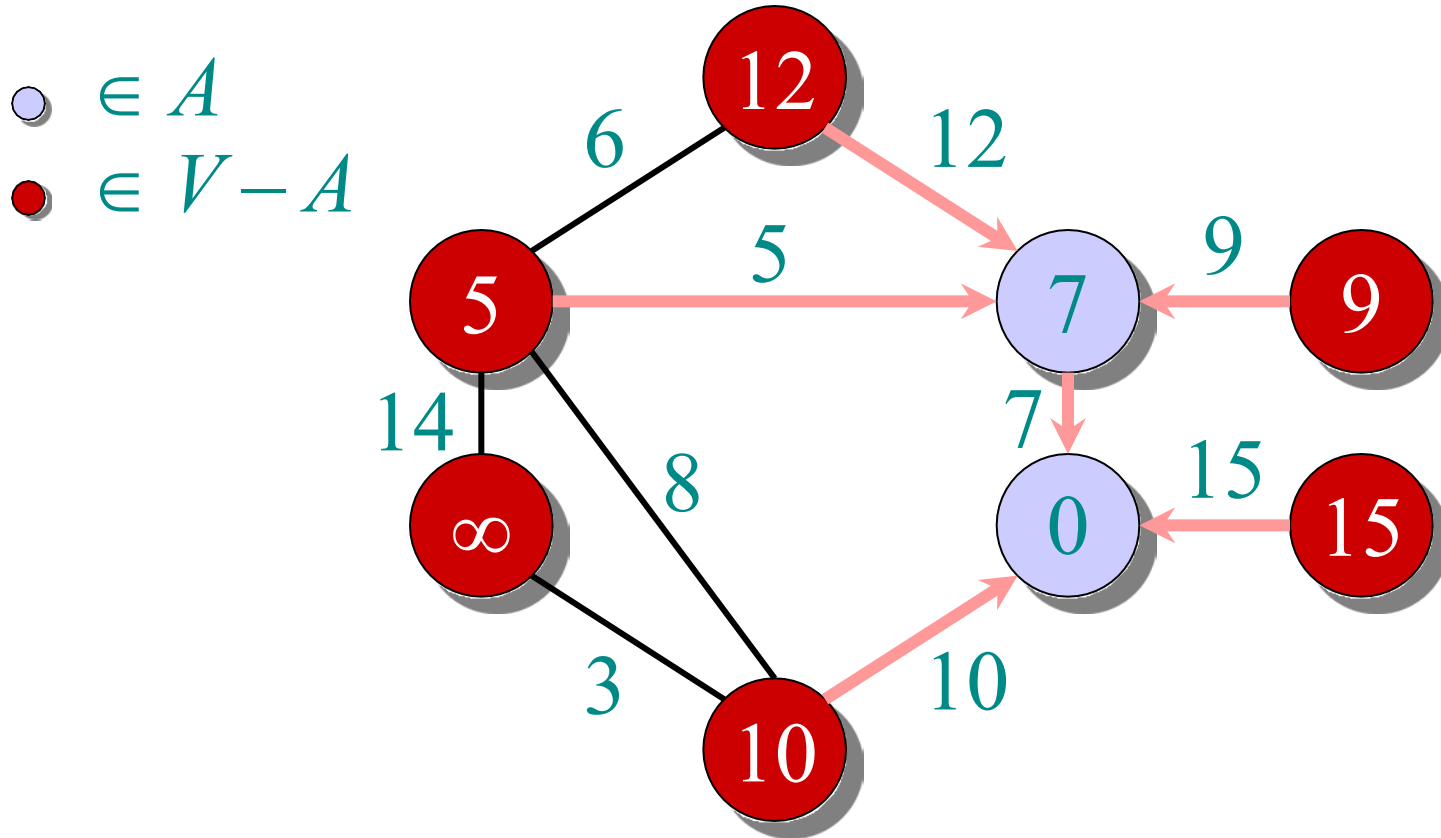


EXAMPLE OF PRIM'S ALGORITHM

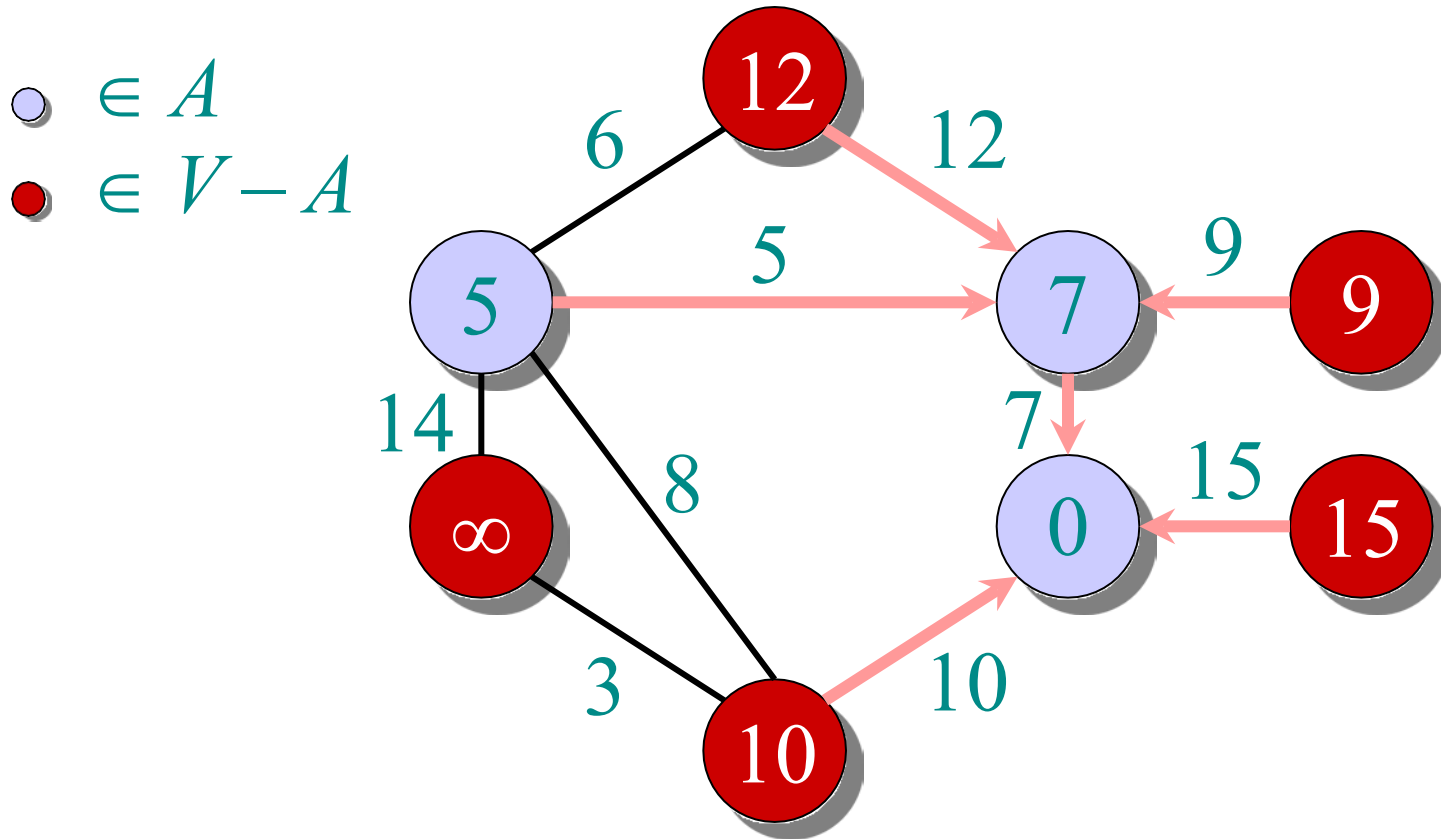
 $\in A$
 $\in V - A$



EXAMPLE OF PRIM'S ALGORITHM

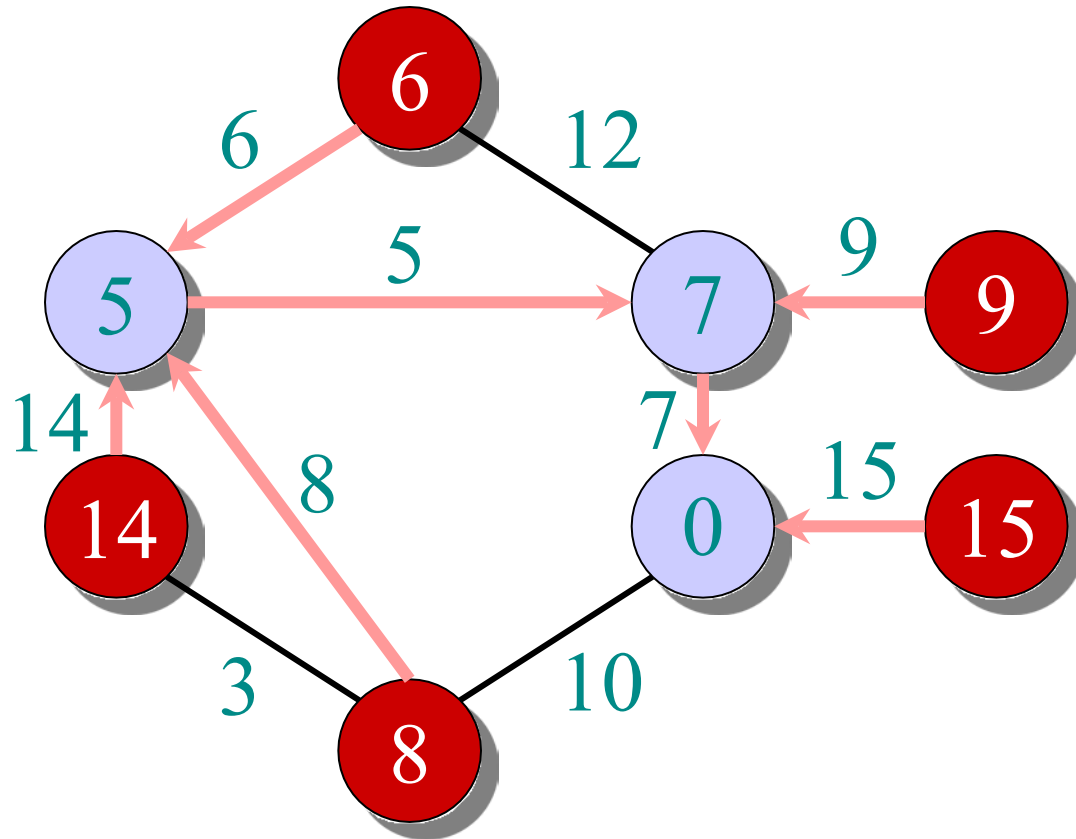


EXAMPLE OF PRIM'S ALGORITHM

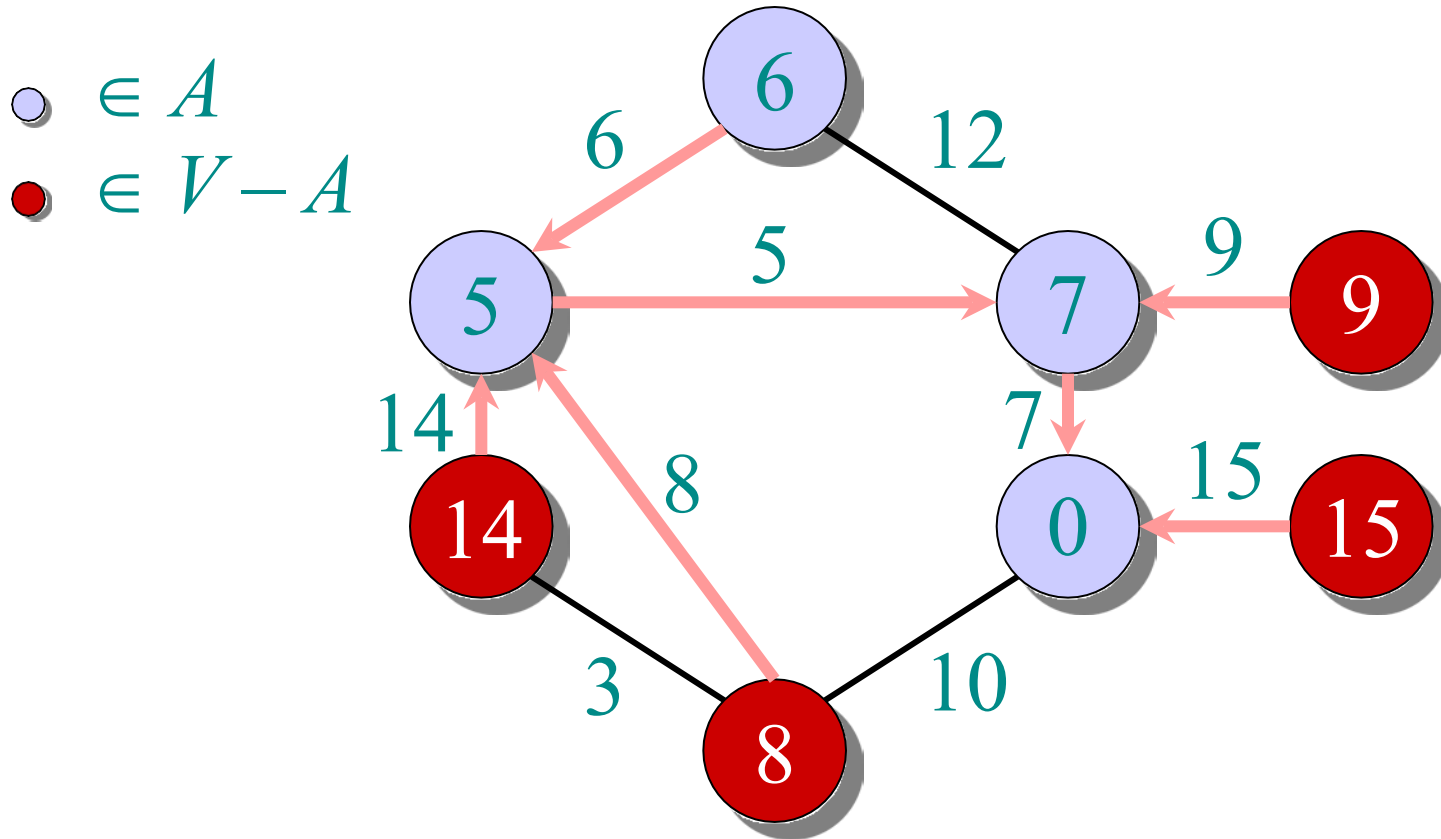


EXAMPLE OF PRIM'S ALGORITHM

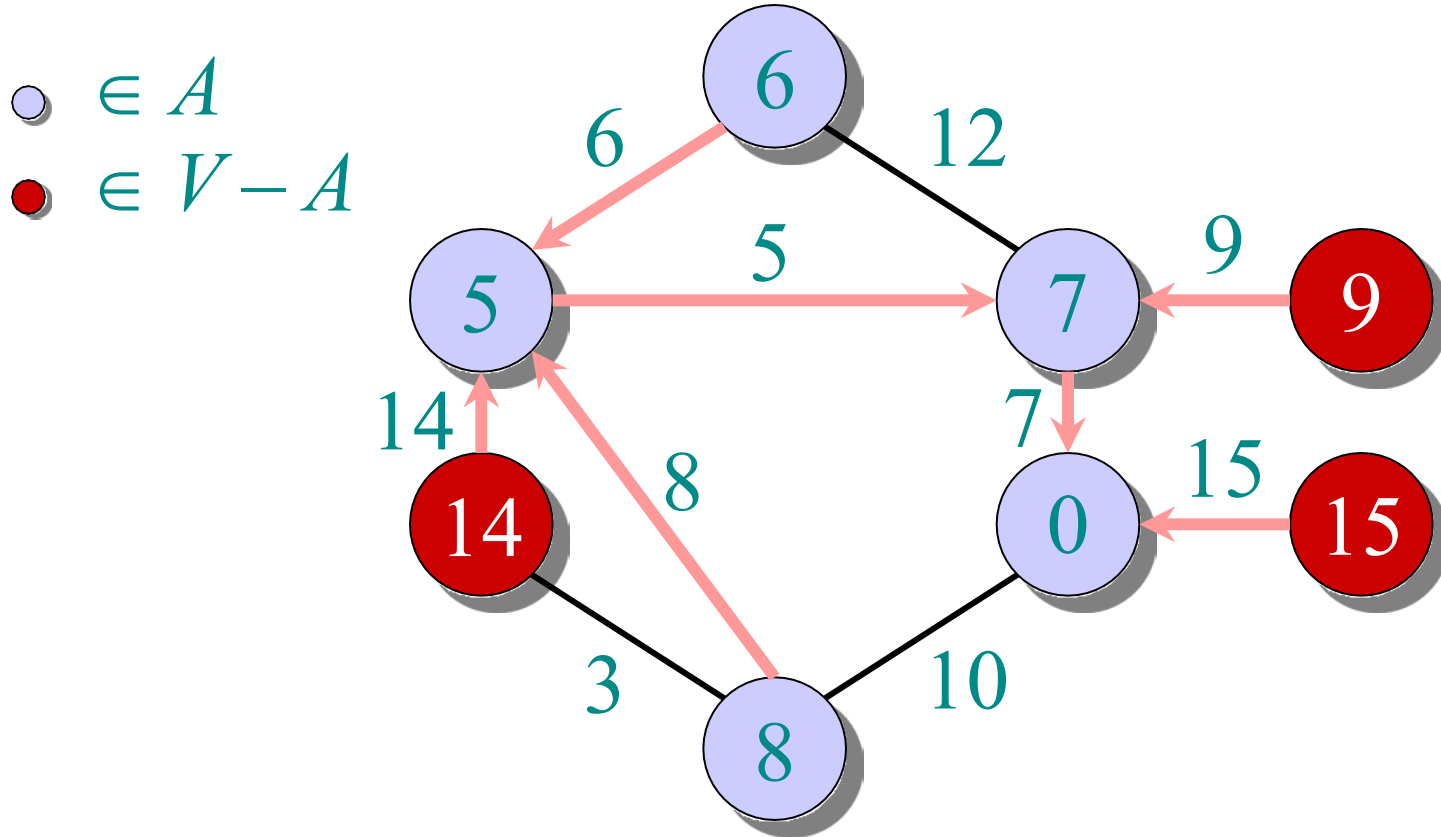
 $\in A$
 $\in V - A$



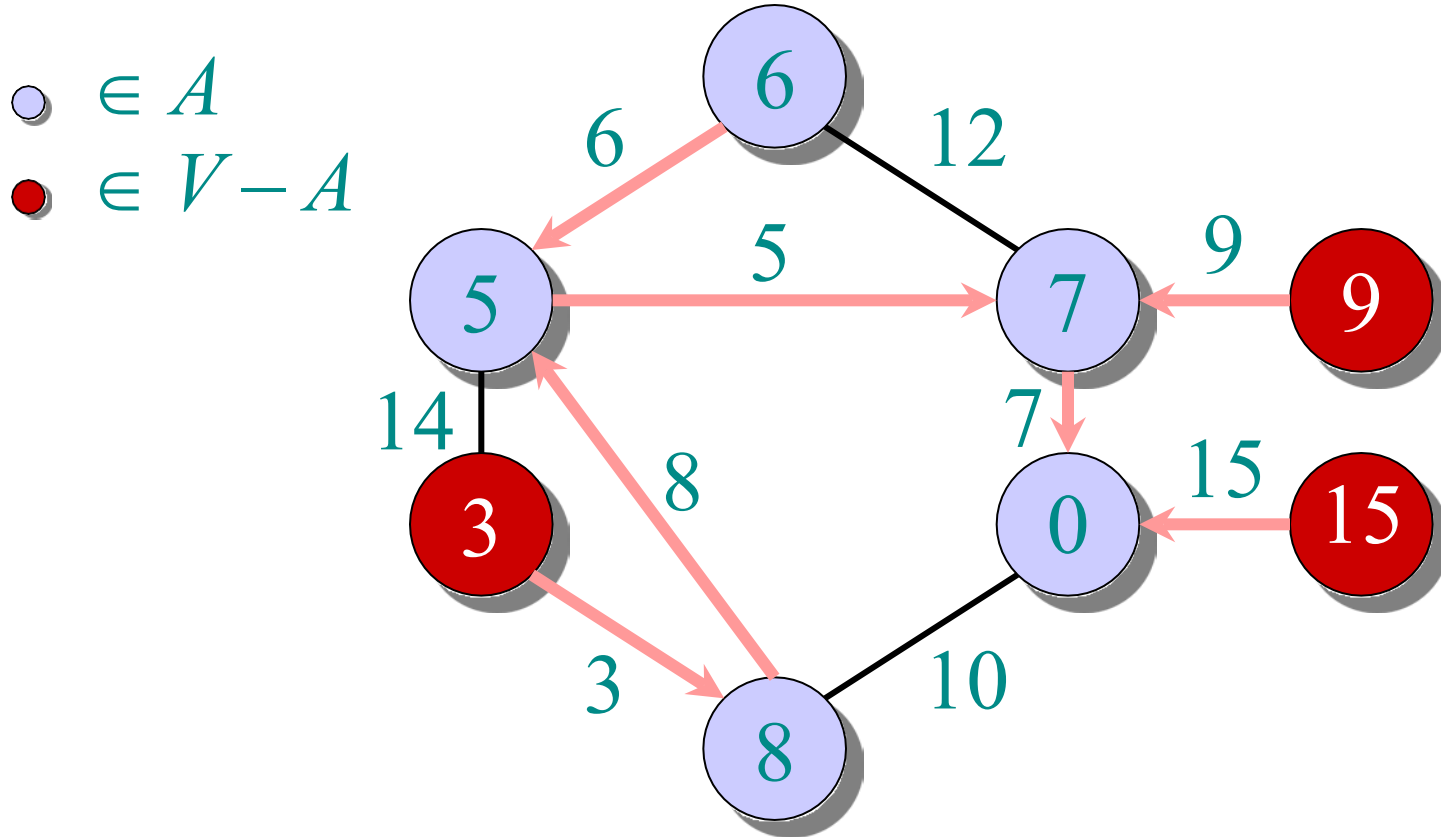
EXAMPLE OF PRIM'S ALGORITHM



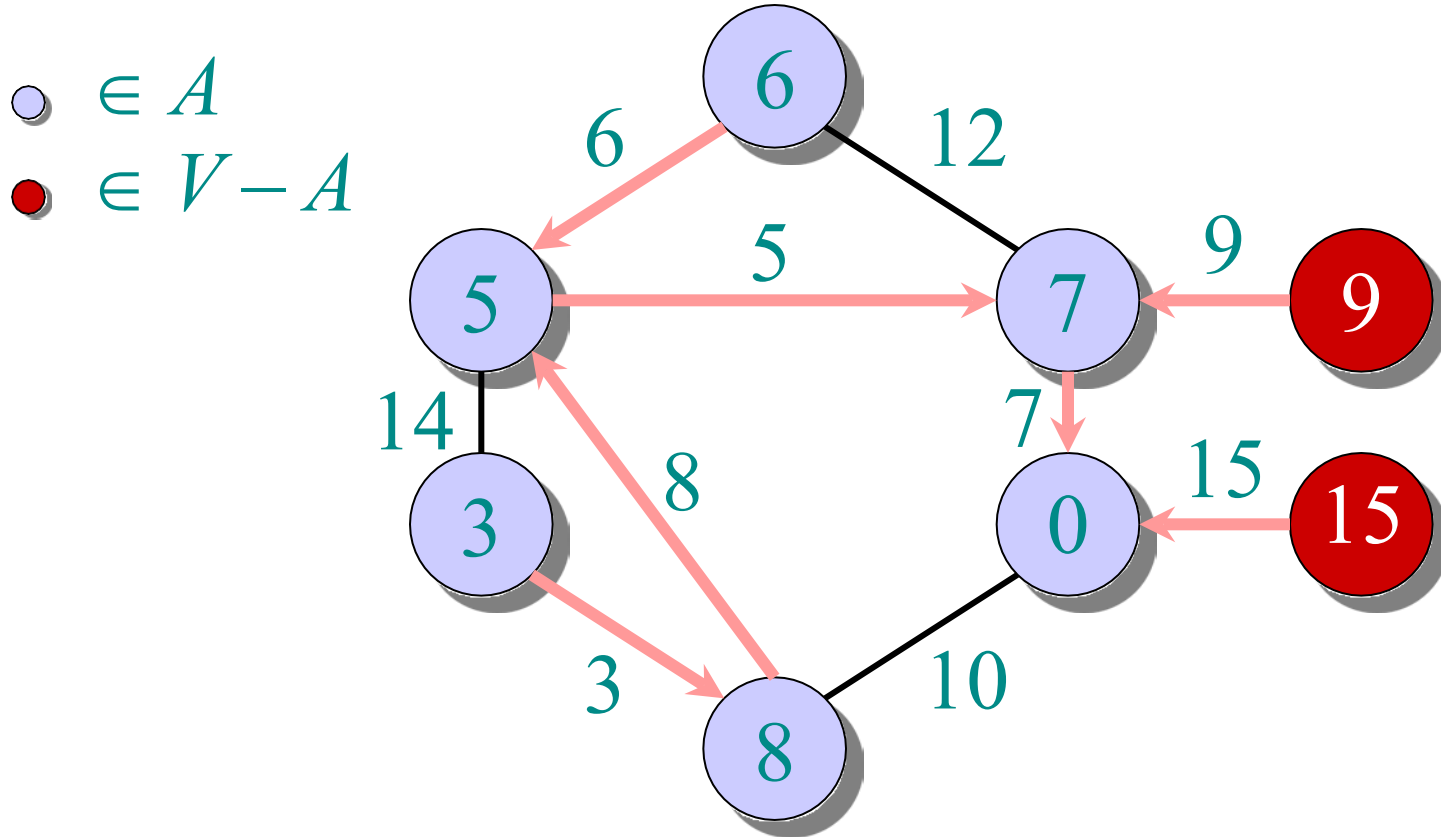
EXAMPLE OF PRIM'S ALGORITHM



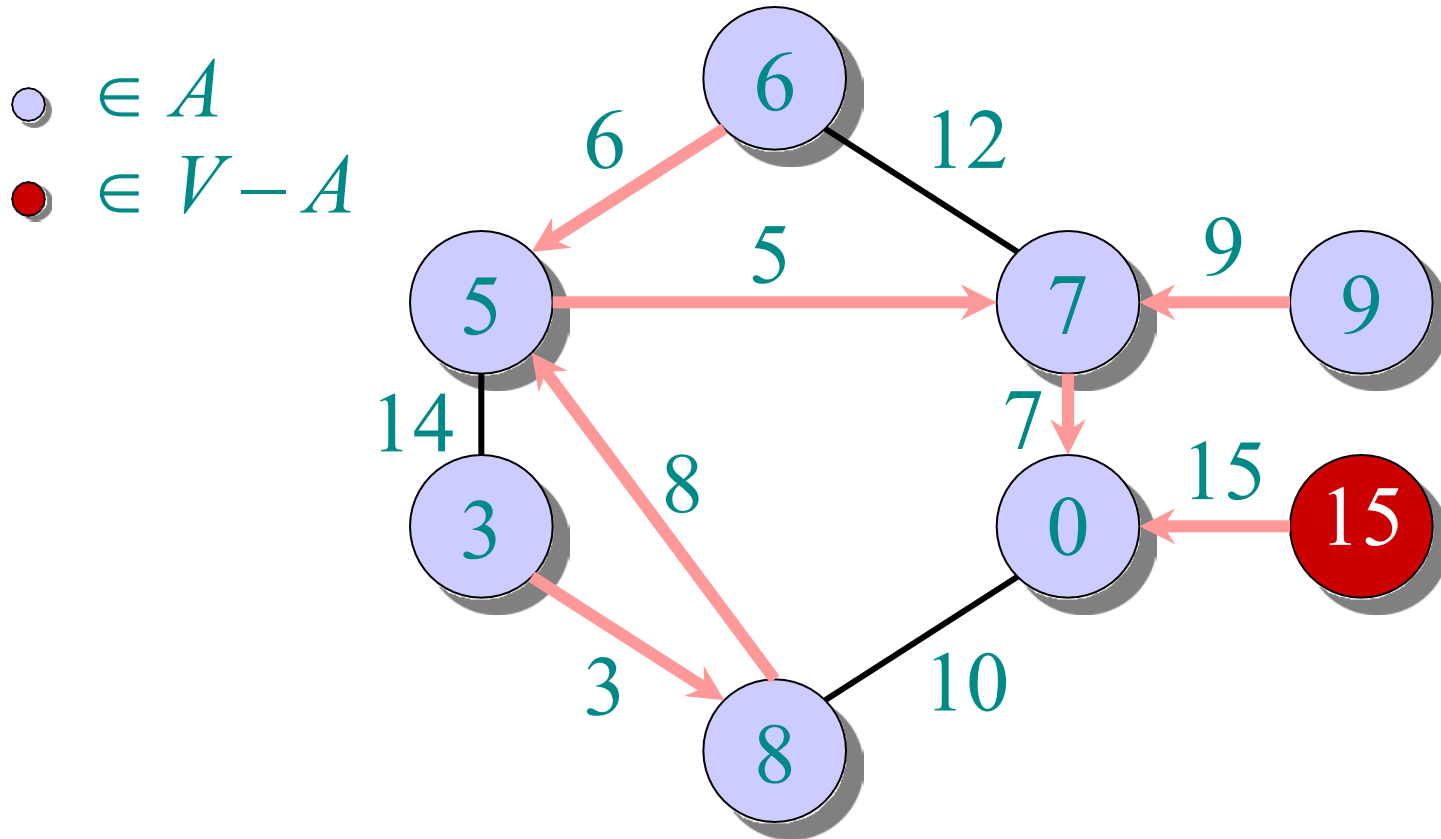
EXAMPLE OF PRIM'S ALGORITHM



EXAMPLE OF PRIM'S ALGORITHM

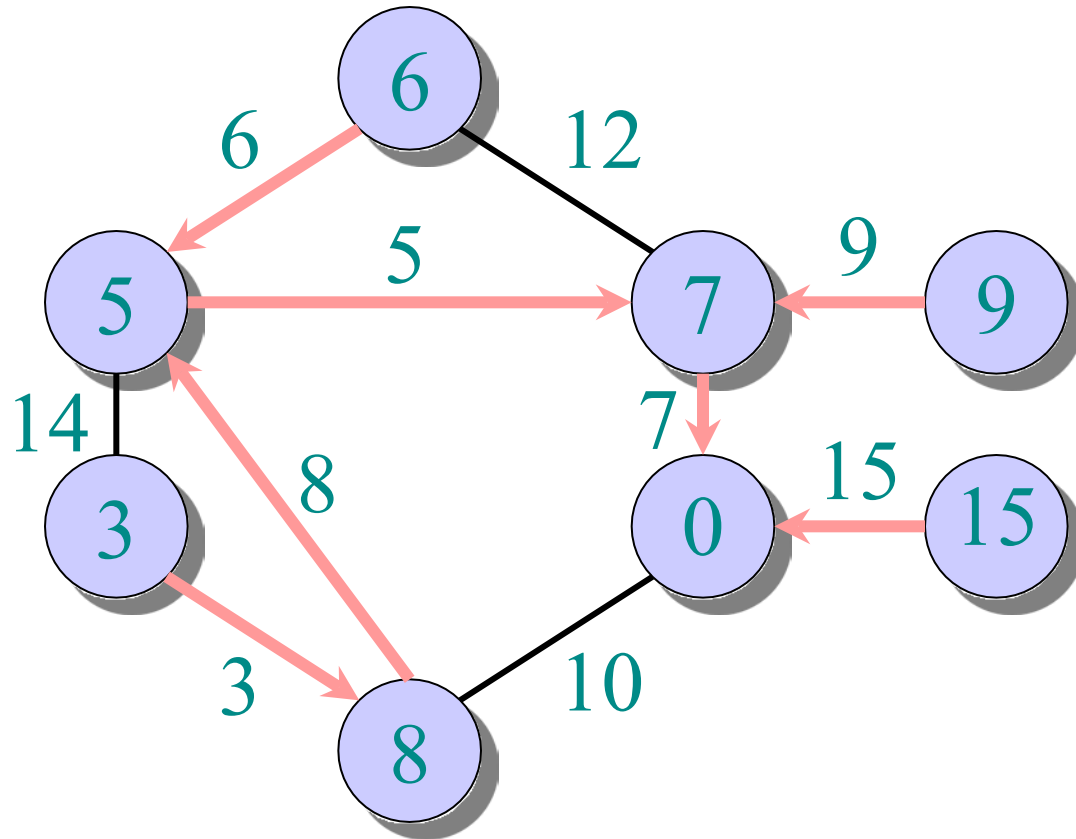


EXAMPLE OF PRIM'S ALGORITHM



EXAMPLE OF PRIM'S ALGORITHM

 $\in A$
 $\in V - A$



ANALYSIS OF PRIM

$Q \leftarrow V$

$key[v] \leftarrow \infty$ for all $v \in V$

$key[s] \leftarrow 0$ for some arbitrary $s \in V$

while $Q \neq \emptyset$

do $u \leftarrow \text{EXTRACT-MIN}(Q)$

for each $v \in \text{Adj}[u]$

do if $v \in Q$ and $w(u, v) < key[v]$

then $key[v] \leftarrow w(u, v)$

$\pi[v] \leftarrow u$

ANALYSIS OF PRIM

$\Theta(V)$
total

$\left\{ \begin{array}{l} Q \leftarrow V \\ key[v] \leftarrow \infty \text{ for all } v \in V \\ key[s] \leftarrow 0 \text{ for some arbitrary } s \in V \\ \textbf{while } Q \neq \emptyset \\ \quad \textbf{do } u \leftarrow \text{EXTRACT-MIN}(Q) \\ \quad \textbf{for each } v \in Adj[u] \\ \quad \quad \textbf{do if } v \in Q \text{ and } w(u, v) < key[v] \\ \quad \quad \quad \textbf{then } key[v] \leftarrow w(u, v) \\ \quad \quad \quad \pi[v] \leftarrow u \end{array} \right.$

ANALYSIS OF PRIM

$\Theta(V)$ total

$|V|$ times

```
 $Q \leftarrow V$   
 $key[v] \leftarrow \infty$  for all  $v \in V$   
 $key[s] \leftarrow 0$  for some arbitrary  $s \in V$   
while  $Q \neq \emptyset$   
    do  $u \leftarrow \text{EXTRACT-MIN}(Q)$   
    for each  $v \in \text{Adj}[u]$   
        do if  $v \in Q$  and  $w(u, v) < key[v]$   
            then  $key[v] \leftarrow w(u, v)$   
                 $\pi[v] \leftarrow u$ 
```

ANALYSIS OF PRIM

$\Theta(V)$ total {
 $Q \leftarrow V$
 $key[v] \leftarrow \infty$ for all $v \in V$
 $key[s] \leftarrow 0$ for some arbitrary $s \in V$
 while $Q \neq \emptyset$
 do $u \leftarrow \text{EXTRACT-MIN}(Q)$
 for each $v \in \text{Adj}[u]$
 do if $v \in Q$ and $w(u, v) < key[v]$
 then $key[v] \leftarrow w(u, v)$
 $\pi[v] \leftarrow u$

$|V|$ times {
 $degree(u)$ times {

ANALYSIS OF PRIM

$\Theta(V)$ total {
 $Q \leftarrow V$
 $key[v] \leftarrow \infty$ for all $v \in V$
 $key[s] \leftarrow 0$ for some arbitrary $s \in V$
 while $Q \neq \emptyset$
 do $u \leftarrow \text{EXTRACT-MIN}(Q)$
 for each $v \in \text{Adj}[u]$
 do if $v \in Q$ and $w(u, v) < key[v]$
 then $key[v] \leftarrow w(u, v)$
 $\pi[v] \leftarrow u$

$|V|$ times {
 $degree(u)$ times {



Handshaking Lemma $\Rightarrow \Theta(E)$ implicit DECREASE-KEY's.

ANALYSIS OF PRIM

```

 $\Theta(V)$  total {
     $Q \leftarrow V$ 
     $key[v] \leftarrow \infty$  for all  $v \in V$ 
     $key[s] \leftarrow 0$  for some arbitrary  $s \in V$ 
    while  $Q \neq \emptyset$ 
    do  $u \leftarrow \text{EXTRACT-MIN}(Q)$ 
    for each  $v \in Adj[u]$ 
    do if  $v \in Q$  and  $w(u, v) < key[v]$ 
    then  $key[v] \leftarrow w(u, v)$ 
     $\pi[v] \leftarrow u$ 
}

```

Handshaking Lemma $\Rightarrow \Theta(E)$ implicit DECREASE-KEY's.

$$\text{Time} = \Theta(V) \cdot T_{\text{EXTRACT-MIN}} + \Theta(E) \cdot T_{\text{DECREASE-KEY}}$$

Analysis of Prim (continued)

$$\text{Time} = \Theta(V) \cdot T_{\text{EXTRACT-MIN}} + \Theta(E) \cdot T_{\text{DECREASE-KEY}}$$

ANALYSIS OF PRIM (CONTINUED)

$$\text{Time} = \Theta(V) \cdot T_{\text{EXTRACT-MIN}} + \Theta(E) \cdot T_{\text{DECREASE-KEY}}$$

Q	$T_{\text{EXTRACT-MIN}}$	$T_{\text{DECREASE-KEY}}$	Total
-----	--------------------------	---------------------------	-------

ANALYSIS OF PRIM (CONTINUED)

$$\text{Time} = \Theta(V) \cdot T_{\text{EXTRACT-MIN}} + \Theta(E) \cdot T_{\text{DECREASE-KEY}}$$

Q	$T_{\text{EXTRACT-MIN}}$	$T_{\text{DECREASE-KEY}}$	Total
-----	--------------------------	---------------------------	-------

array	$O(V)$	$O(1)$	$O(V^2)$
-------	--------	--------	----------

ANALYSIS OF PRIM (CONTINUED)

$$\text{Time} = \Theta(V) \cdot T_{\text{EXTRACT-MIN}} + \Theta(E) \cdot T_{\text{DECREASE-KEY}}$$

Q	$T_{\text{EXTRACT-MIN}}$	$T_{\text{DECREASE-KEY}}$	Total
array	$O(V)$	$O(1)$	$O(V^2)$
binary heap	$O(\lg V)$	$O(\lg V)$	$O(E \lg V)$

ANALYSIS OF PRIM (CONTINUED)

$$\text{Time} = \Theta(V) \cdot T_{\text{EXTRACT-MIN}} + \Theta(E) \cdot T_{\text{DECREASE-KEY}}$$

Q	$T_{\text{EXTRACT-MIN}}$	$T_{\text{DECREASE-KEY}}$	Total
array	$O(V)$	$O(1)$	$O(V^2)$
binary heap	$O(\lg V)$	$O(\lg V)$	$O(E \lg V)$
Fibonacci heap	$O(\lg V)$ amortized	$O(1)$ amortized	$O(E + V \lg V)$ worst case

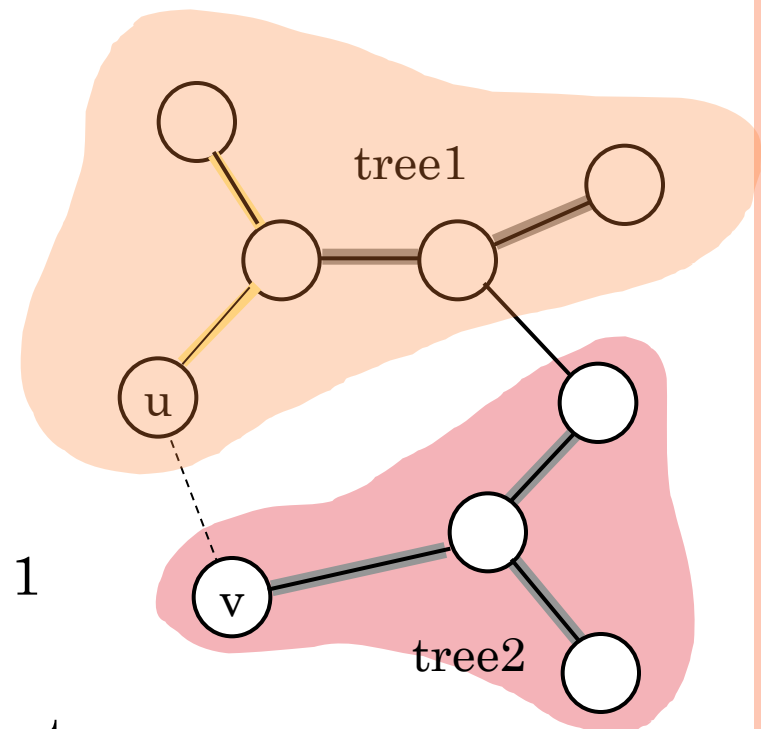
MST ALGORITHMS

Kruskal's algorithm (see CLRS):

- Uses the *disjoint-set data structure* (see CLRS, Ch. 21).
- Running time = $O(E \lg V)$.

KRUSKAL'S ALGORITHM

- How is it different from Prim's algorithm?
 - Prim's algorithm grows one tree all the time
 - Kruskal's algorithm grows multiple trees (i.e., a forest) at the same time.
 - Trees are merged together using **safe** edges
 - Since an MST has exactly $|V| - 1$ edges, after $|V| - 1$ merges, we would have only one component



KRUSKAL'S ALGORITHM

- Kruskal's algorithm creates a forest of trees
- Initially forest consists of N single node trees (and no edges)
- Sorts the edges by weight and goes through the edges from least weight to greatest weight
- At each step one edge (with least weight) is added so that it joins two trees together
 - As long as the addition does not create a cycle

KRUSKAL'S ALGORITHM

The halting conditions are as follows:

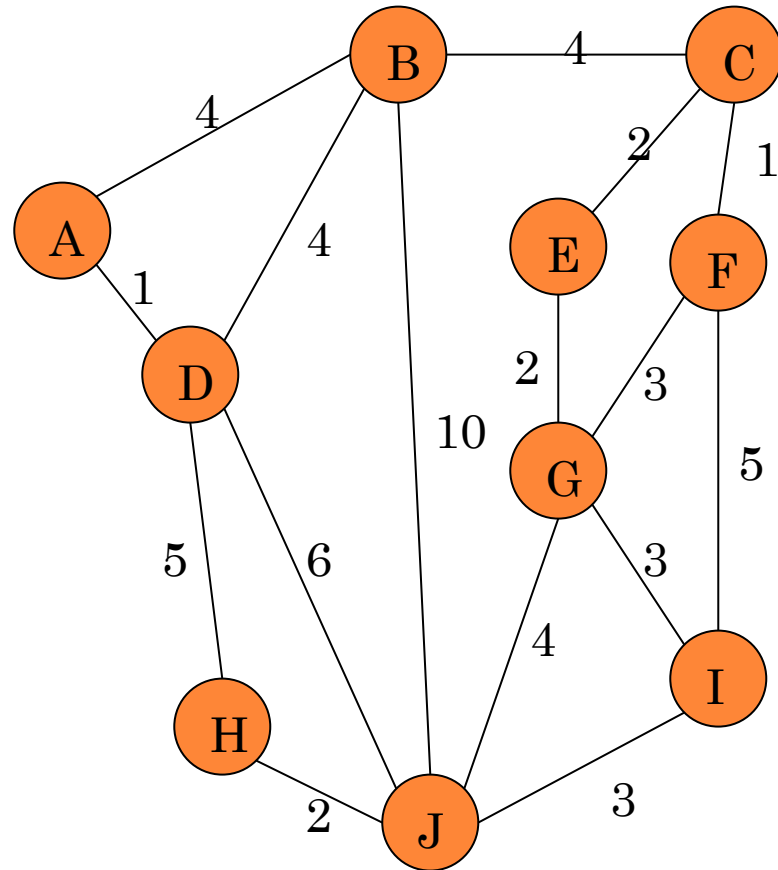
1. When $|V| - 1$ edges have been added
 - In this case we have a minimum spanning tree
2. We have gone through all edges
 - A forest of minimum spanning trees on all connected sub-graphs

KRUSKAL'S ALGORITHM

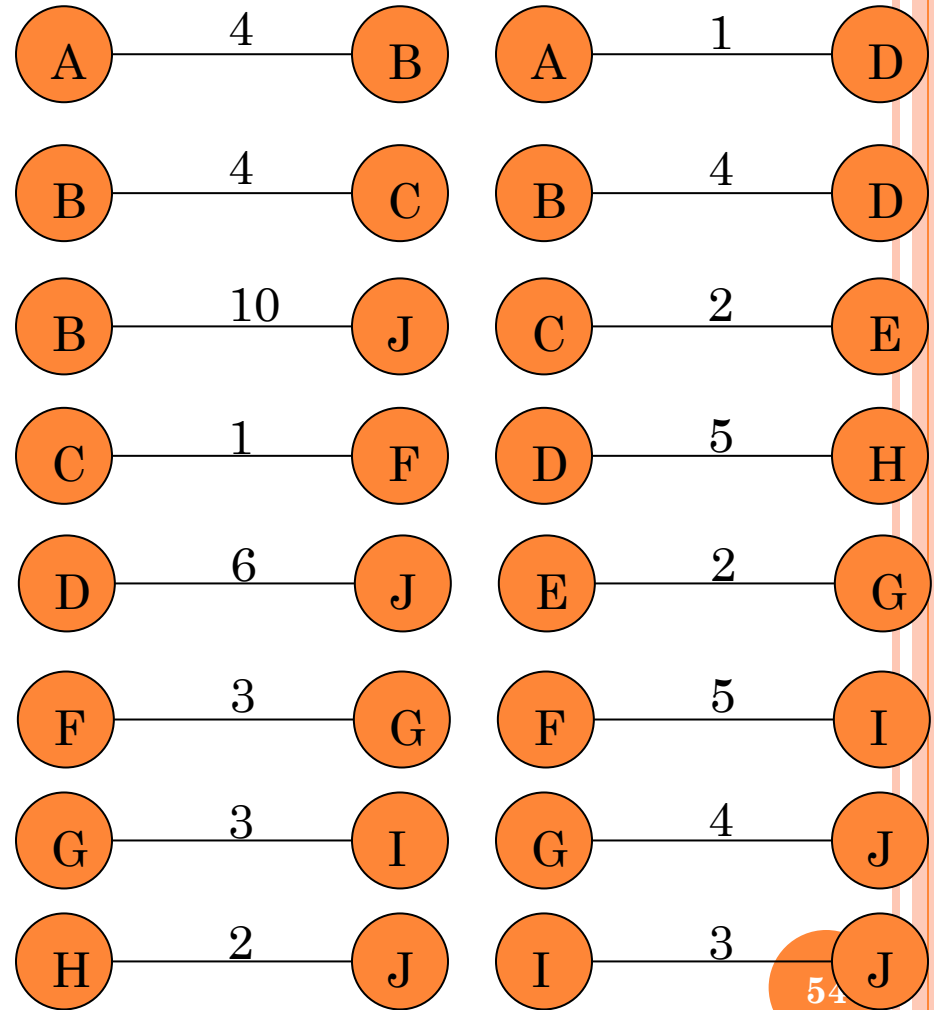
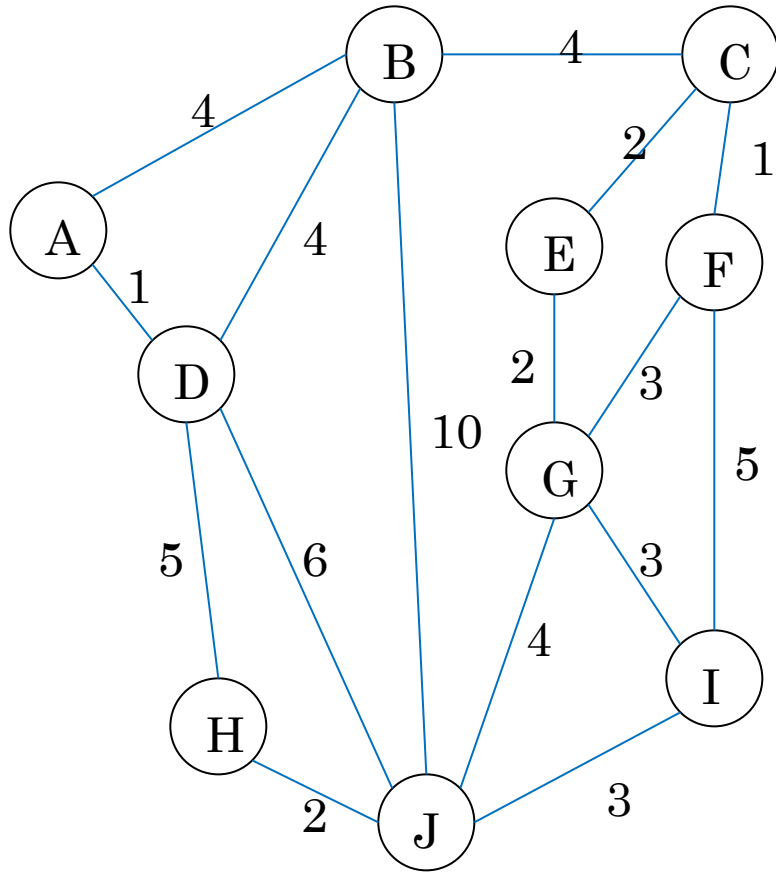
1. The forest is constructed – with each node in a separate tree
 2. The edges are placed in a priority queue
 3. Until we've added $n-1$ edges (assumption: connected graph)
 1. Extract the cheapest edge from the queue
 2. If it forms a cycle, reject it
 3. Else add it to the forest. Adding it to the forest will join two trees
- Every step will have joined two trees in the forest together, so that at the end, there will only be one tree

KRUSKAL'S ALGORITHM – EXAMPLE

- Complete Graph

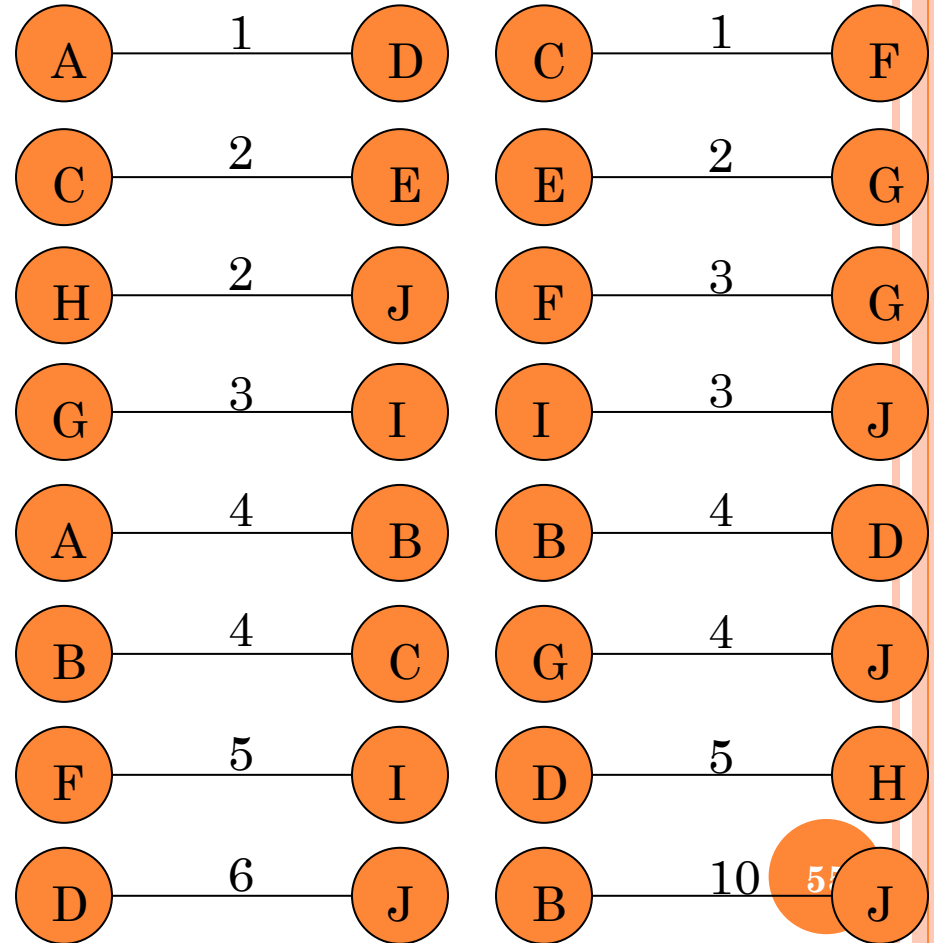
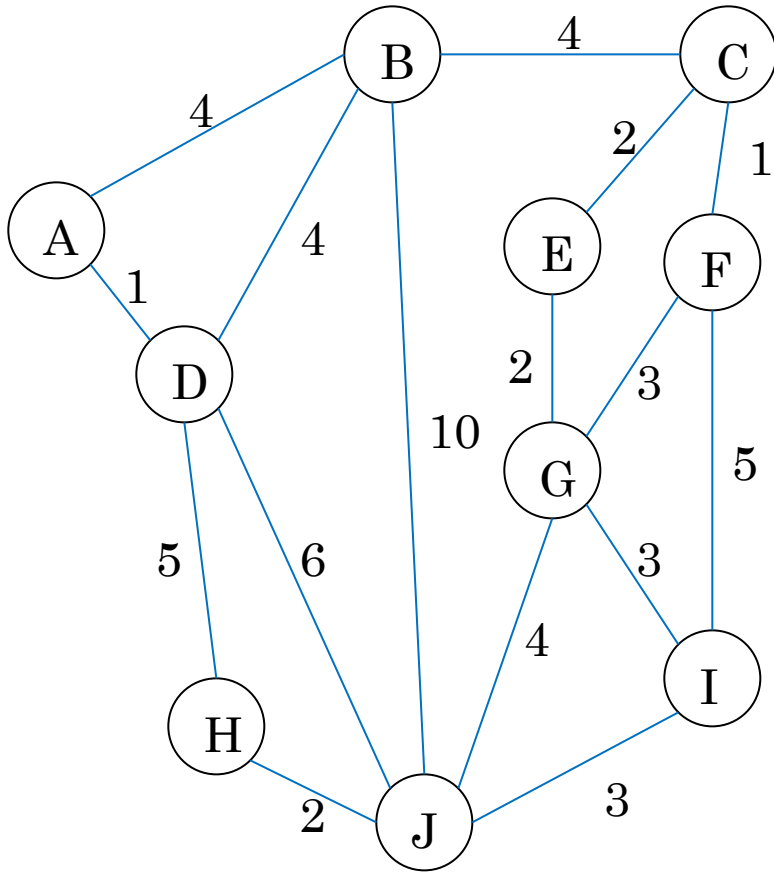


KRUSKAL'S ALGORITHM – EXAMPLE



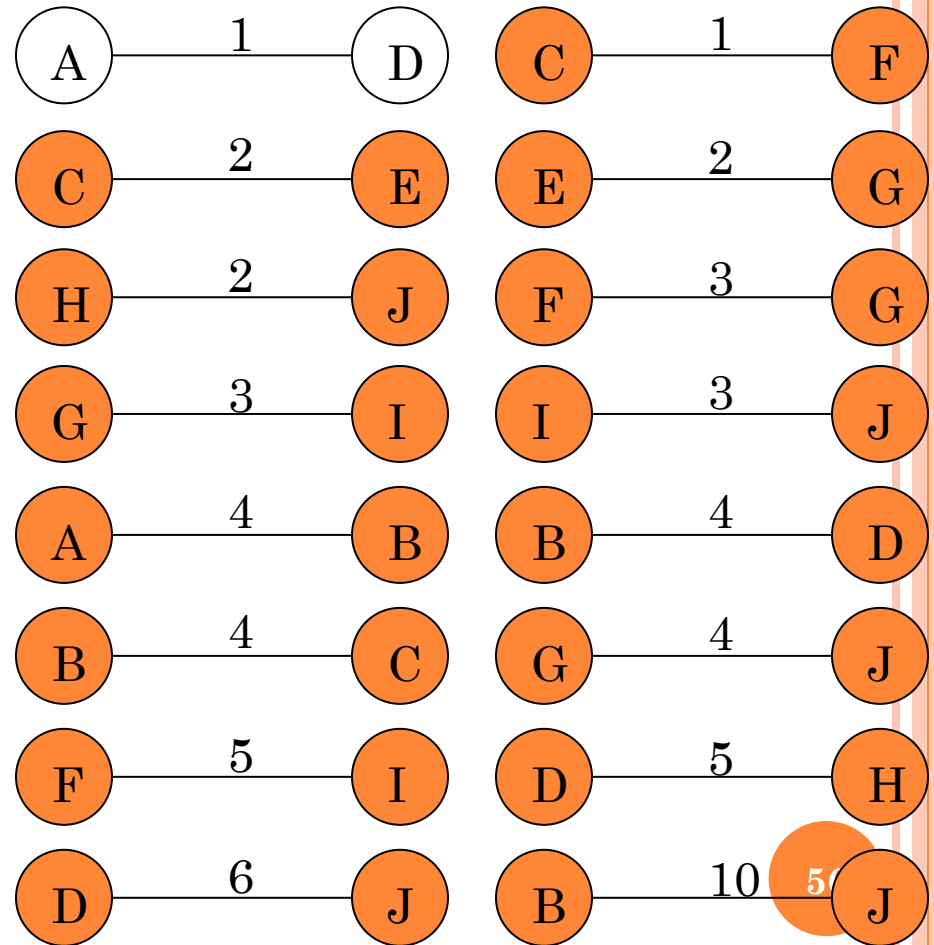
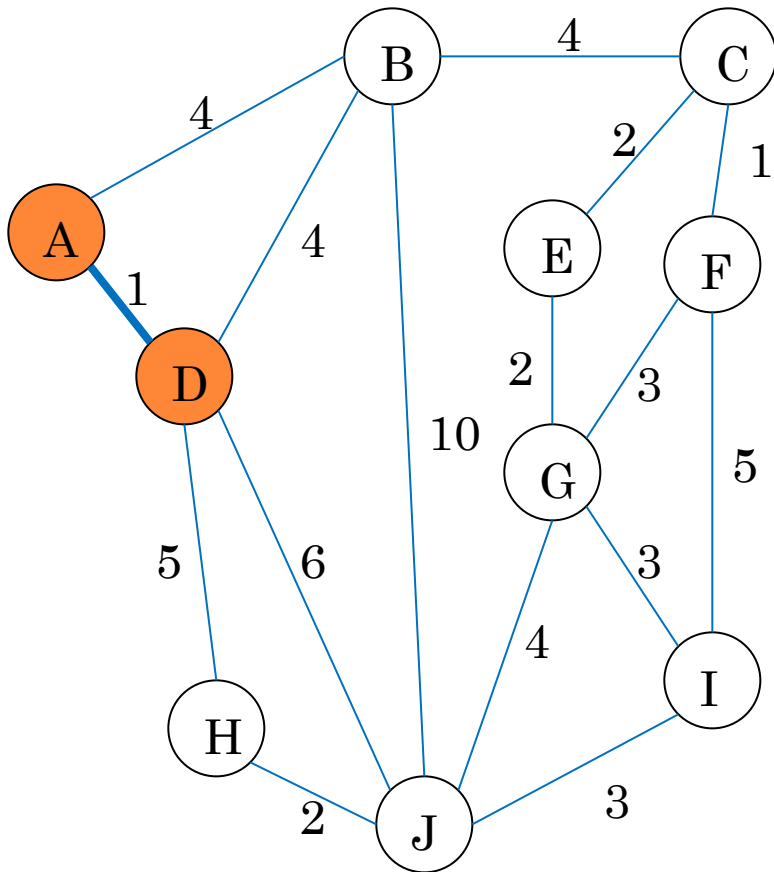
KRUSKAL'S ALGORITHM – EXAMPLE

- Sort Edges: In reality priority queue is used



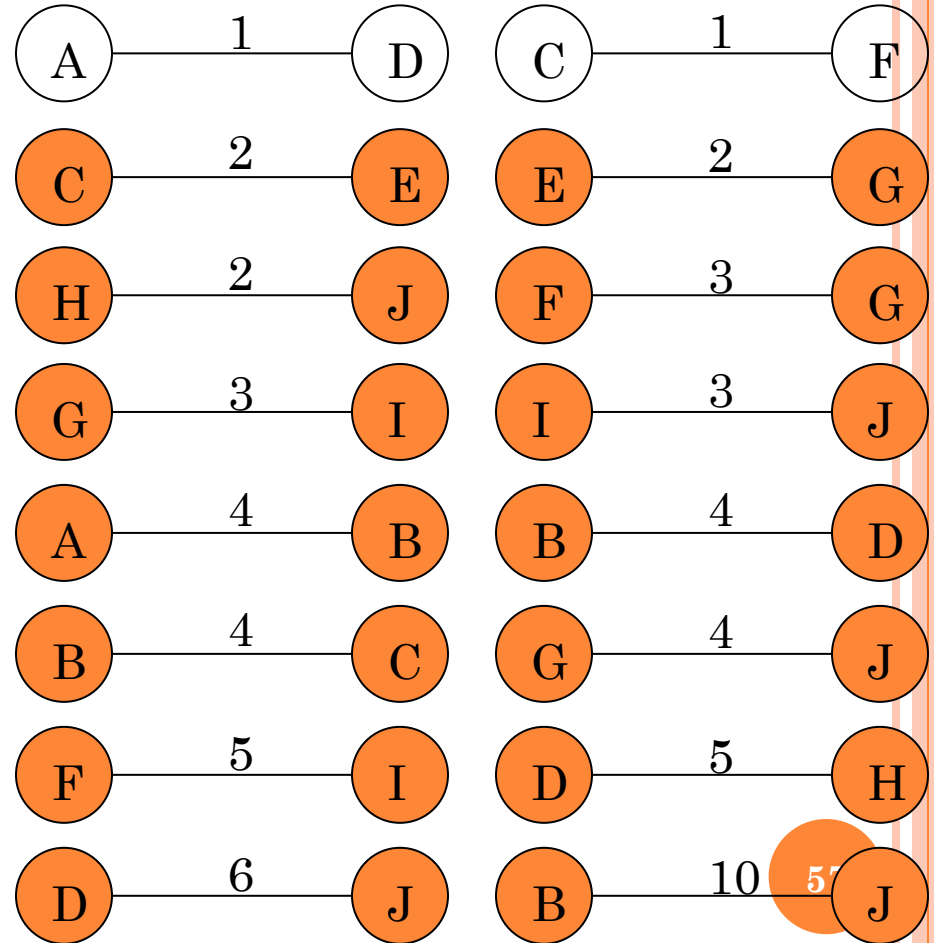
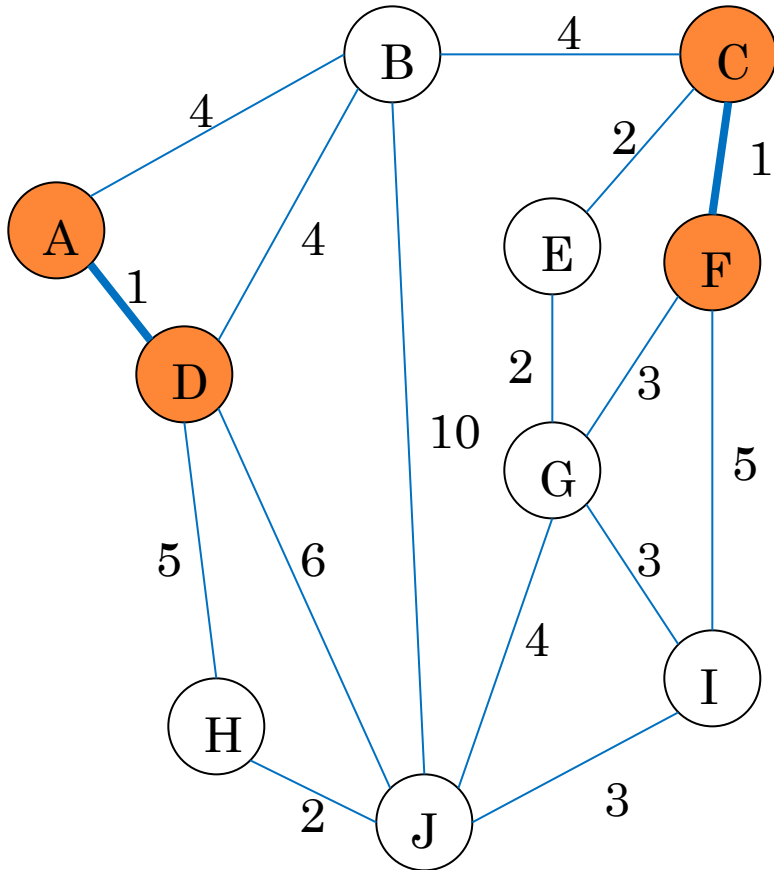
KRUSKAL'S ALGORITHM – EXAMPLE

○ Add Edge



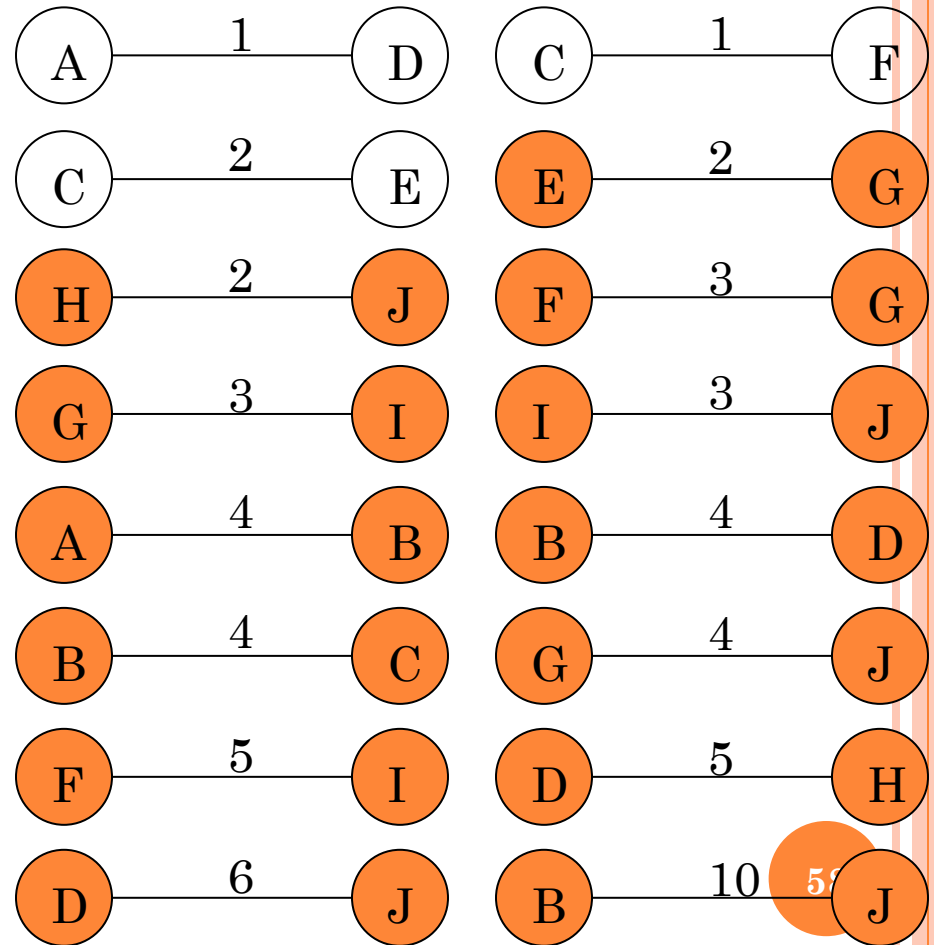
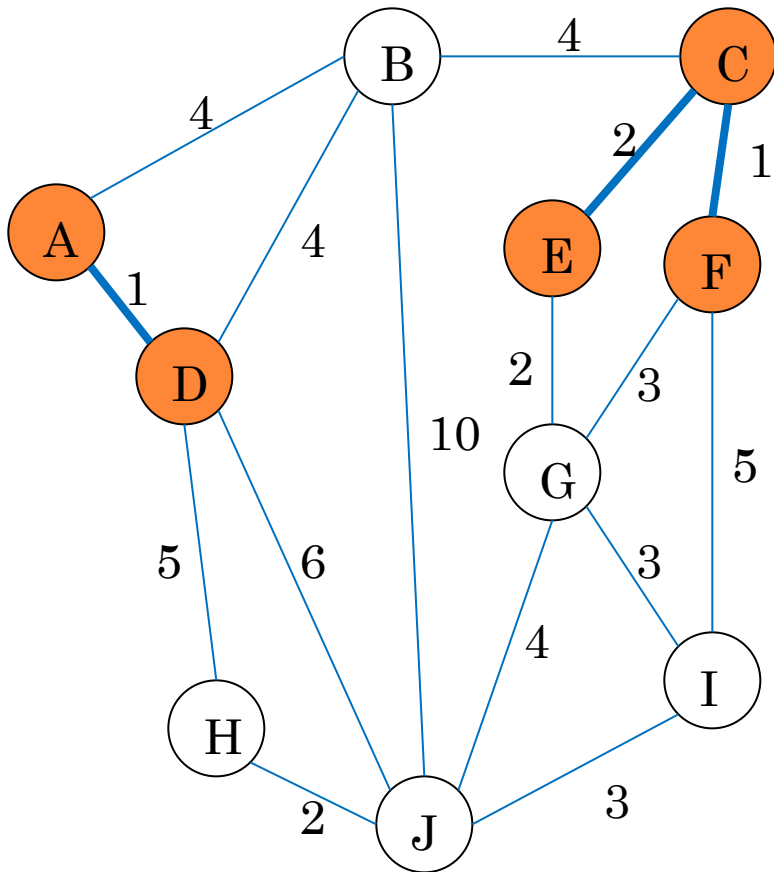
KRUSKAL'S ALGORITHM – EXAMPLE

○ Add Edge



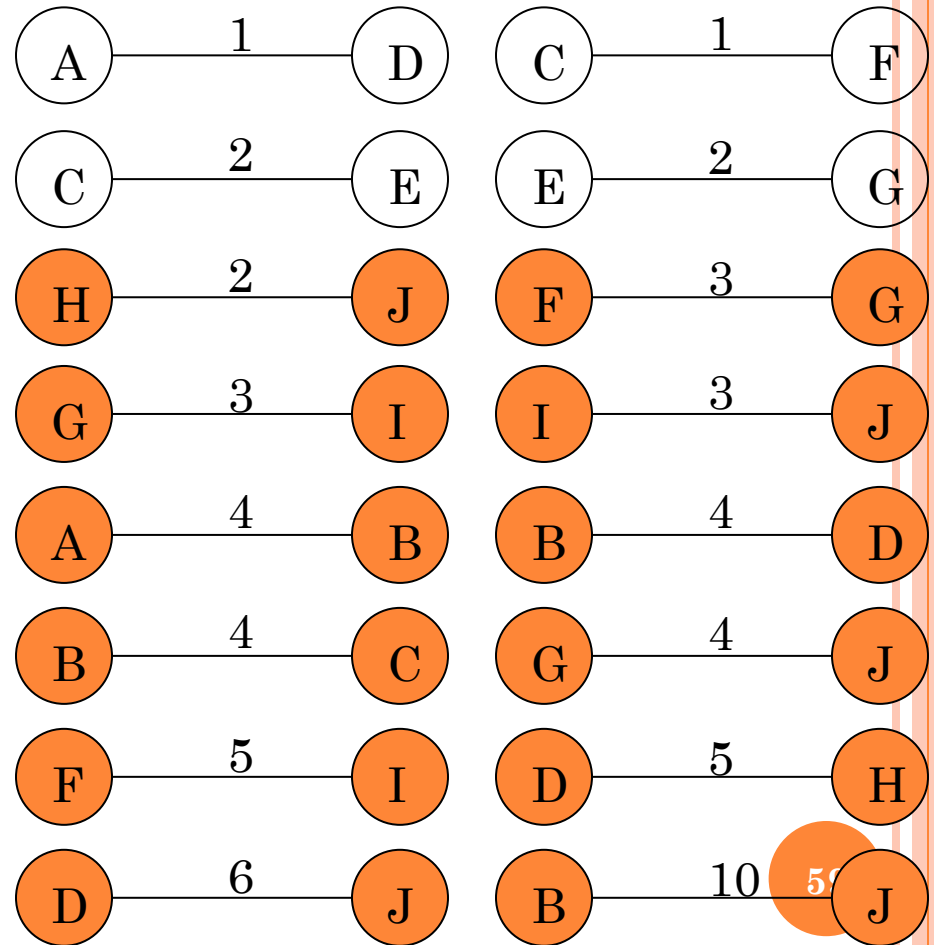
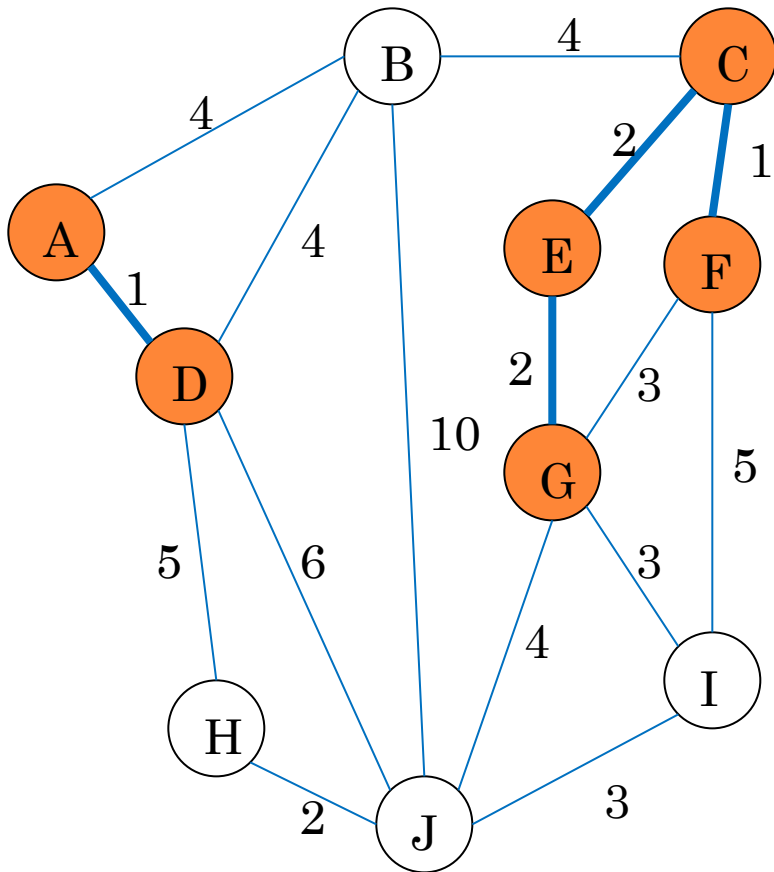
KRUSKAL'S ALGORITHM – EXAMPLE

○ Add Edge



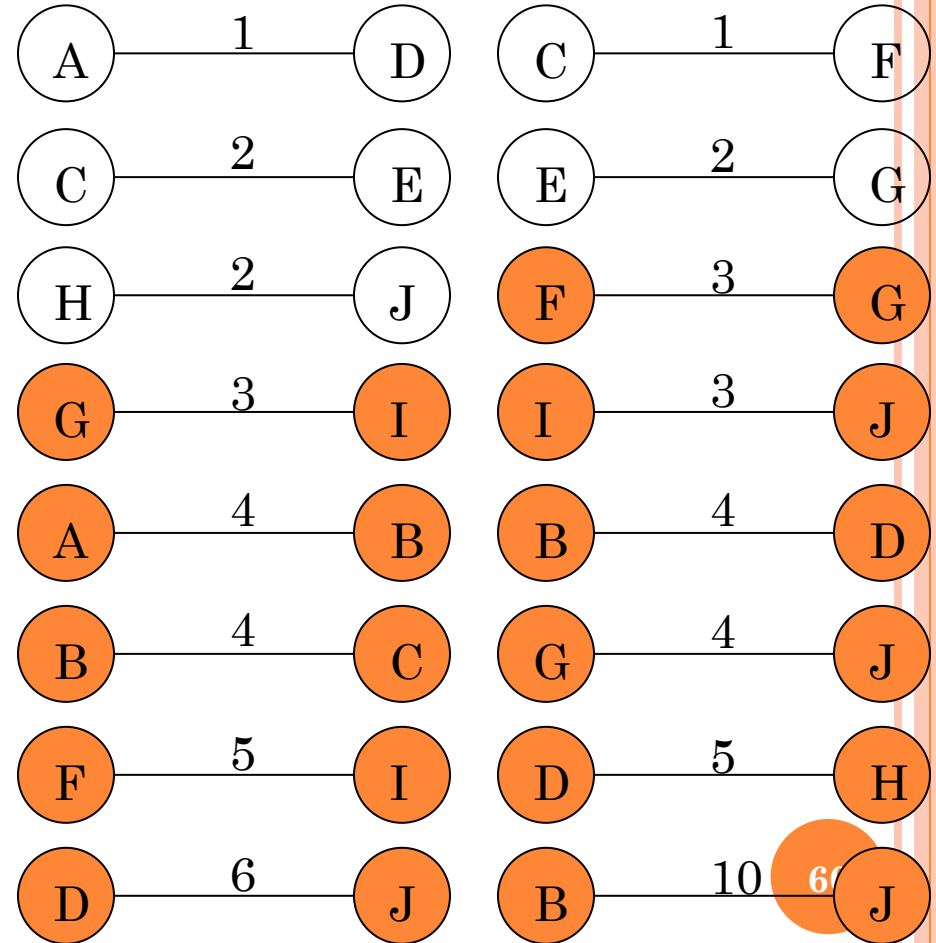
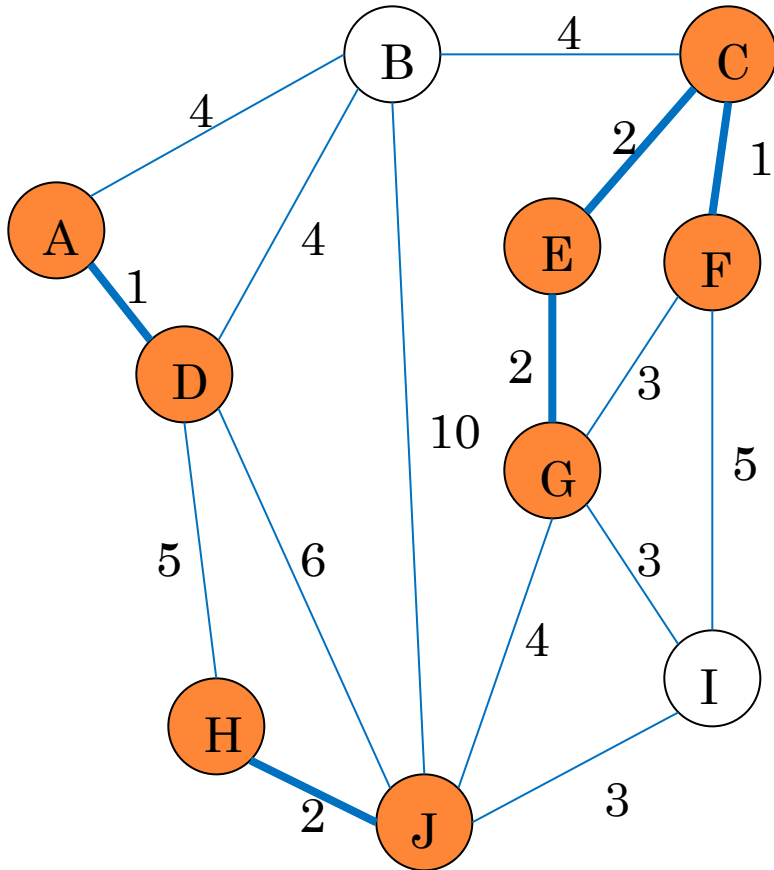
KRUSKAL'S ALGORITHM – EXAMPLE

○ Add Edge



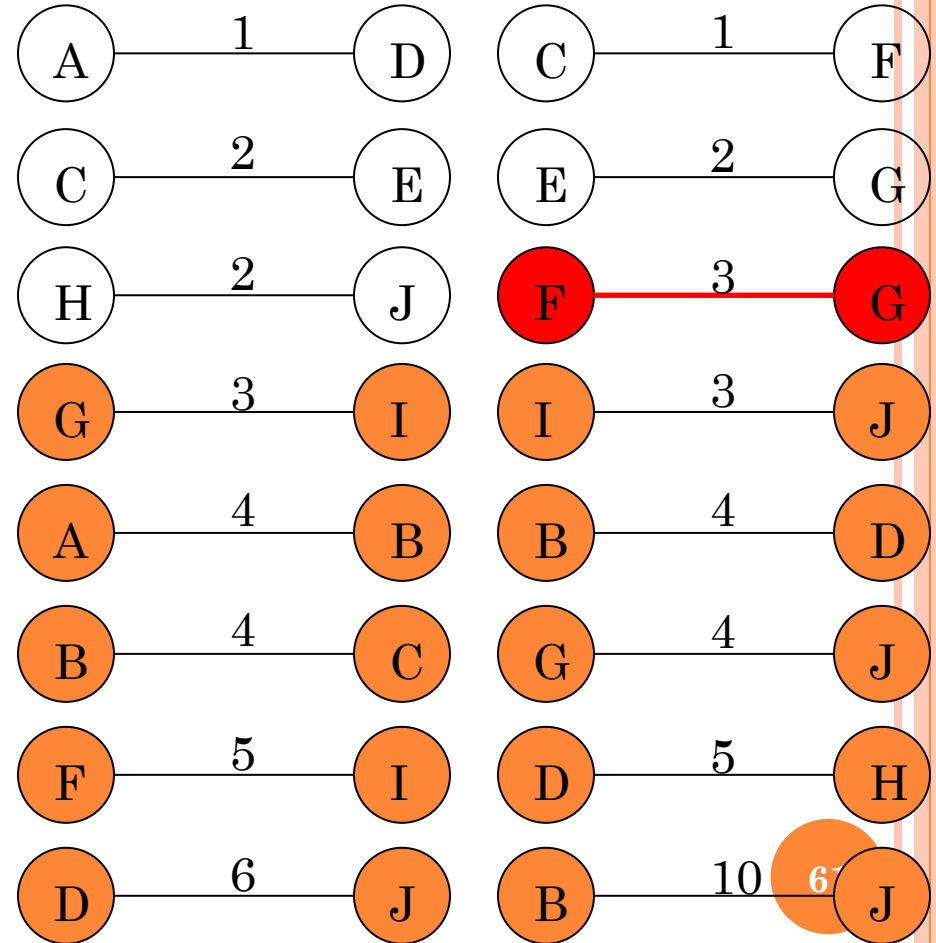
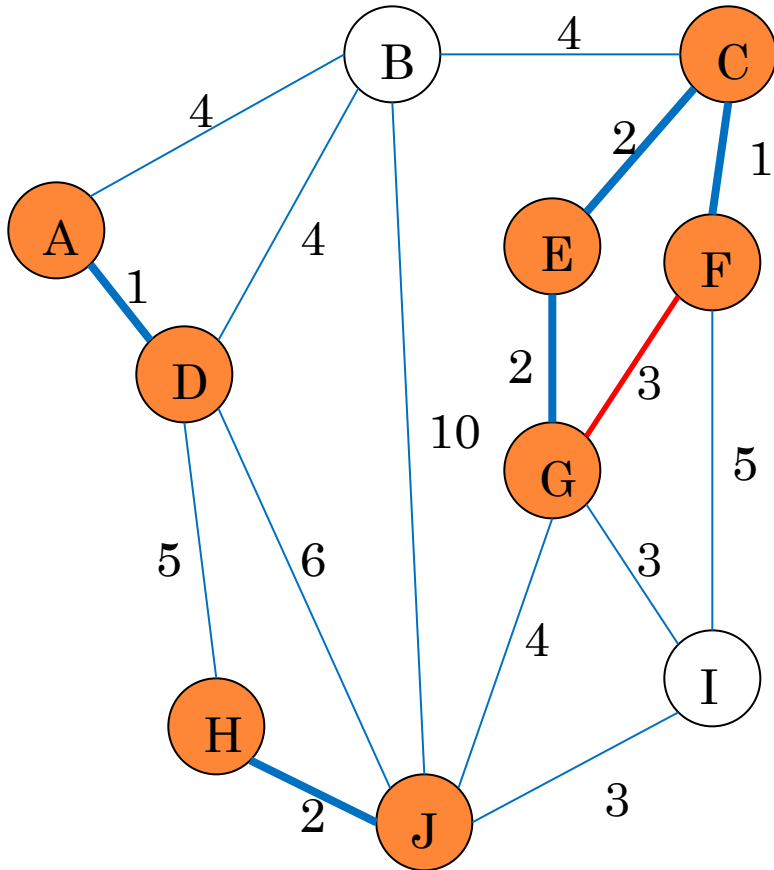
KRUSKAL'S ALGORITHM – EXAMPLE

○ Add Edge



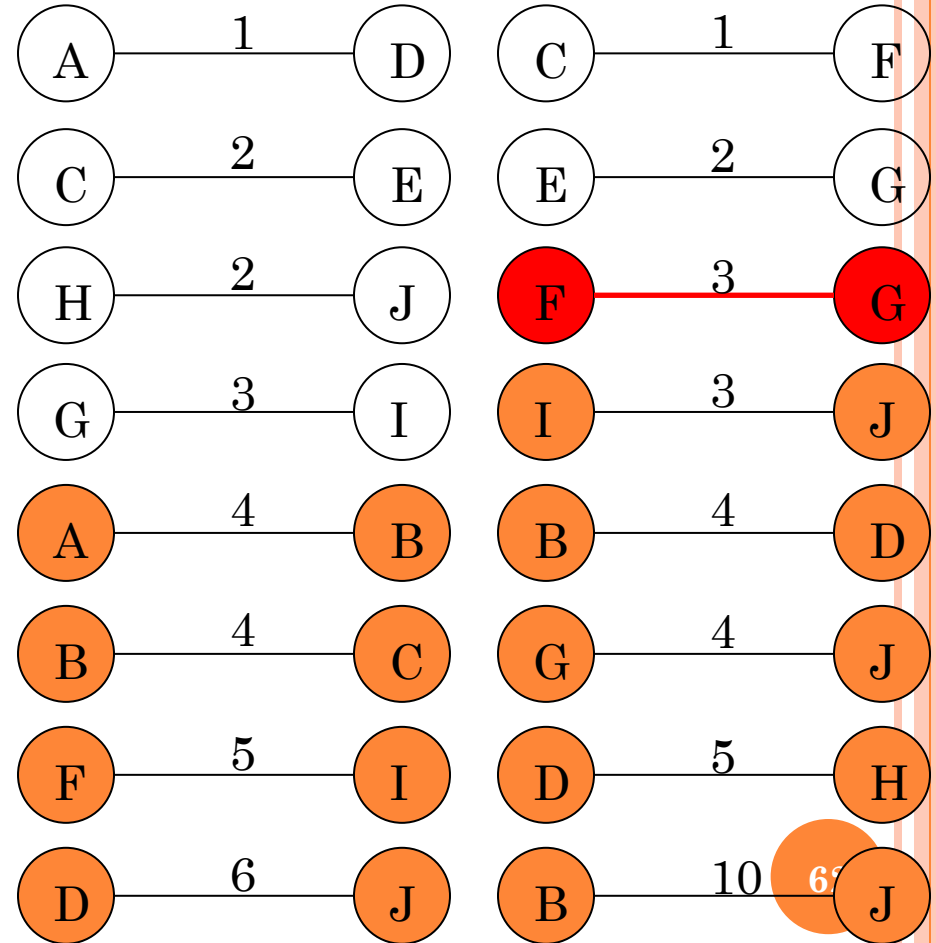
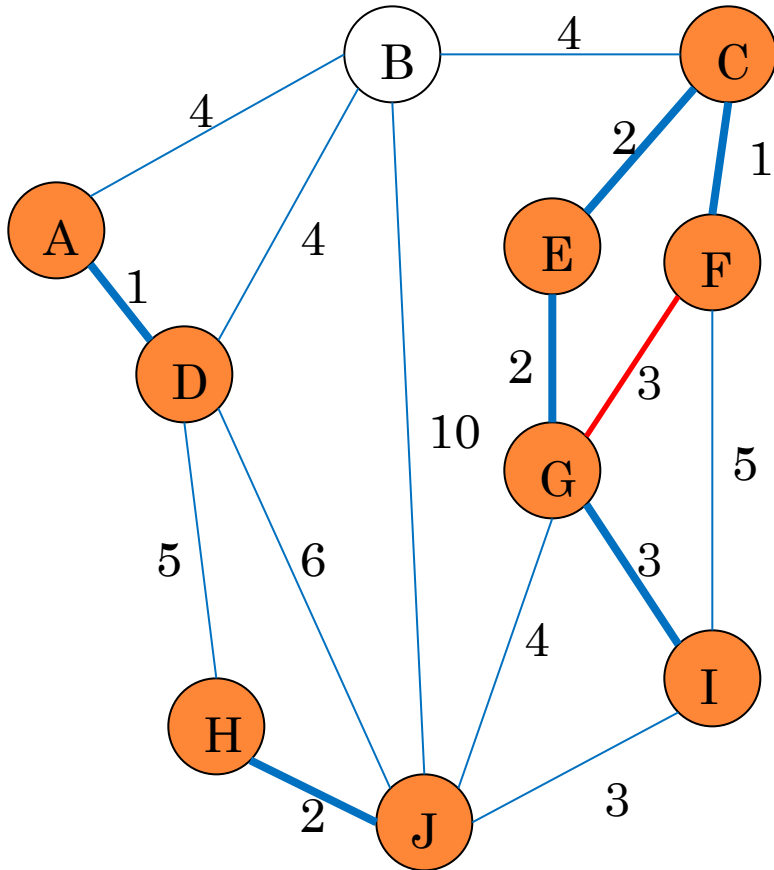
KRUSKAL'S ALGORITHM – EXAMPLE

○ Add Edge



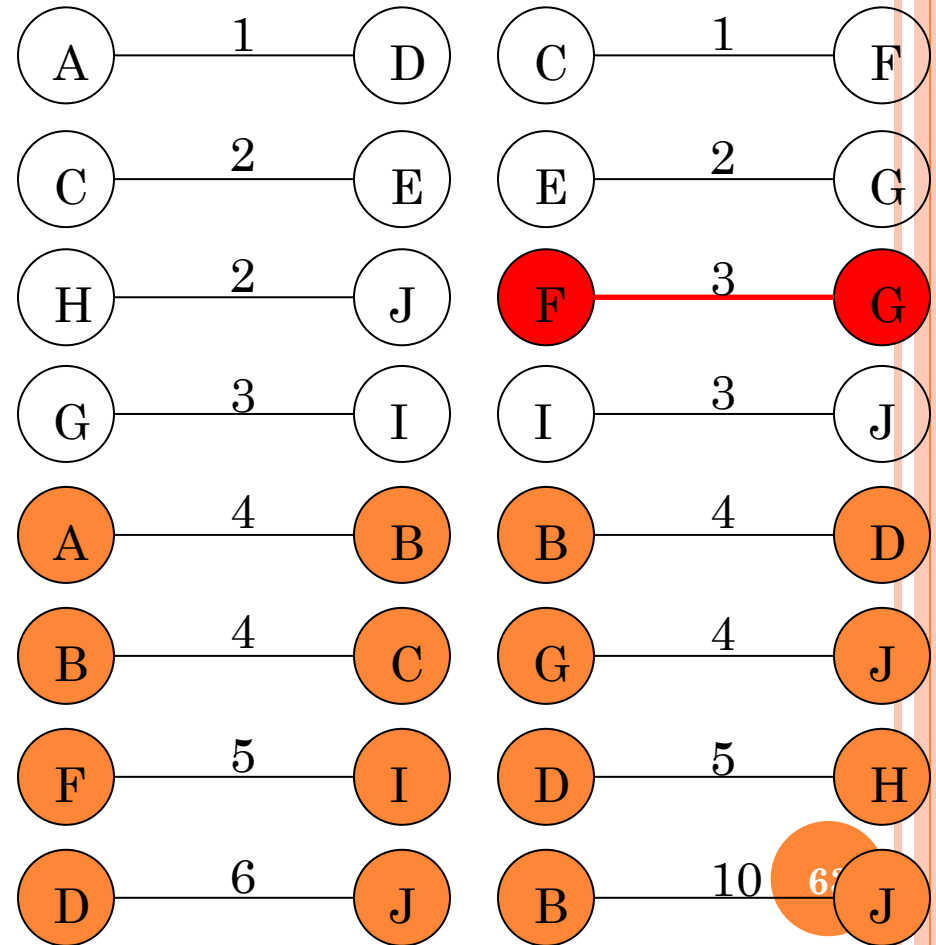
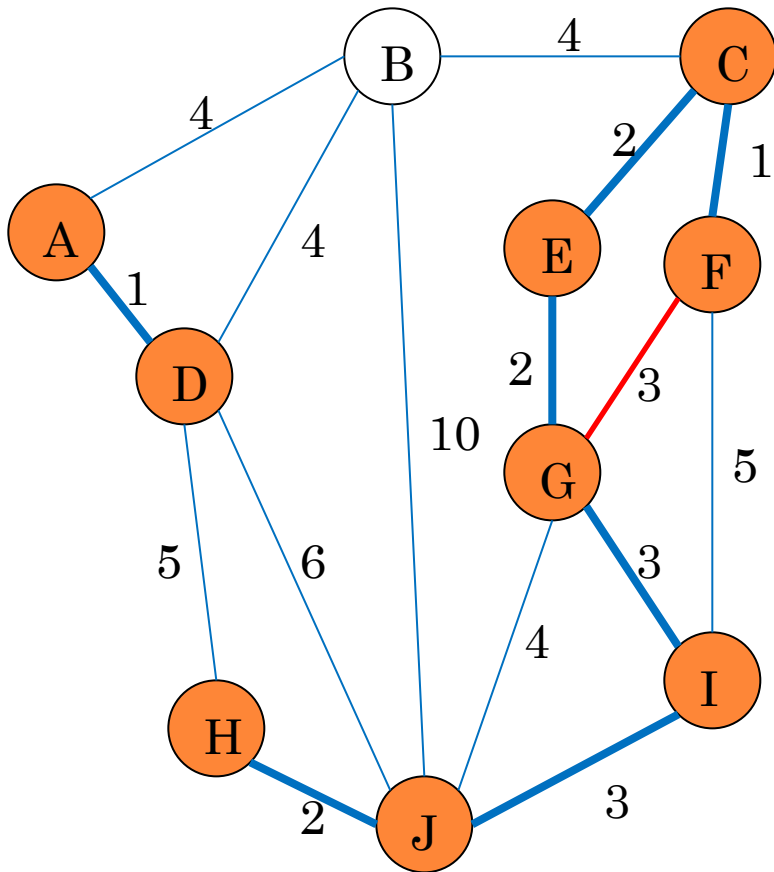
KRUSKAL'S ALGORITHM – EXAMPLE

○ Add Edge



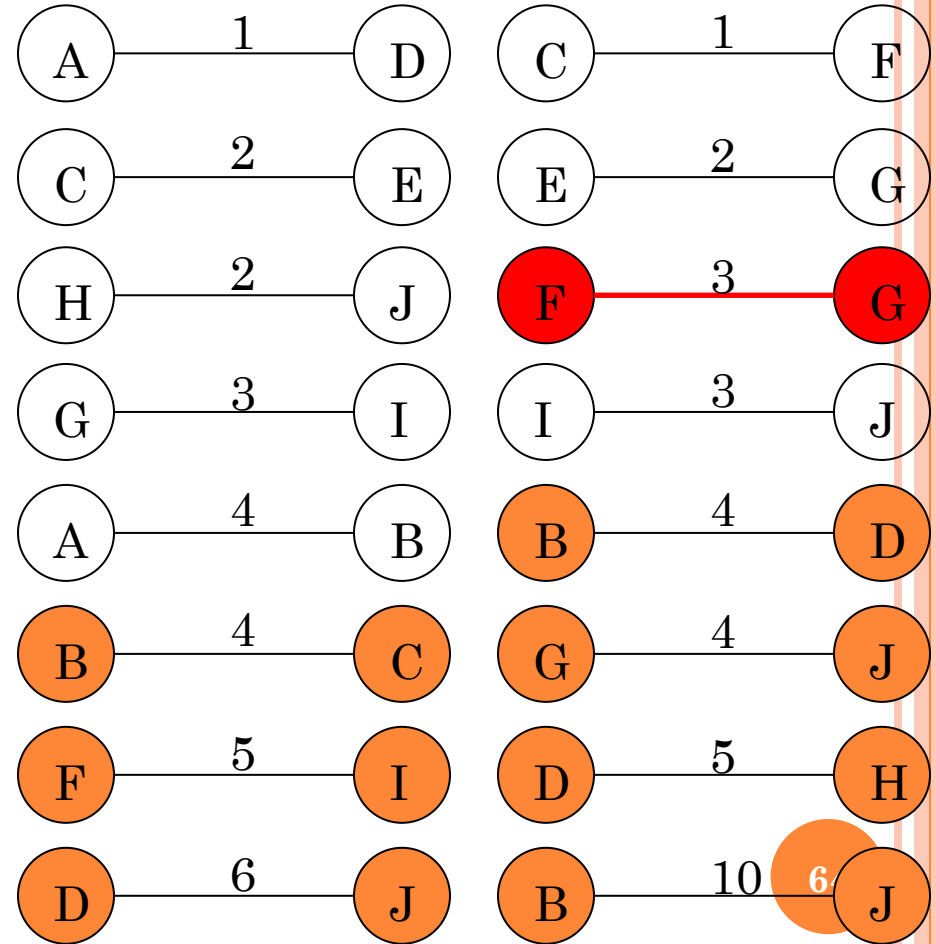
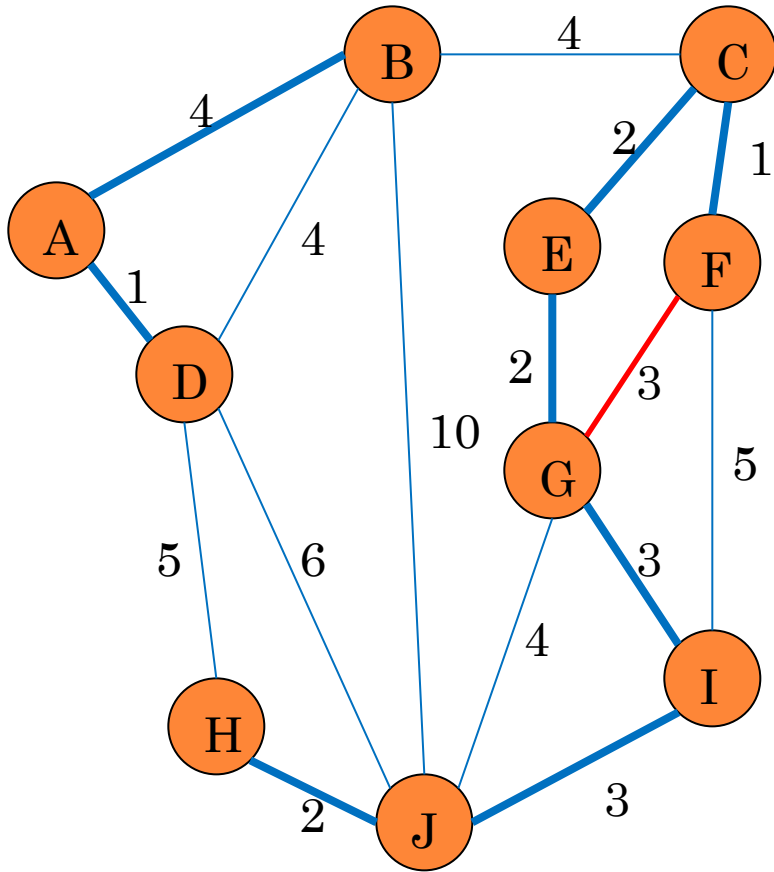
KRUSKAL'S ALGORITHM – EXAMPLE

○ Add Edge



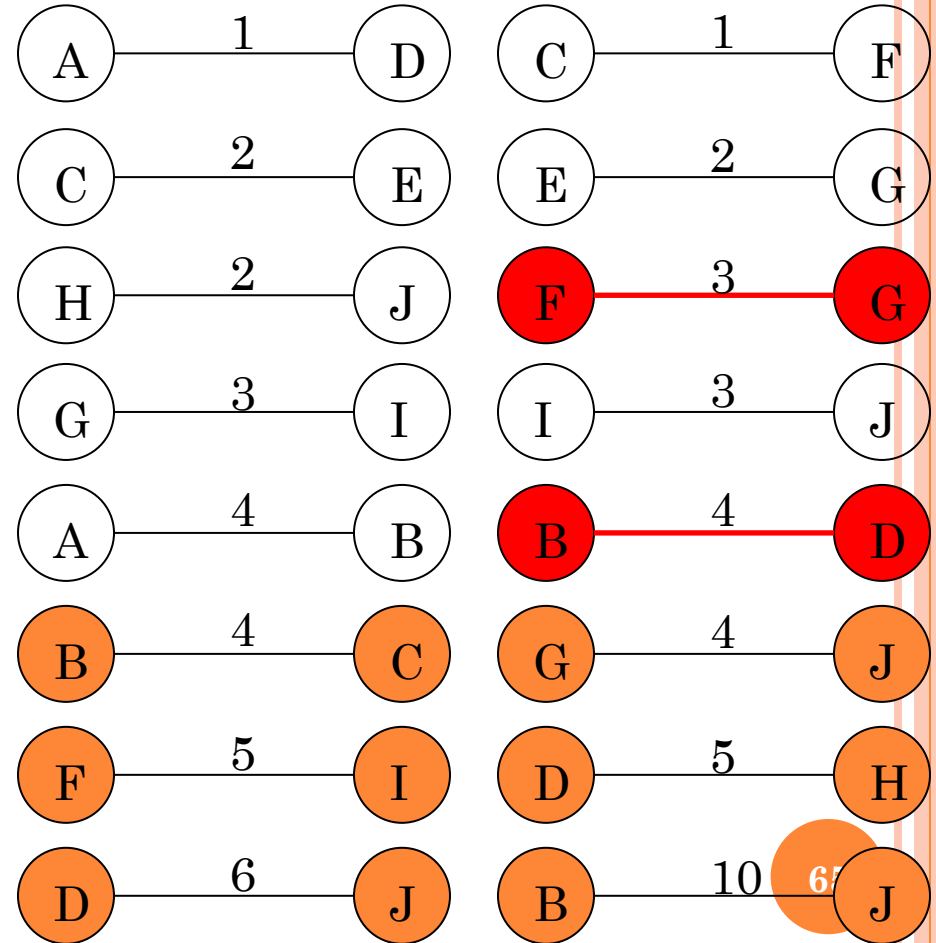
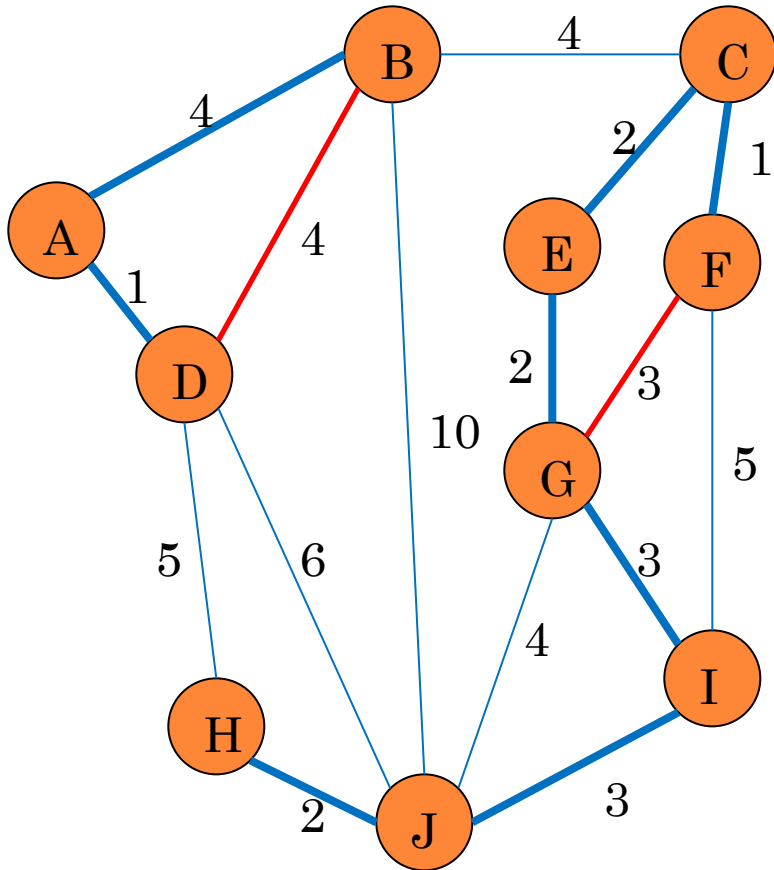
KRUSKAL'S ALGORITHM – EXAMPLE

○ Add Edge



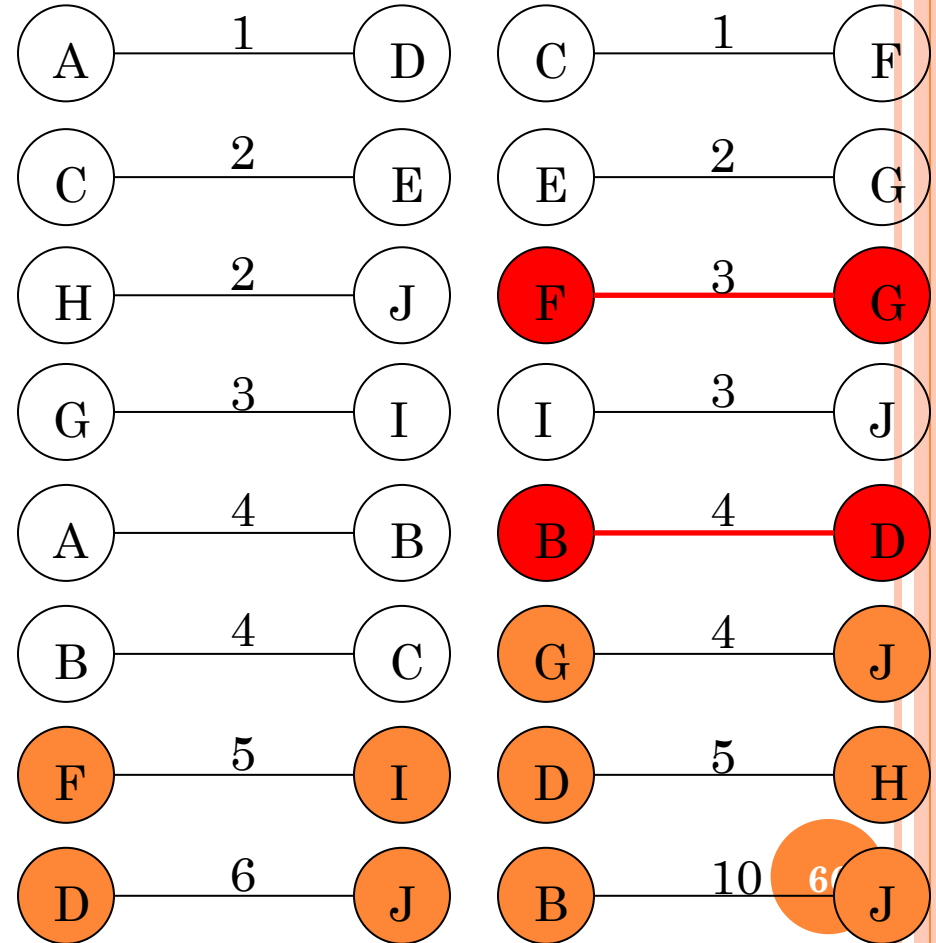
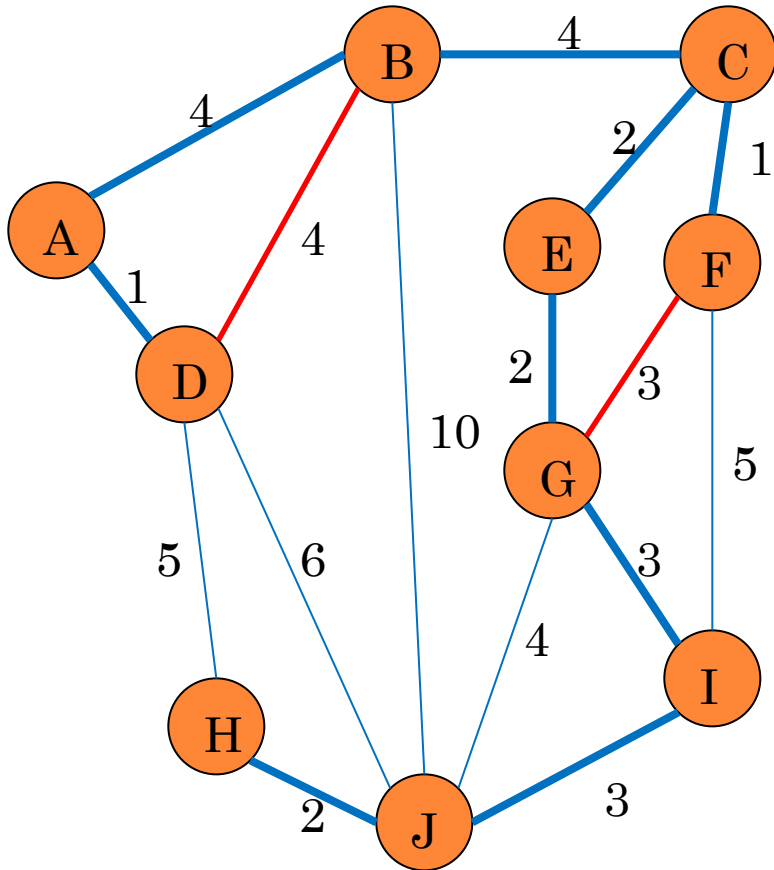
KRUSKAL'S ALGORITHM – EXAMPLE

○ Add Edge



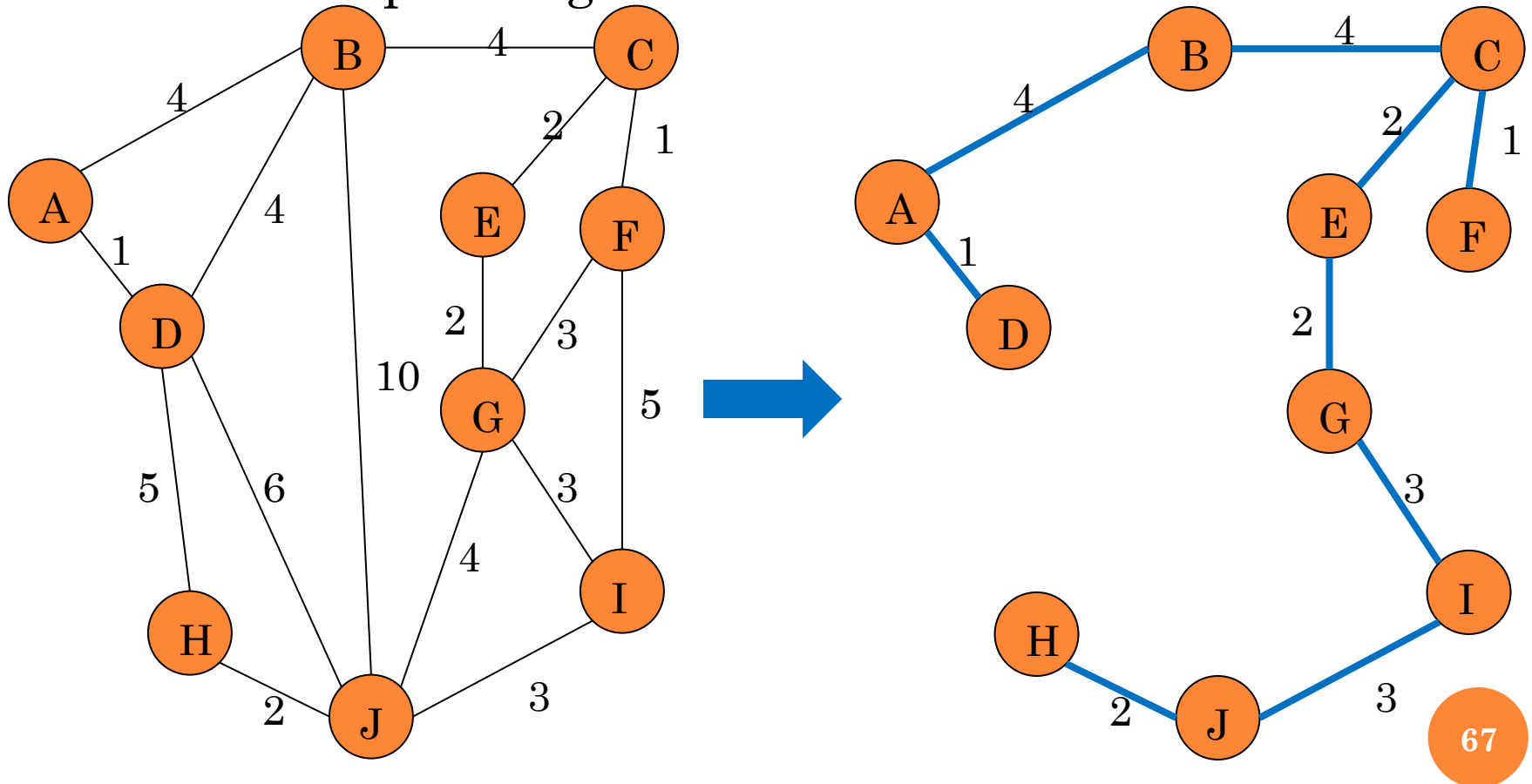
KRUSKAL'S ALGORITHM – EXAMPLE

○ Add Edge



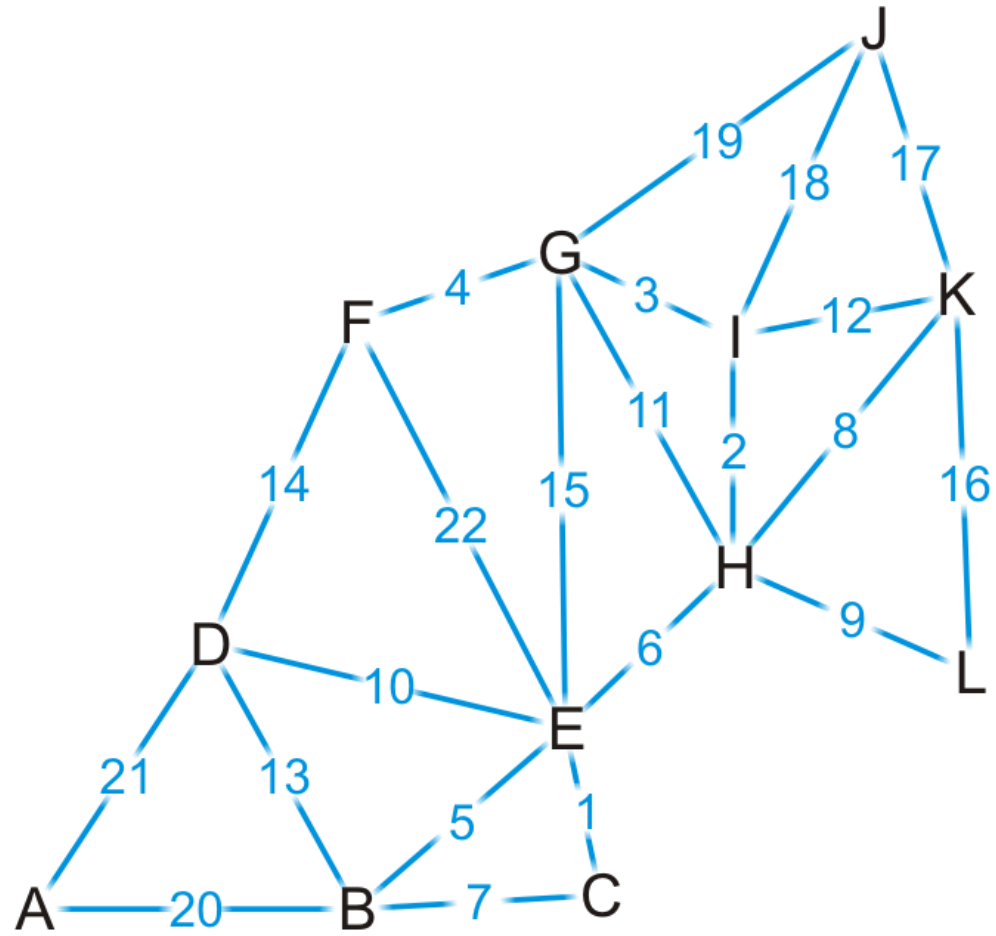
KRUSKAL'S ALGORITHM – EXAMPLE

- Minimum spanning tree

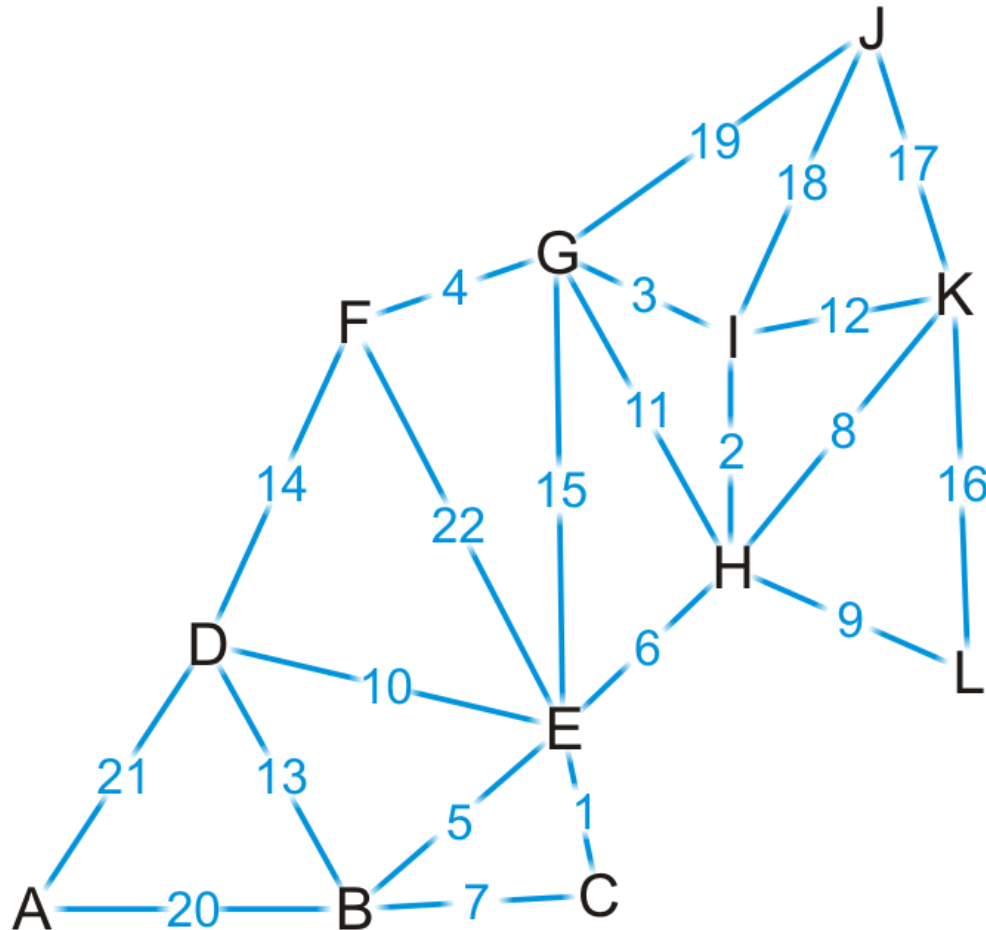


KRUSKAL'S ALGORITHM – EXAMPLE

- Complete graph

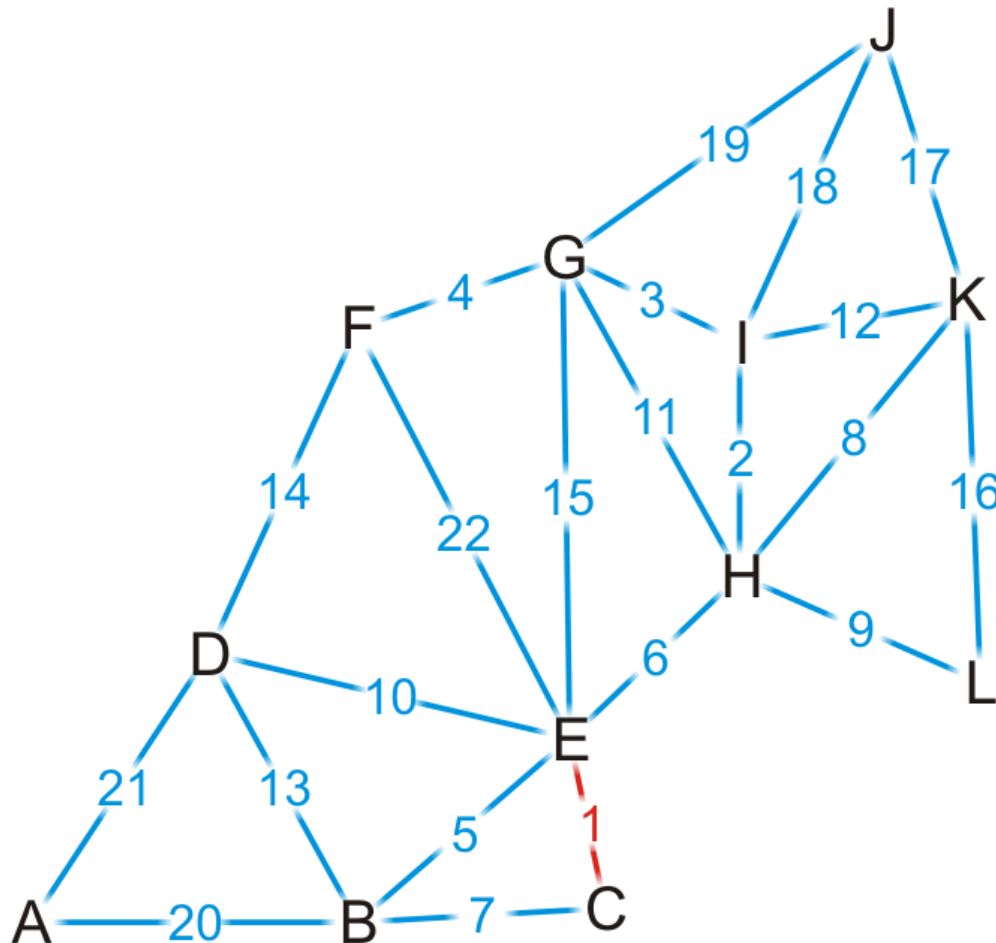


KRUSKAL'S ALGORITHM – EXAMPLE



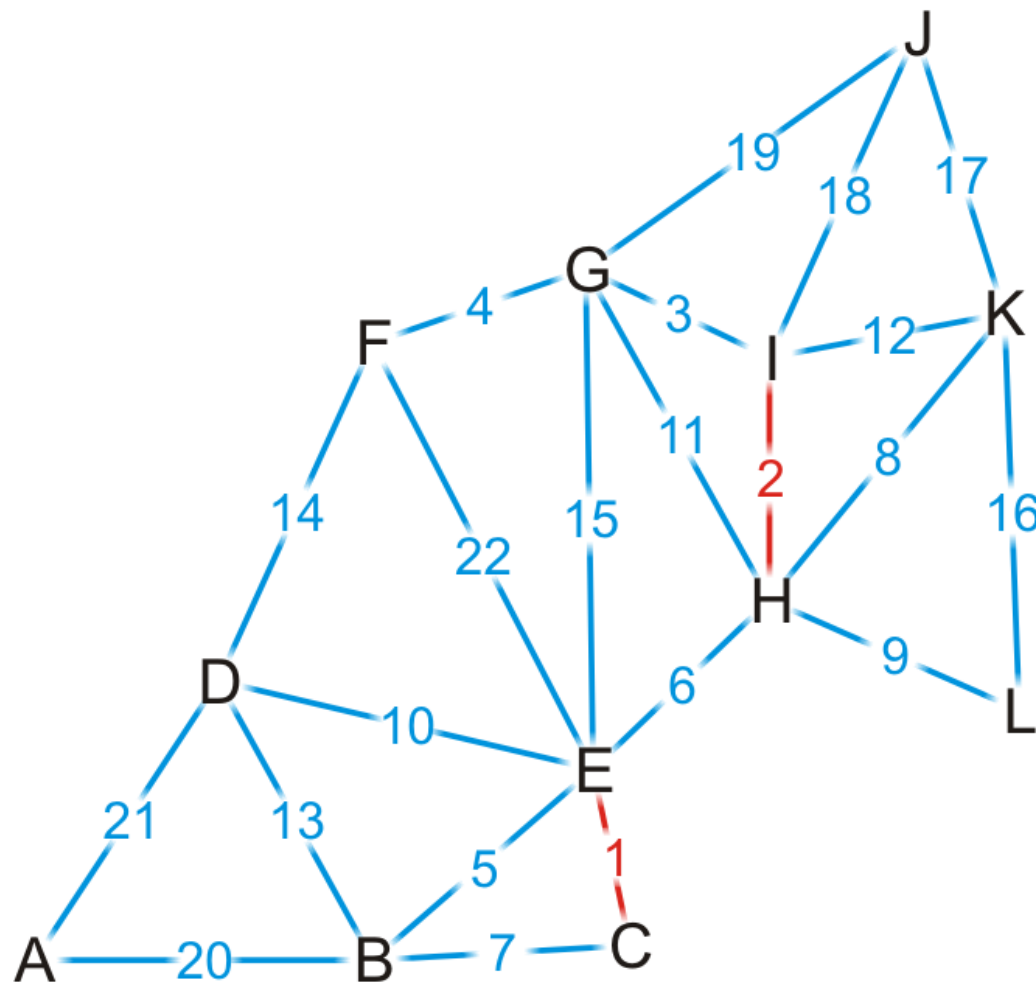
{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

KRUSKAL'S ALGORITHM – EXAMPLE → {C, E}



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

KRUSKAL'S ALGORITHM – EXAMPLE

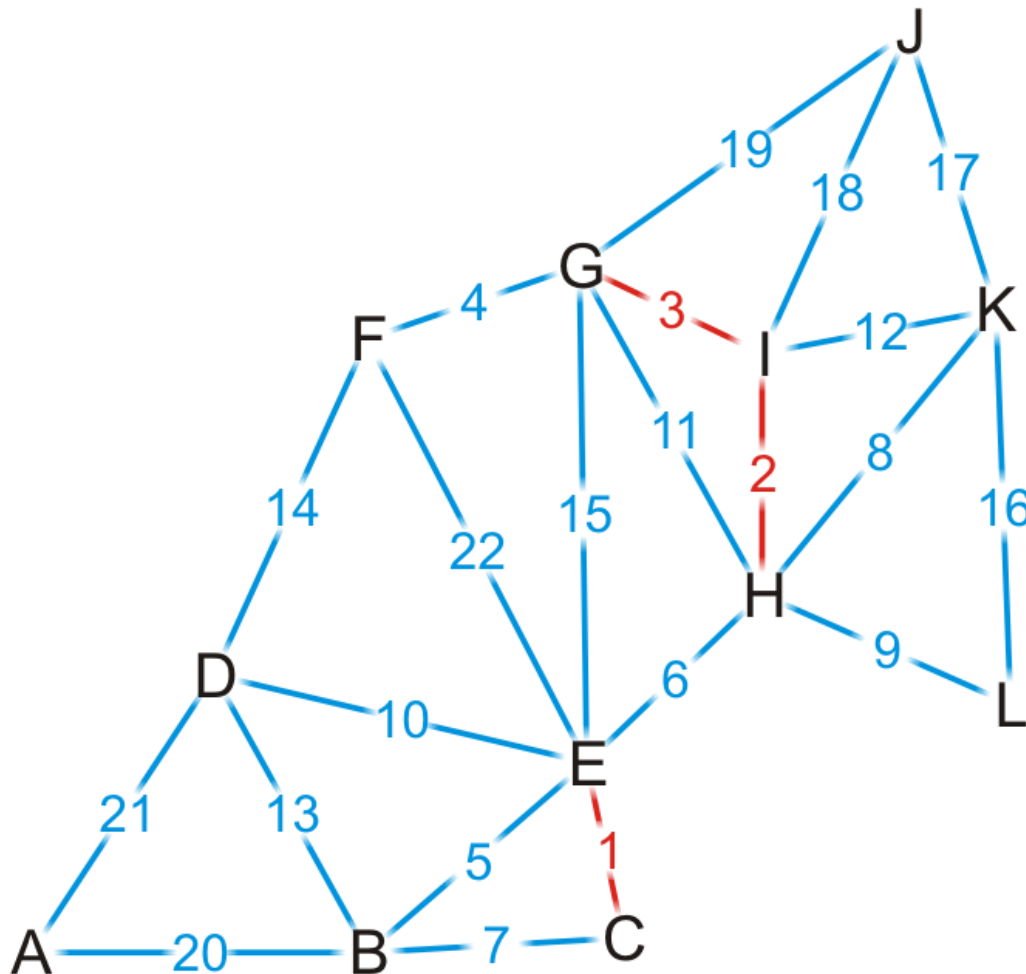


→

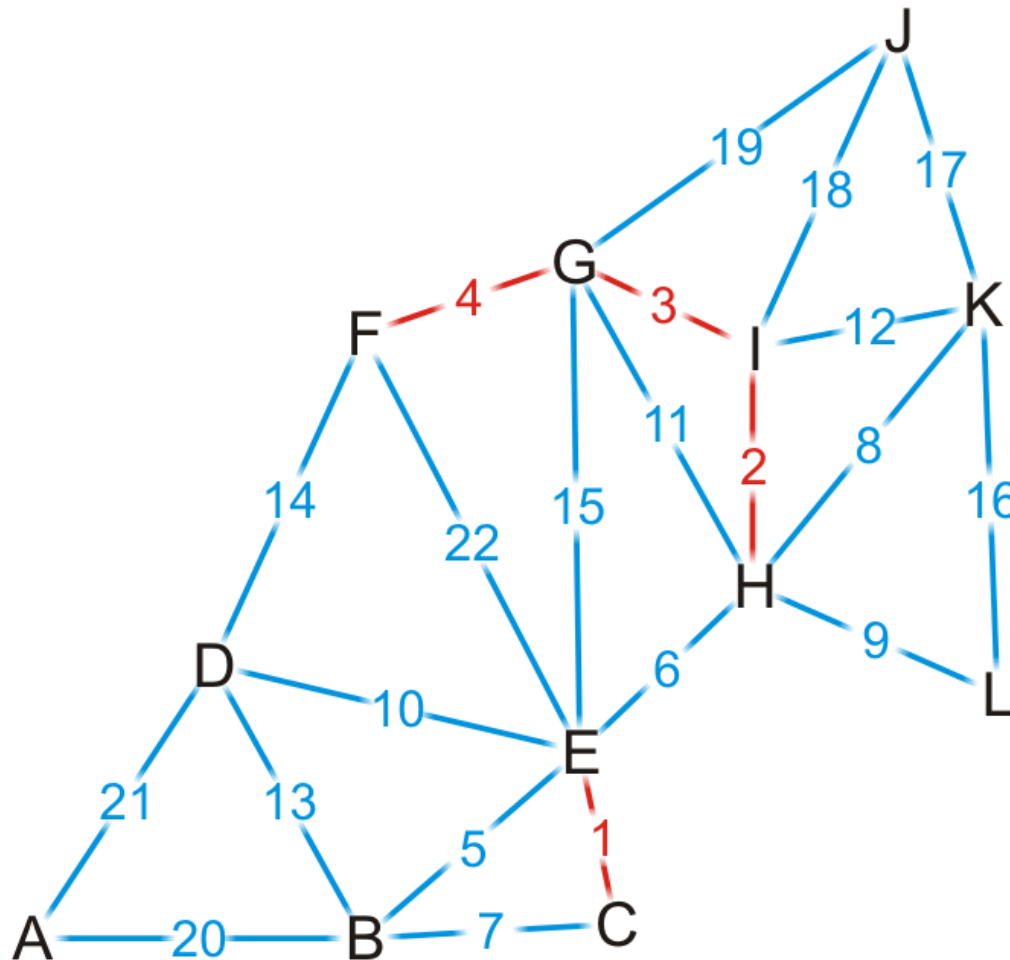
- {C, E}
- {H, I}
- {G, I}
- {F, G}
- {B, E}
- {E, H}
- {B, C}
- {H, K}
- {H, L}
- {D, E}
- {G, H}
- {I, K}
- {B, D}
- {D, F}
- {E, G}
- {K, L}
- {J, K}
- {J, I}
- {J, G}
- {A, B}
- {A, D}
- {E, F}

KRUSKAL'S ALGORITHM – EXAMPLE

-
- $\{C, E\}$
 - $\{H, I\}$
 - $\{G, I\}$
 - $\{F, G\}$
 - $\{B, E\}$
 - $\{E, H\}$
 - $\{B, C\}$
 - $\{H, K\}$
 - $\{H, L\}$
 - $\{D, E\}$
 - $\{G, H\}$
 - $\{I, K\}$
 - $\{B, D\}$
 - $\{D, F\}$
 - $\{E, G\}$
 - $\{K, L\}$
 - $\{J, K\}$
 - $\{J, I\}$
 - $\{J, G\}$
 - $\{A, B\}$
 - $\{A, D\}$
 - $\{E, F\}$

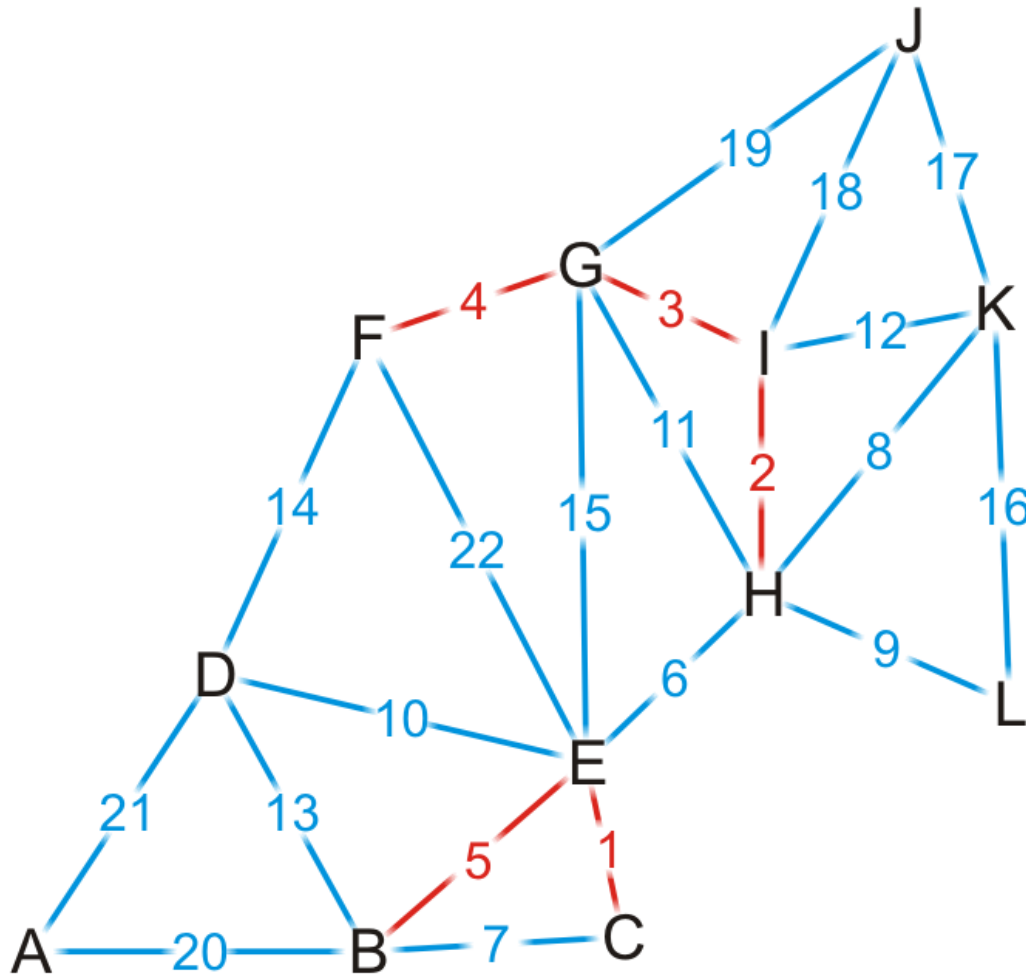


KRUSKAL'S ALGORITHM – EXAMPLE



$\{C, E\}$
 $\{H, I\}$
 $\{G, I\}$
 $\rightarrow \{F, G\}$
 $\{B, E\}$
 $\{E, H\}$
 $\{B, C\}$
 $\{H, K\}$
 $\{H, L\}$
 $\{D, E\}$
 $\{G, H\}$
 $\{I, K\}$
 $\{B, D\}$
 $\{D, F\}$
 $\{E, G\}$
 $\{K, L\}$
 $\{J, K\}$
 $\{J, I\}$
 $\{J, G\}$
 $\{A, B\}$
 $\{A, D\}$
 $\{E, F\}$

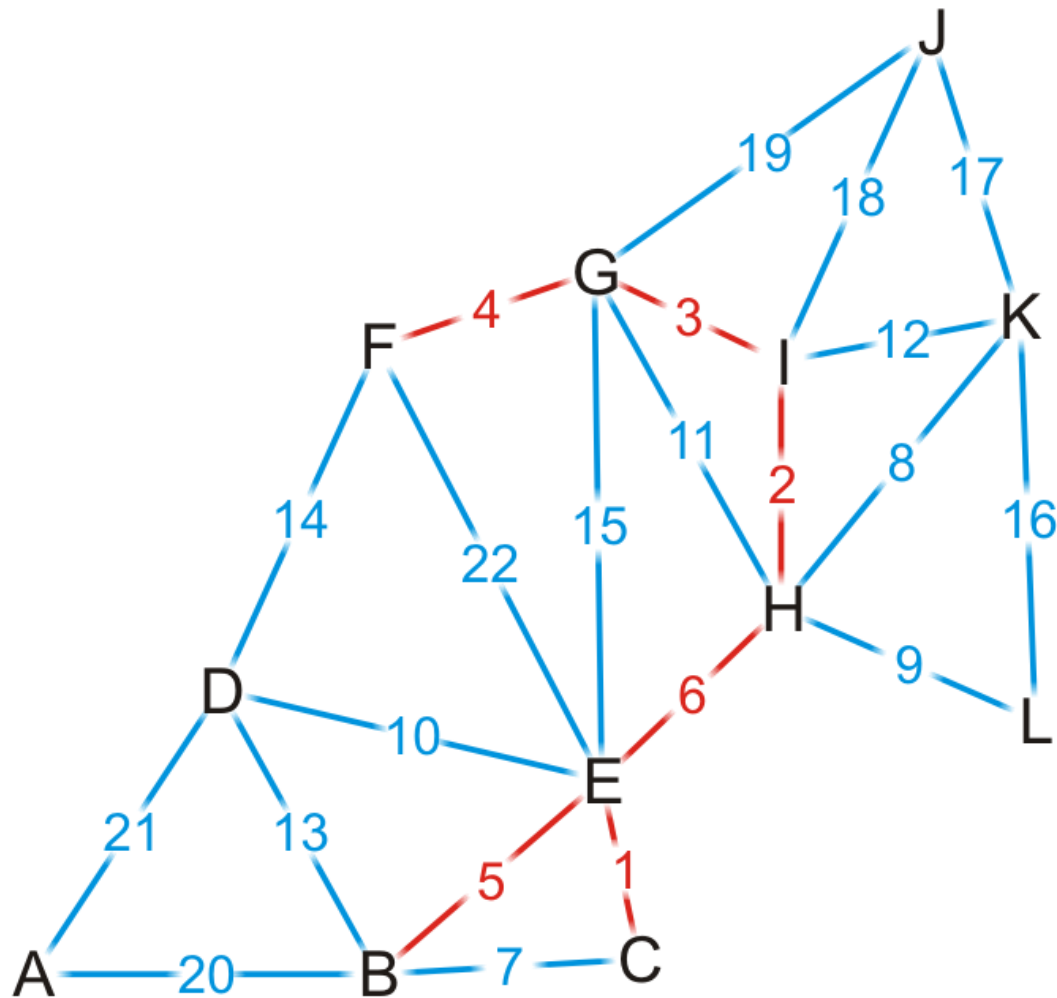
KRUSKAL'S ALGORITHM – EXAMPLE



$\{C, E\}$
 $\{H, I\}$
 $\{G, I\}$
 $\{F, G\}$
 $\{B, E\}$
 $\{E, H\}$
 $\{B, C\}$
 $\{H, K\}$
 $\{H, L\}$
 $\{D, E\}$
 $\{G, H\}$
 $\{I, K\}$
 $\{B, D\}$
 $\{D, F\}$
 $\{E, G\}$
 $\{K, L\}$
 $\{J, K\}$
 $\{J, I\}$
 $\{J, G\}$
 $\{A, B\}$
 $\{A, D\}$
 $\{E, F\}$

KRUSKAL'S ALGORITHM – EXAMPLE

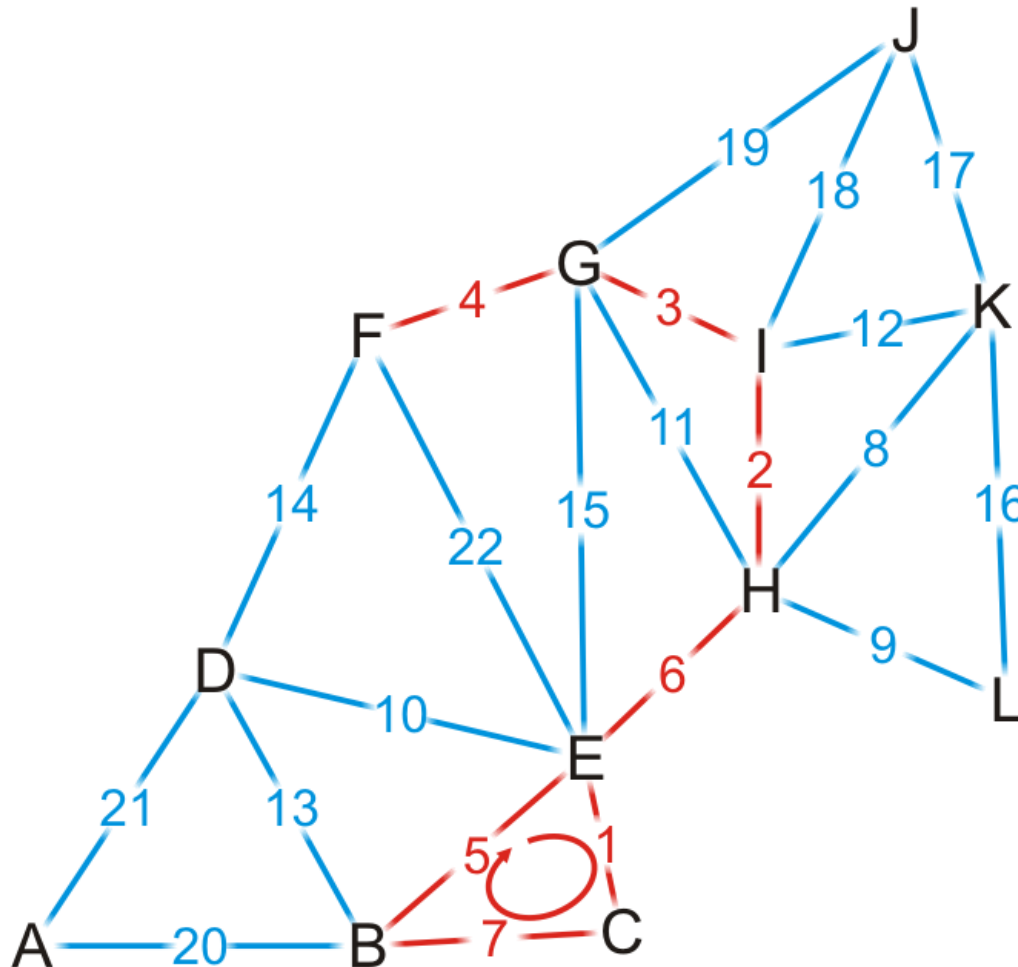
- We add edge {E, H}
 - This coalesces the two spanning sub-trees into one



~~{C, E}~~
~~{H, I}~~
~~{G, I}~~
~~{F, G}~~
~~{B, E}~~
→ ~~{E, H}~~
~~{B, C}~~
~~{H, K}~~
~~{H, L}~~
~~{D, E}~~
~~{G, H}~~
~~{I, K}~~
~~{B, D}~~
~~{D, F}~~
~~{E, G}~~
~~{K, L}~~
~~{J, K}~~
~~{J, I}~~
~~{J, G}~~
~~{A, B}~~
~~{A, D}~~
~~{E, F}~~

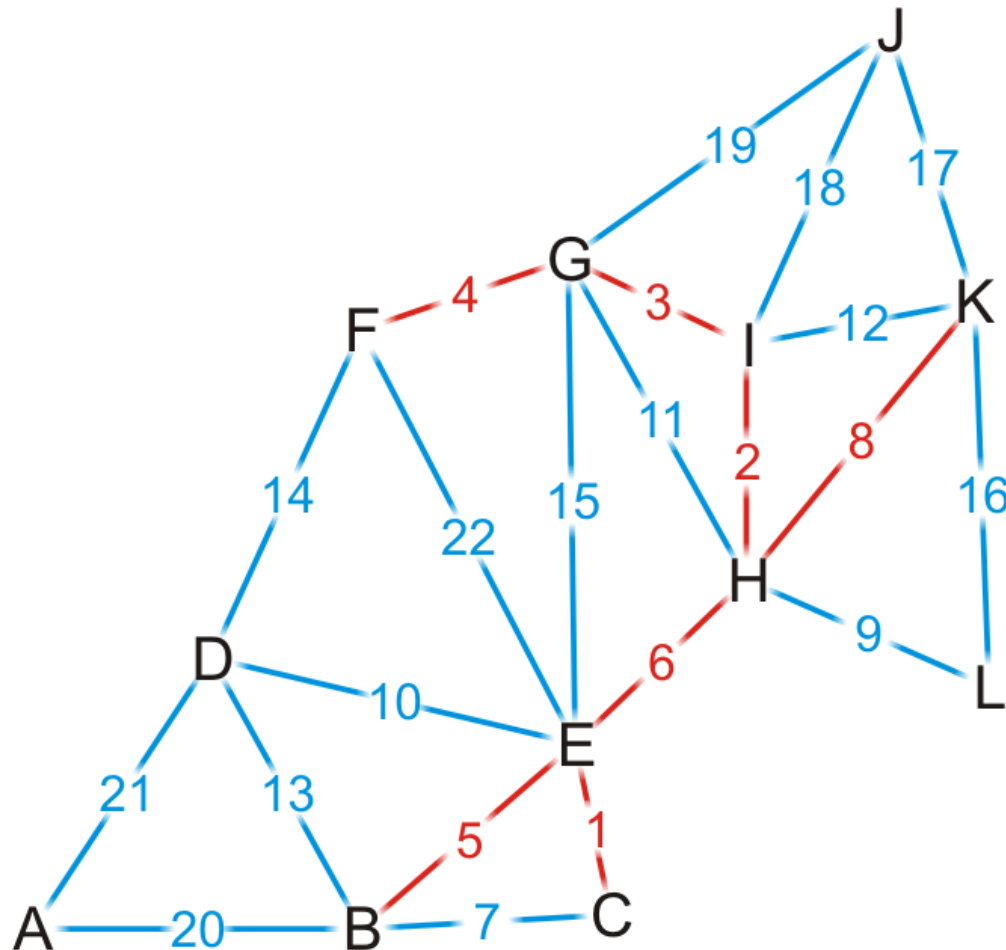
KRUSKAL'S ALGORITHM – EXAMPLE

- We try adding {B, C}, but it creates a cycle



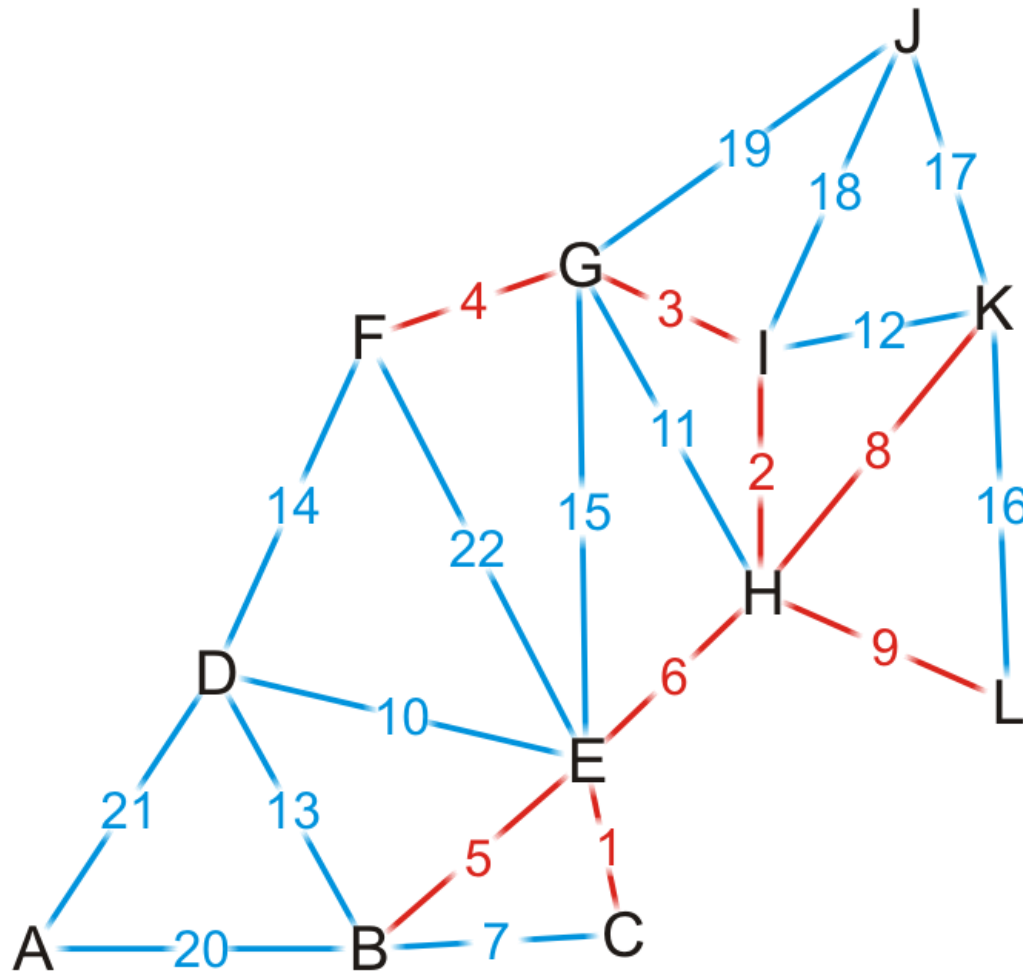
$\{\mathbf{C}, \mathbf{E}\}$
 $\{\mathbf{H}, \mathbf{I}\}$
 $\{\mathbf{G}, \mathbf{I}\}$
 $\{\mathbf{F}, \mathbf{G}\}$
 $\{\mathbf{B}, \mathbf{E}\}$
 $\{\mathbf{E}, \mathbf{H}\}$
 $\{\mathbf{B}, \mathbf{C}\}$
 $\{\mathbf{H}, \mathbf{K}\}$
 $\{\mathbf{H}, \mathbf{L}\}$
 $\{\mathbf{D}, \mathbf{E}\}$
 $\{\mathbf{G}, \mathbf{H}\}$
 $\{\mathbf{I}, \mathbf{K}\}$
 $\{\mathbf{B}, \mathbf{D}\}$
 $\{\mathbf{D}, \mathbf{F}\}$
 $\{\mathbf{E}, \mathbf{G}\}$
 $\{\mathbf{K}, \mathbf{L}\}$
 $\{\mathbf{J}, \mathbf{K}\}$
 $\{\mathbf{J}, \mathbf{I}\}$
 $\{\mathbf{J}, \mathbf{G}\}$
 $\{\mathbf{A}, \mathbf{B}\}$
 $\{\mathbf{A}, \mathbf{D}\}$
 $\{\mathbf{E}, \mathbf{F}\}$

KRUSKAL'S ALGORITHM – EXAMPLE



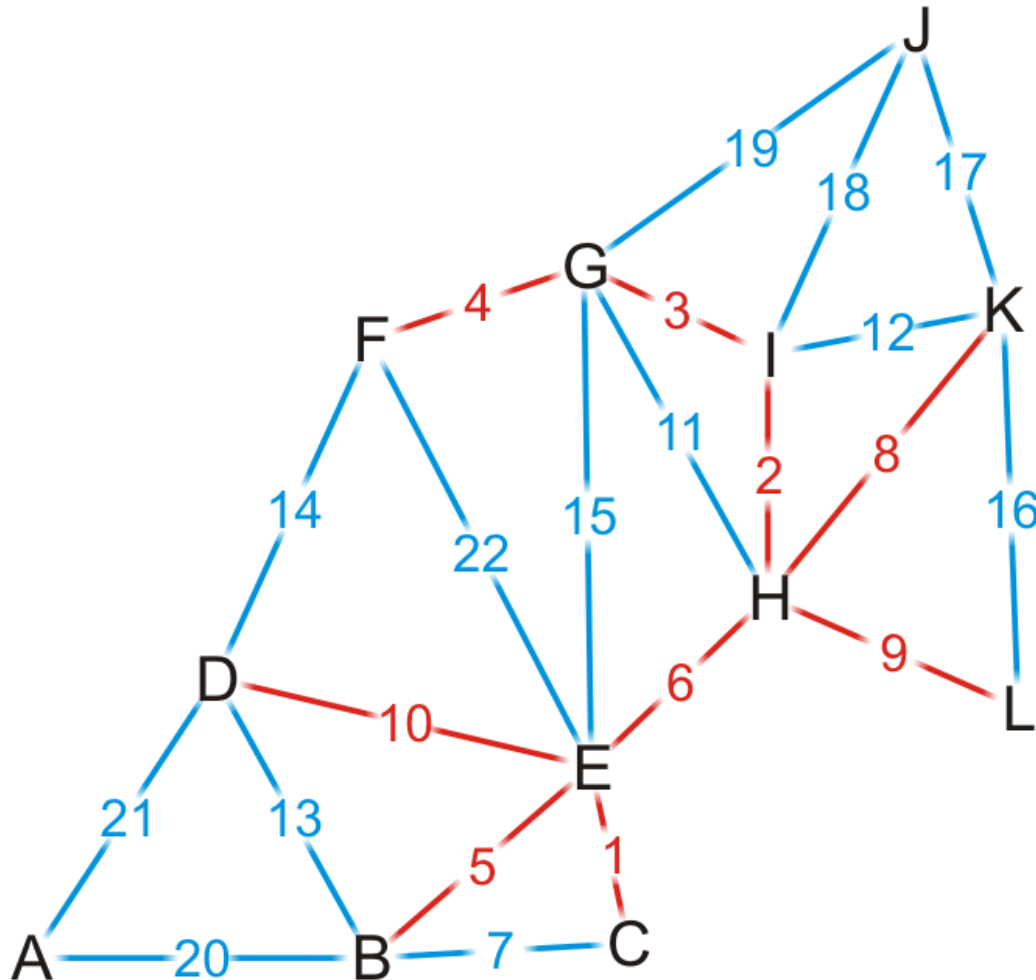
$\{C, E\}$
 $\{H, I\}$
 $\{G, I\}$
 $\{F, G\}$
 $\{B, E\}$
 $\{E, H\}$
 $\{B, C\}$
 $\{H, K\}$
 $\{H, L\}$
 $\{D, E\}$
 $\{G, H\}$
 $\{I, K\}$
 $\{B, D\}$
 $\{D, F\}$
 $\{E, G\}$
 $\{K, L\}$
 $\{J, K\}$
 $\{J, I\}$
 $\{J, G\}$
 $\{A, B\}$
 $\{A, D\}$
 $\{E, F\}$

KRUSKAL'S ALGORITHM – EXAMPLE



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

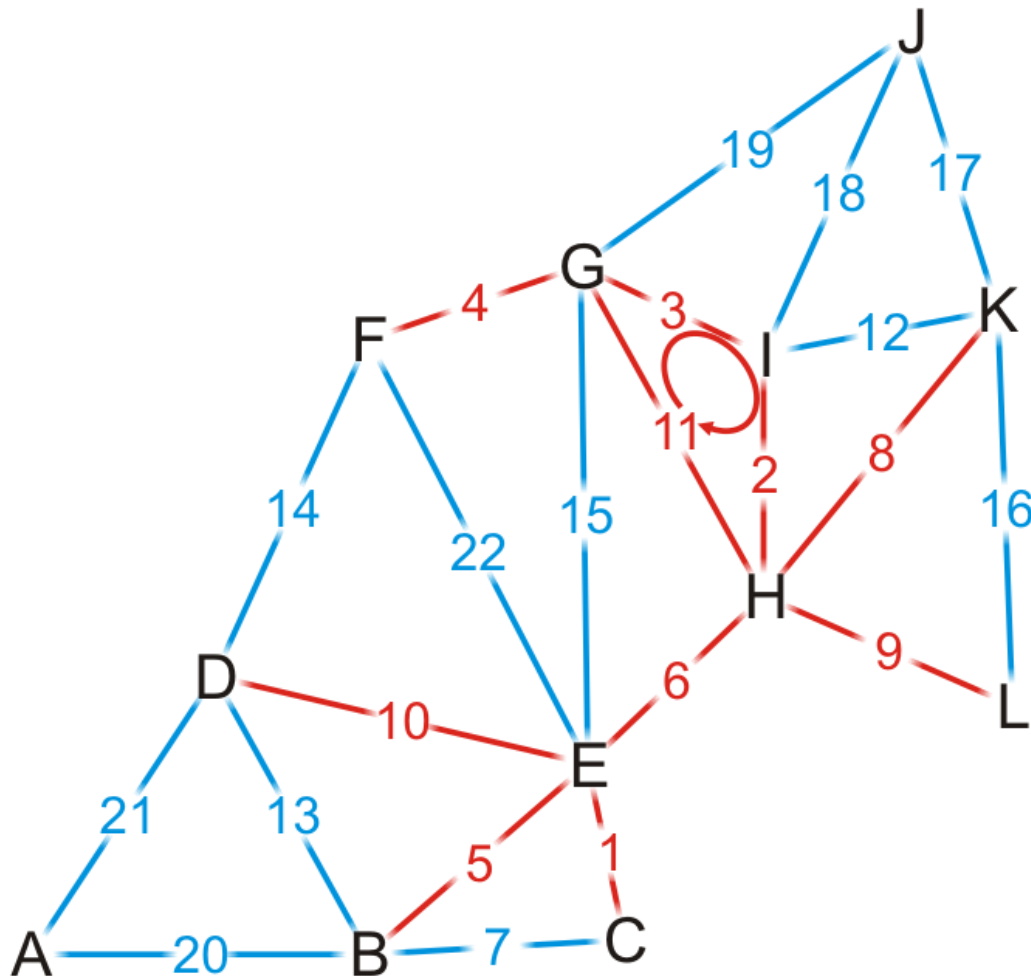
KRUSKAL'S ALGORITHM – EXAMPLE



$\{C, E\}$
 $\{H, I\}$
 $\{G, I\}$
 $\{F, G\}$
 $\{B, E\}$
 $\{E, H\}$
 $\{B, C\}$
 $\{H, K\}$
 $\{H, L\}$
 $\{D, E\}$
 $\{G, H\}$
 $\{I, K\}$
 $\{B, D\}$
 $\{D, F\}$
 $\{E, G\}$
 $\{K, L\}$
 $\{J, K\}$
 $\{J, I\}$
 $\{J, G\}$
 $\{A, B\}$
 $\{A, D\}$
 $\{E, F\}$

KRUSKAL'S ALGORITHM – EXAMPLE

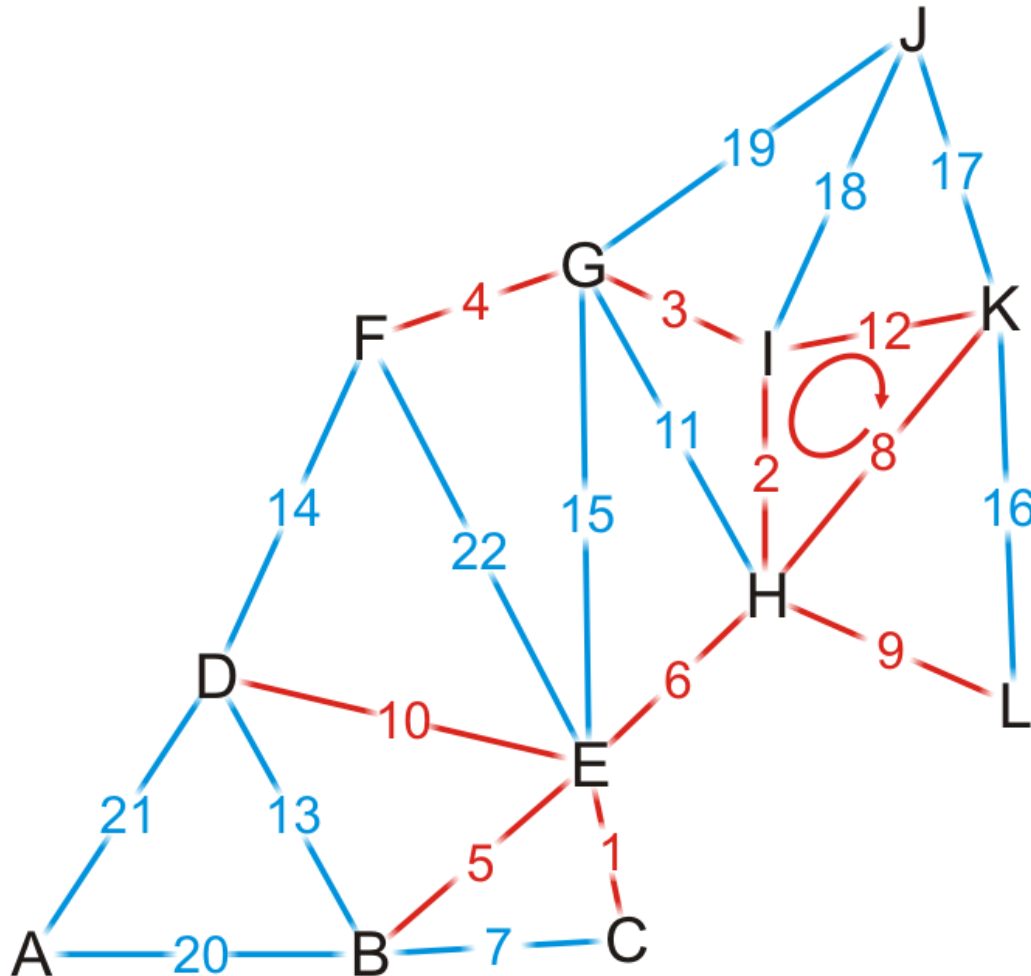
- We try adding {G, H}, but it creates a cycle



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
→ {G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

KRUSKAL'S ALGORITHM – EXAMPLE

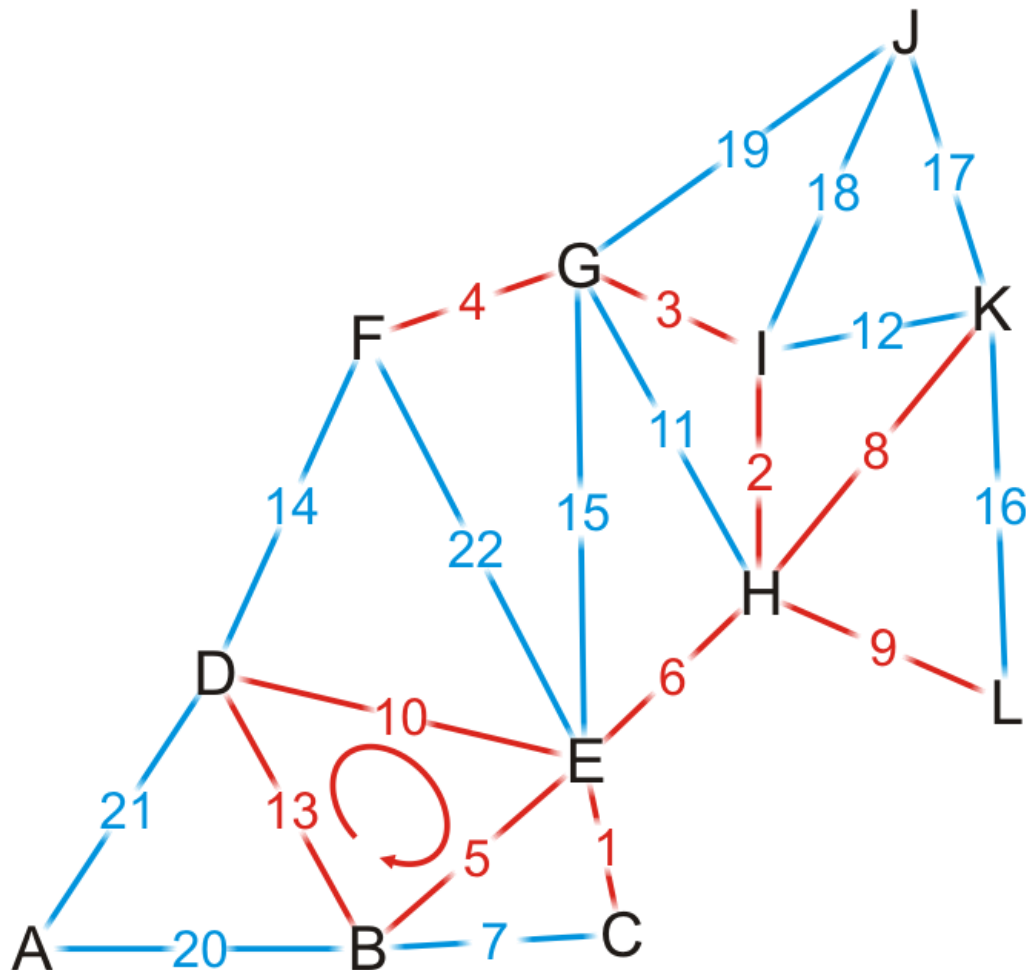
- We try adding {I, K}, but it creates a cycle



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
→ {I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

KRUSKAL'S ALGORITHM – EXAMPLE

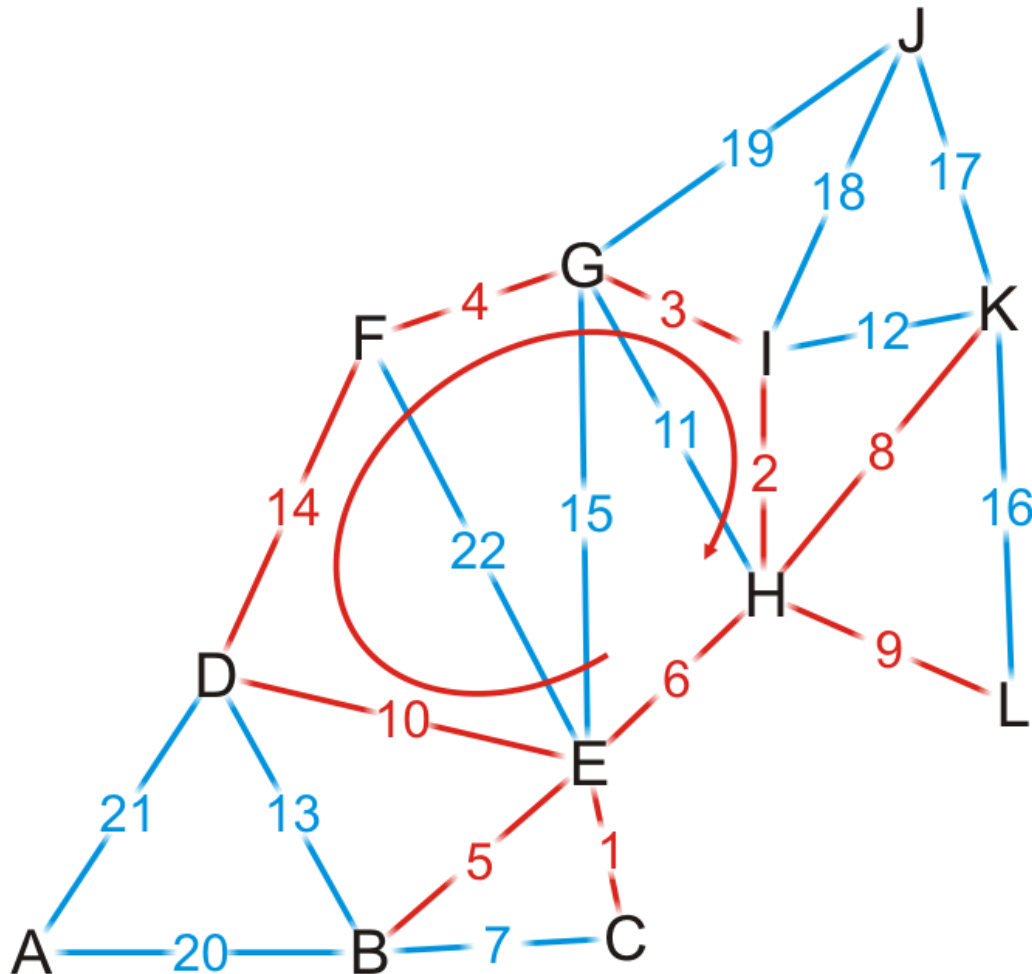
- We try adding {B, D}, but it creates a cycle



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
→ {B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

KRUSKAL'S ALGORITHM – EXAMPLE

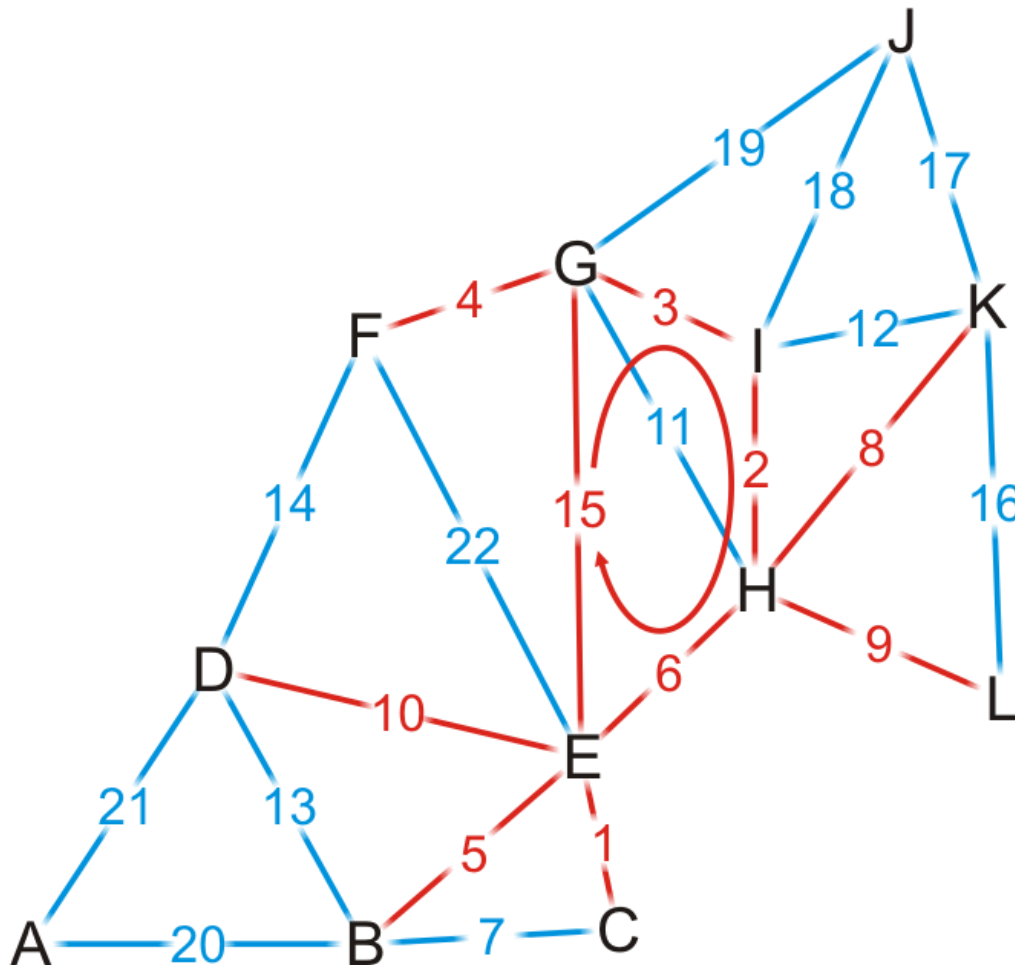
- We try adding {D, F}, but it creates a cycle



~~{C, E}~~
~~{H, I}~~
~~{G, I}~~
~~{F, G}~~
~~{B, E}~~
~~{E, H}~~
~~{B, C}~~
~~{H, K}~~
~~{H, L}~~
~~{D, E}~~
~~{G, H}~~
~~{I, K}~~
~~{B, D}~~
→ {D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, B}
{A, D}
{E, F}

KRUSKAL'S ALGORITHM – EXAMPLE

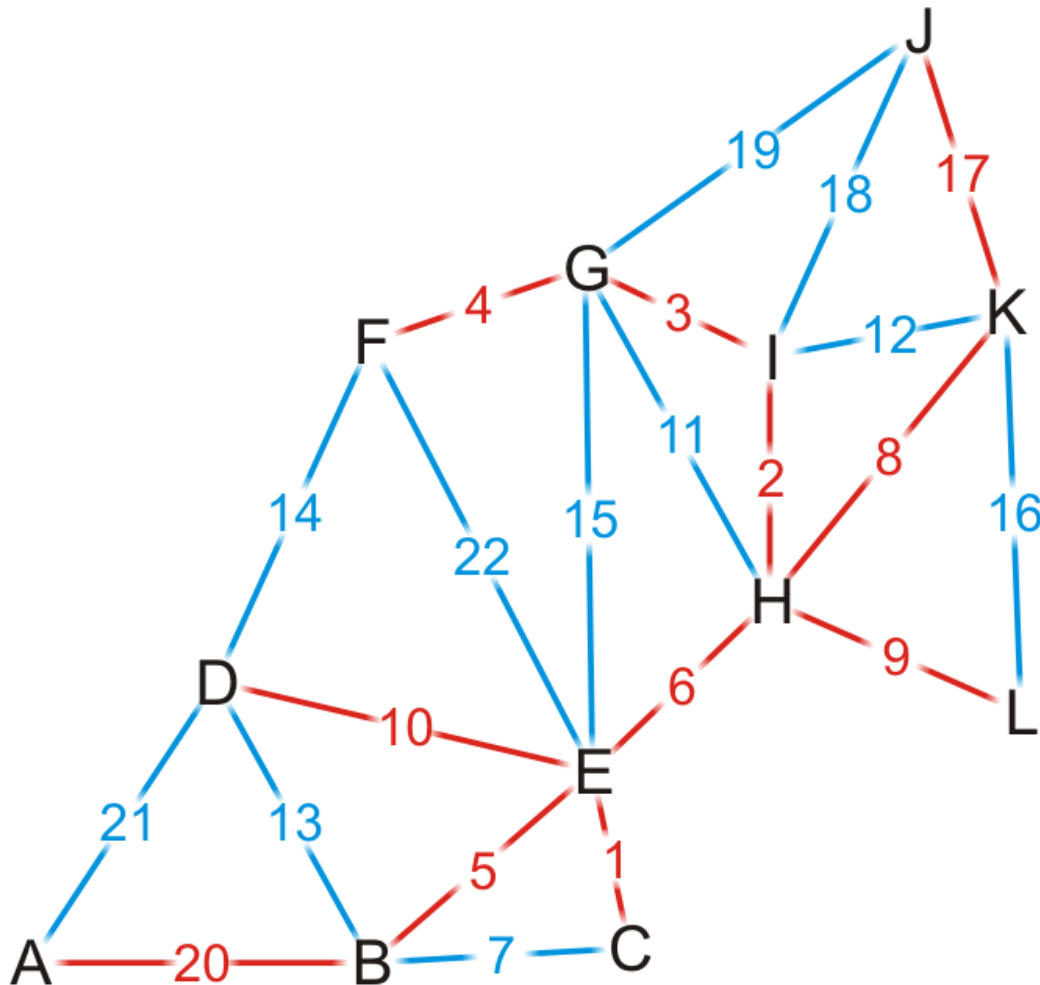
- We try adding $\{E, G\}$, but it creates a cycle



{C, E}
 {H, I}
 {G, I}
 {F, G}
 {B, E}
 {E, H}
 {B, C}
 {H, K}
 {H, L}
 {D, E}
 {G, H}
 {I, K}
 {B, D}
 {D, F}
 {E, G}
 {K, L}
 {J, K}
 {J, I}
 {J, G}
 {A, B}
 {A, D}
 {E, F}

KRUSKAL'S ALGORITHM – EXAMPLE

- By observation, we can still add edges $\{J, K\}$ and $\{A, B\}$

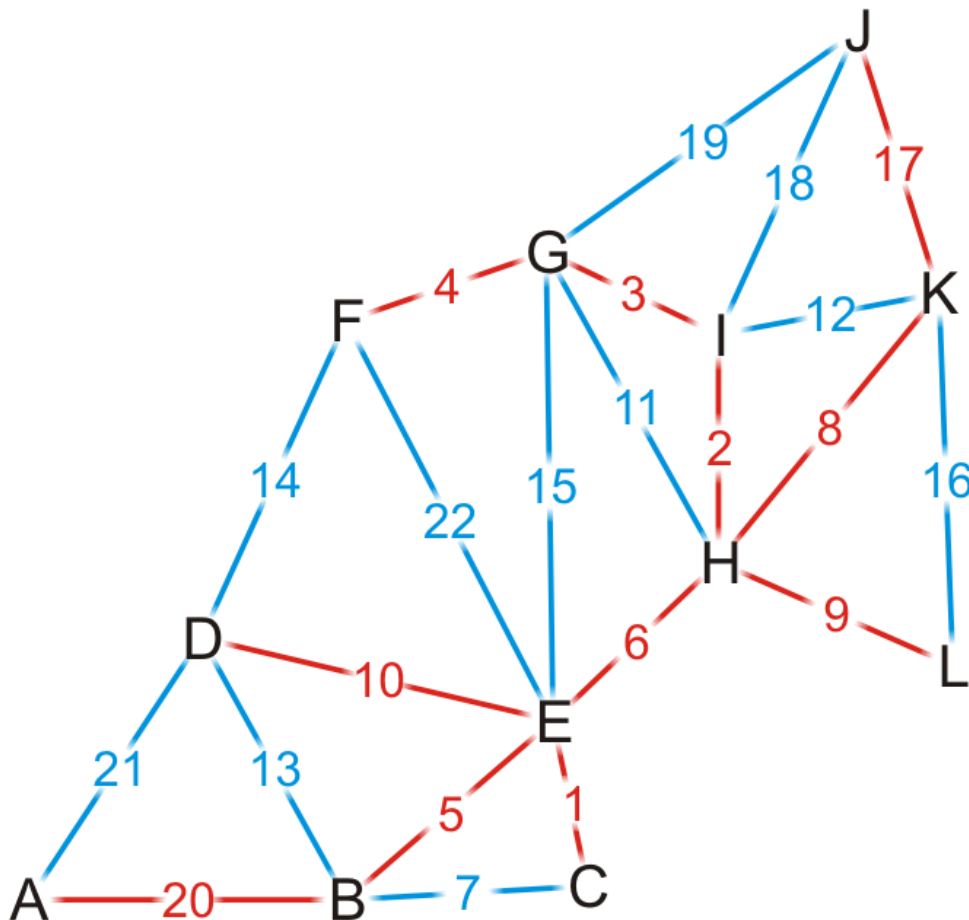


{C, E}
 {H, I}
 {G, I}
 {F, G}
 {B, E}
 {E, H}
 {B, C}
 {H, K}
 {H, L}
 {D, E}
 {G, H}
 {I, K}
 {B, D}
 {D, F}
 {E, G}
 {K, L}
 {J, K}
 {J, I}
 {J, G}
 {A, B}
 {A, D}
 {E, F}



KRUSKAL'S ALGORITHM – EXAMPLE

- Having added {A, B}, we now have 11 edges
 - We terminate the loop
 - We have our minimum spanning tree



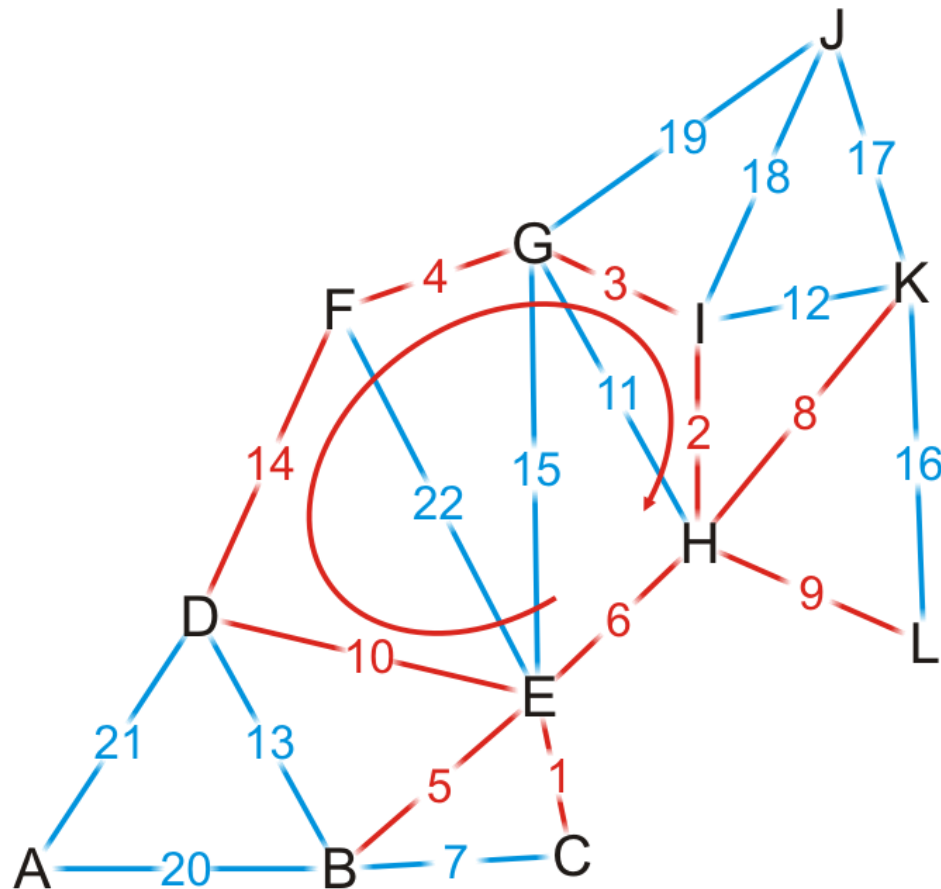
{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}



{A, B}
{A, D}
{E, F}

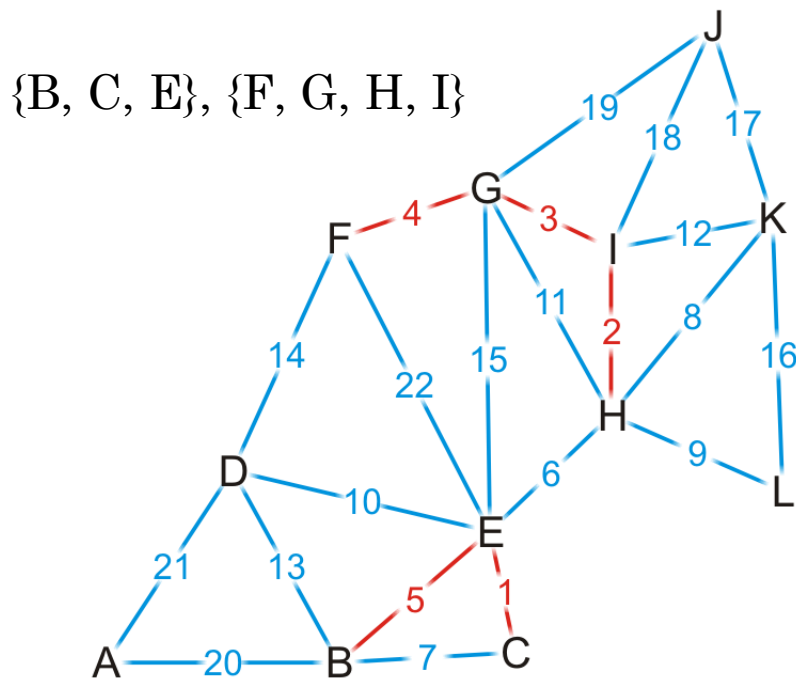
DETECTING A CYCLE

- To determine if a cycle is created, we could perform a traversal
 - A run-time of $O(|V|)$



DETECTING A CYCLE – DISJOINT SETS

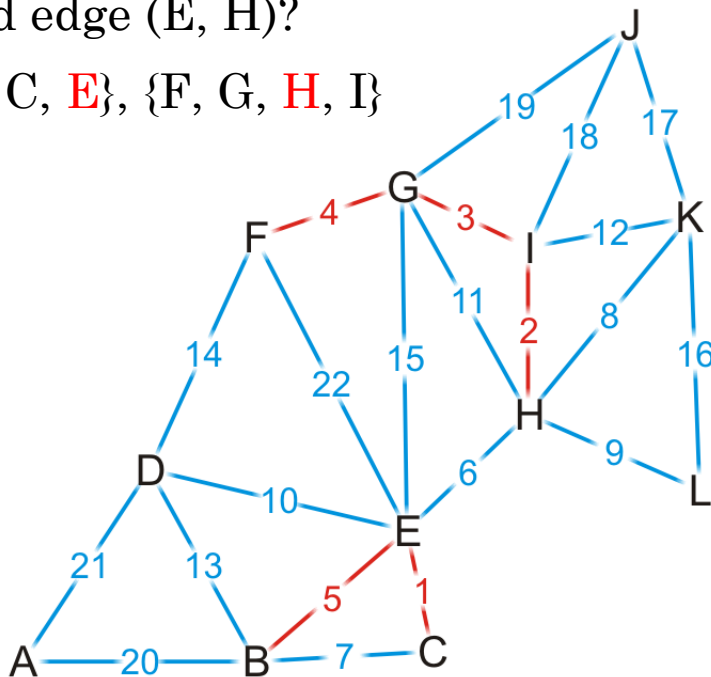
- Consider edges in the same connected sub-graph as forming a set



DETECTING A CYCLE – DISJOINT SETS

- Consider edges in the same connected sub-graph as forming a set
- If the vertices of the next edge are in different sets
 - Take the union of the two sets

Add edge (E, H)?

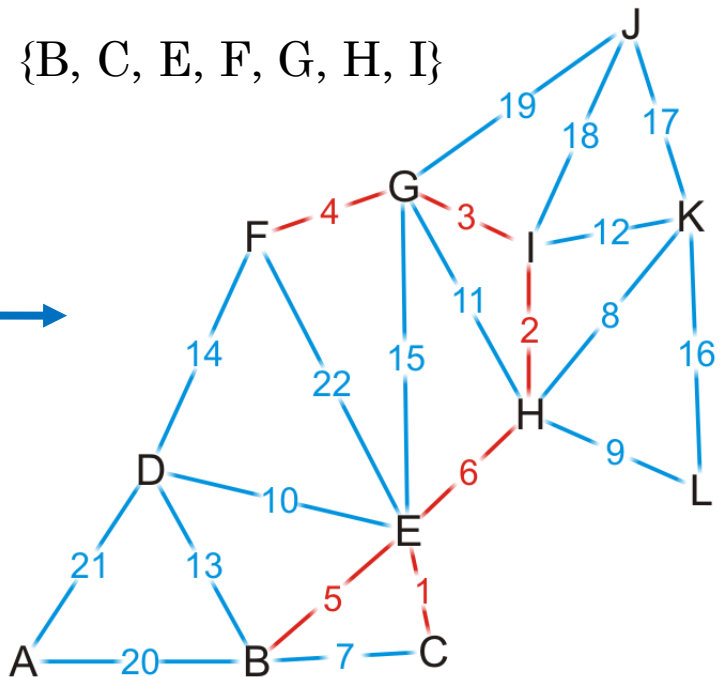
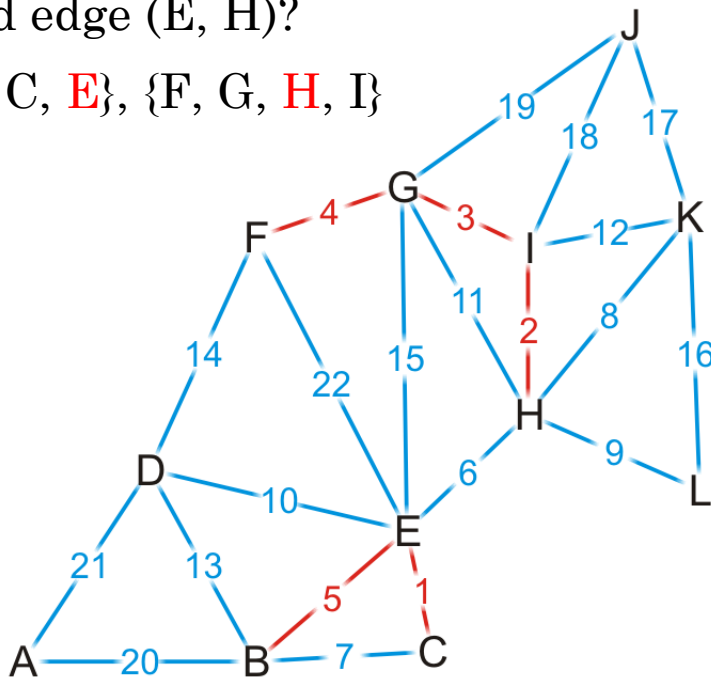
 $\{\mathbf{B}, \mathbf{C}, \mathbf{E}\}, \{\mathbf{F}, \mathbf{G}, \mathbf{H}, \mathbf{I}\}$ 

DETECTING A CYCLE – DISJOINT SETS

- Consider edges in the same connected sub-graph as forming a set
- If the vertices of the next edge are in different sets
 - Take the union of the two sets

Add edge (E, H)?

{B, C, **E**}, {F, G, **H**, I}

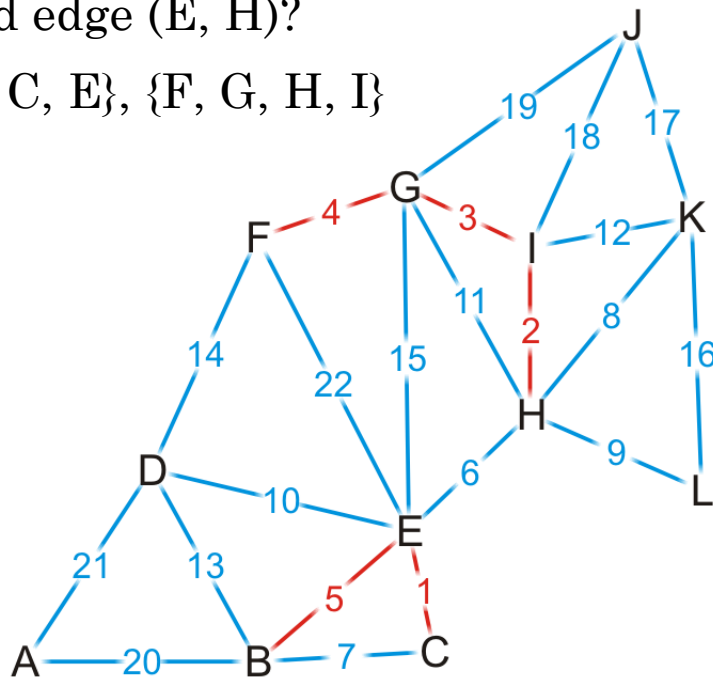


DETECTING A CYCLE – DISJOINT SETS

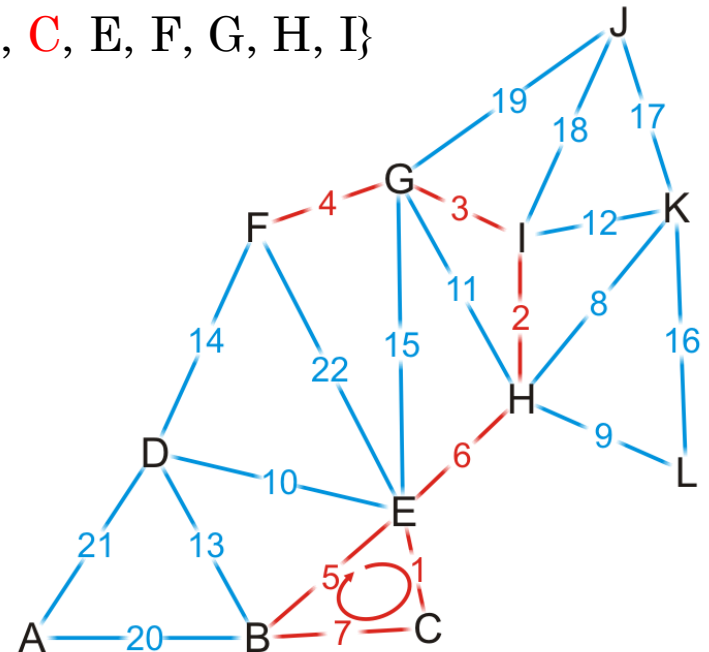
- Consider edges in the same connected sub-graph as forming a set
- If the vertices of the next edge are in different sets
 - Take the union of the two sets
- Do not add an edge if both vertices are in the same set

Add edge (E, H)?

{B, C, E}, {F, G, H, I}



{B, C, E, F, G, H, I}



KRUSKAL(V, E, w)

1. $A \leftarrow \emptyset$
 2. **for** each vertex $v \in V$
 3. **do** MAKE-SET(v)
 4. sort E into non-decreasing order by w
 5. **for** each (u, v) taken from the sorted list
 6. **do if** FIND-SET(u) $\neq$ FIND-SET(v)
 7. **then** $A \leftarrow A \cup \{(u, v)\}$
 8. UNION(u, v)
 9. **return** A
- Complexity annotations:
- $\left. \begin{array}{l} \text{lines 2-3} \end{array} \right\} O(V)$
 - $\left. \begin{array}{l} \text{lines 4-5} \end{array} \right\} O(E \lg E)$
 - $\left. \begin{array}{l} \text{lines 5-6} \end{array} \right\} \leftarrow O(E)$
 - $\left. \begin{array}{l} \text{lines 6-8} \end{array} \right\} \leftarrow O(\lg V)$

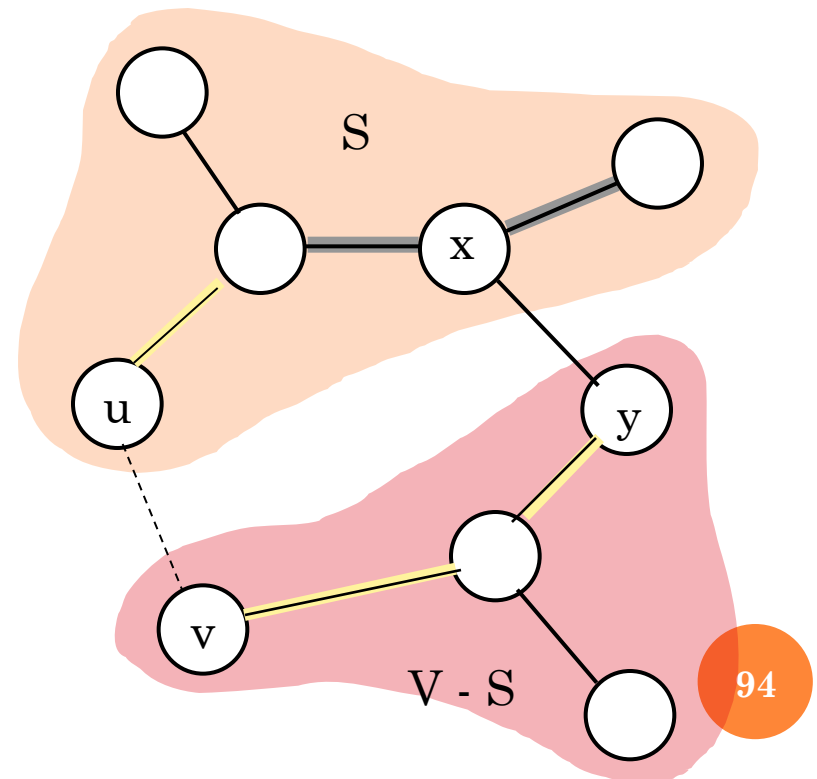
Running time: $O(V + E \lg E + E \lg V) = O(E \lg E)$ –
dependent on the implementation of the
disjoint-set data structure

KRUSKAL(V, E, w) (CONT.)

1. $A \leftarrow \emptyset$
 2. **for** each vertex $v \in V$
 3. **do** MAKE-SET(v)
 4. sort E into non-decreasing order by w
 5. **for** each (u, v) taken from the sorted list
 6. **do if** FIND-SET(u) $\neq$ FIND-SET(v)
 7. **then** $A \leftarrow A \cup \{(u, v)\}$
 8. UNION(u, v)
 9. **return** A
- Running time: $O(V + ElgE + ElgV) = O(ElgE)$
- Since $E = O(V^2)$, we have $lgE = O(2lgV) = O(lgV)$
- $O(ElgV)$**

KRUSKAL'S ALGORITHM

- Kruskal's algorithm is a “**greedy**” algorithm
- Kruskal's greedy strategy produces a globally optimum solution
- Proof for generic approach applies to Kruskal's algorithm too



KRUSKAL'S ALGORITHM VS PRIM'S ALGORITHM

- In **prim's algorithm**, graph must be a **connected**
- **Kruskal's** algorithm can function on **disconnected** graphs too
- **Prim's algorithm** is significantly faster for **dense graphs** with more number of edges than vertices
- **Kruskal's algorithm** runs faster in the case of **sparse graphs**

PROBLEM 1

- **(Exercise 23.2-3, page 637)** Compare Prim's algorithm with and Kruskal's algorithm assuming:

(a) sparse graphs:

In this case, $E=O(V)$

Kruskal:

$$O(E \lg E) = O(V \lg V)$$

Prim:

- binary heap: $O(E \lg V) = O(V \lg V)$
- Fibonacci heap: $O(V \lg V + E) = O(V \lg V)$

PROBLEM 1 (CONT.)

(b) dense graphs

In this case, $E=O(V^2)$

Kruskal:

$$O(E \lg E) = O(V^2 \lg V^2) = O(2V^2 \lg V) = O(V^2 \lg V)$$

Prim:

- binary heap: $O(E \lg V) = O(V^2 \lg V)$
- Fibonacci heap: $O(V \lg V + E) = O(V \lg V + V^2) = O(V^2)$

PROBLEM 2

- Suppose that some of the weights in a connected graph G are negative. Will Prim's algorithm still work? What about Kruskal's algorithm? Justify your answers.
 - Yes, both algorithms will work with negative weights. Review the proof of the generic approach; there is no assumption in the proof about the weights being positive.

PROBLEM 3

- Find an algorithm for the “maximum” spanning tree. That is, given an undirected weighted graph G , find a spanning tree of G of maximum cost. Prove the correctness of your algorithm.
 - Consider choosing the “heaviest” edge (i.e., the edge associated with the largest weight) in a cut. The generic proof can be modified easily to show that this approach will work.
 - Alternatively, multiply the weights by -1 and apply either Prim’s or Kruskal’s algorithms without any modification at all!

MST ALGORITHMS

Kruskal's algorithm (see CLRS):

- Uses the *disjoint-set data structure* (see CLRS, Ch. 21).
- Running time = $O(E \lg V)$.

Best to date:

- Karger, Klein, and Tarjan [1993].
- Randomized algorithm.
- $O(V + E)$ expected time.

REFERENCE

○ Introduction to Algorithms

- Minimum Spanning Tree
- Chapter # 23
- Thomas H. Cormen

- Disjoint Set
- <http://www.csl.mtu.edu/cs4321/www/Lectures/Lecture%2019%20-%20Kruskal%20Algorithm%20and%20Dis-joint%20Sets.htm>

SUMMARY OF GRAPH ALGORITHMS

BREADTH-FIRST SEARCH

```
BFS( $G, s$ )
1 for each vertex  $u \in V[G] - \{s\}$ 
2   do  $color[u] \leftarrow \text{WHITE}$ 
3      $d[u] \leftarrow \infty$ 
4      $\pi[u] \leftarrow \text{NIL}$ 
5  $color[s] \leftarrow \text{GRAY}$ 
6  $d[s] \leftarrow 0$ 
7  $\pi[s] \leftarrow \text{NIL}$ 
8  $Q \leftarrow \emptyset$ 
9 ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11   do  $u \leftarrow \text{DEQUEUE}(Q)$ 
12     for each  $v \in Adj[u]$ 
13       do if  $color[v] = \text{WHITE}$ 
14         then  $color[v] \leftarrow \text{GRAY}$ 
15            $d[v] \leftarrow d[u] + 1$ 
16            $\pi[v] \leftarrow u$ 
17           ENQUEUE( $Q, v$ )
18  $color[u] \leftarrow \text{BLACK}$ 
```

DEPTH-FIRST SEARCH

DFS(G)

```
1 for each vertex  $u \in G.V$ 
2    $u.color = \text{WHITE}$ 
3    $u.\pi = \text{NIL}$ 
4  $time = 0$ 
5 for each vertex  $u \in G.V$ 
6   if  $u.color == \text{WHITE}$ 
7     DFS-VISIT( $G, u$ )
```

DFS-VISIT(G, u)

```
1  $time = time + 1$  // white
2  $u.d = time$ 
3  $u.color = \text{GRAY}$ 
4 for each  $v \in G.Adj[u]$  // explore
5   if  $v.color == \text{WHITE}$ 
6      $v.\pi = u$ 
7     DFS-VISIT( $G, v$ )
8  $u.color = \text{BLACK}$  // black
9  $time = time + 1$ 
10  $u.f = time$ 
```

ANALYSIS OF PRIM

$\Theta(V)$ total {
 $Q \leftarrow V$
 $key[v] \leftarrow \infty$ for all $v \in V$
 $key[s] \leftarrow 0$ for some arbitrary $s \in V$
while $Q \neq \emptyset$
 do $u \leftarrow \text{EXTRACT-MIN}(Q)$
 for each $v \in \text{Adj}[u]$
 do if $v \in Q$ and $w(u, v) < key[v]$
 then $key[v] \leftarrow w(u, v)$
 $\pi[v] \leftarrow u$

$|V|$ times {
 $degree(u)$ times {



Handshaking Lemma $\Rightarrow \Theta(E)$ implicit DECREASE-KEY's.

$$\text{Time} = \Theta(V) \cdot T_{\text{EXTRACT-MIN}} + \Theta(E) \cdot T_{\text{DECREASE-KEY}}$$

KRUSKAL(V, E, w) (CONT.)

1. $A \leftarrow \emptyset$
 2. **for** each vertex $v \in V$
 3. **do** MAKE-SET(v)
 4. sort E into non-decreasing order by w
 5. **for** each (u, v) taken from the sorted list
 6. **do if** FIND-SET(u) $\neq$ FIND-SET(v)
 7. **then** $A \leftarrow A \cup \{(u, v)\}$
 8. UNION(u, v)
 9. **return** A
- Running time: $O(V + ElgE + ElgV) = O(ElgE)$
- Since $E = O(V^2)$, we have $lgE = O(2lgV) = O(lgV)$
- $O(ElgV)$**

SHORTEST PATH